

University of Padova

Department of Information Engineering

Master Thesis in Big Data Management and Analytics

Data-Aware Process Simulation in a Physically Constrained Setting: A Container Terminal Truck Operations Case Study

Supervisor

Prof. Massimiliano de Leoni
University of Padova

Co-supervisors

Marcel Petersen
HHLA - Port of Hamburg

Prof. Esteban Zimányi
Université Libre de Bruxelles

Master Candidate

Jule Grigat

Student ID

2185651

Academic Year

2025-2026

Abstract

Container terminals use simulation to evaluate operational changes without disrupting live operations, but conventional models are commonly constructed and calibrated manually. Data-aware process simulation instead derives executable models from event logs. Its validity in physical logistics is still under-evaluated.

This thesis examines ProSiT as a data-aware Process Simulation framework for truck operations at HHLA Container Terminal Burchardkai. Two months of operational data were transformed into an event log of 89,460 truck visits and divided chronologically into discovery, validation and untouched test datasets. Preserving the ITS schedule-derived stop order prevents a timestamp-induced parallelism artefact, but the raw Inductive-Miner model still violates known truck semantics through parallel behavior and a gate-only bypass. A transparent domain repair therefore enforces sequential yard handling while retaining alternative orders and repetitions. The resulting Petri net fits all 17,892 held-out visits with fitness 1.0 and test precision 0.7641 and serves as the common executable control-flow model.

A 2×2 validation experiment separates manually constructed static demand and utilisation proxies from ProSiT's native workload and queue state. No added-state configuration provides a resolved multi-perspective improvement. The parsimonious *visit-only* configuration is therefore rediscovered on the complete 80 % development period and evaluated once on the final test period. It reproduces inter-arrival spacing closely and parts of yard service timing, but underestimates mean turnaround by 5.930 minutes (33.812 vs. 39.742) and P90 by 13.100 minutes (57.900 vs. 71.000). The discrepancy is stable across Monte-Carlo seeds. Physical validity is audited separately. Truck activities must remain sequential and no more than three RMGs may operate concurrently in one block; the latter upper bound is imposed transparently because overlap-derived capacities exceed the physical maximum. Historical fidelity alone selects the workload-blind *visit-only* benchmark, which lacks an endogenous pathway for demand propagation. A mechanism-focused audit therefore evaluates the minimal *native-only* configuration with a dynamic workload state. Closing block T22 and increasing working-time demand by 20 % produce no resolved mean-turnaround effect, while a matched six-level stress test produces a non-linear delay response, only at substantially higher realised arrival rates. This reflects an internal response pathway in the fitted simulator, not a validated physical CTB congestion response. The thesis contributes a layered validity audit, separating event order, control-flow language, historical fidelity, physical semantics, and intervention response. The case shows that good historical fit is an important first step, but is not enough to show that the model is reliable for what-if analysis. For direct decision support, the model would need a more explicit representation of location, movement, equipment, and queues, as well as further validation with future operational data.

Contents

Abstract **iii**

List of figures **ix**

List of tables **xiii**

Listing of acronyms **xix**

1 Introduction **1**
- 1.1 Motivation . . . 1
- 1.2 Problem Statement . . . 2
- 1.3 Research Objective and Contributions . . . 3
- 1.4 Research Questions . . . 4
- 1.5 Thesis Outline . . . 4

2 Background and Related Work **6**
- 2.1 Container Terminal Truck Operations . . . 6
- 2.2 Event Logs . . . 8
- 2.3 Process Discovery and Petri Nets . . . 9
- 2.4 Conformance Checking . . . 13
- 2.5 Business Process Simulation . . . 16
- 2.6 ProSiT . . . 17
- 2.7 Research Gap and Positioning . . . 23

3 Data Preparation **25**
- 3.1 Case Study Environment . . . 25
- 3.2 Event-Log Construction . . . 27
- 3.3 Event Log Overview . . . 31
- 3.4 Domain Rules . . . 33
- 3.5 Chapter Summary . . . 33

4 Model Construction and Simulation **35**
- 4.1 From Event Log to Executable Model . . . 35
- 4.2 Control Flow . . . 36
- 4.3 Learned Components . . . 38
- 4.4 Execution of a Simulated Visit . . . 42

4.5 Adaptations to the Terminal . . . 43
4.6 Chapter Summary . . . 43

5 Methodology and Experimental Setup 45
5.1 Overview of the Experiments . . . 45
5.2 Data Partitions and Their Use . . . 49
5.3 Outcome Measures and Peak Definitions . . . 49
5.4 Simulation Runs, Replications and Uncertainty . . . 50
5.5 Domain Review . . . 50
5.6 Control-Flow Discovery (E1) . . . 51
5.7 Domain-Rule Audit and Targeted Repair (E2) . . . 55
5.8 Baseline Simulation of Control-Flow Candidates (E3) . . . 59
5.9 Mechanism Audit and Simulation Validity (E4) . . . 62
5.10 What-If Demonstration (E5) . . . 69
5.11 Threats Addressed by the Design . . . 71
5.12 Chapter Summary . . . 72

6 Results 73
6.1 Discovery Comparison and Model Choice (E1) . . . 73
6.2 Domain-Rule Audit and Repair (E2) . . . 83
6.3 Control-Flow Candidates in the Simulation (E3) . . . 89
6.4 Answer to RQ1 . . . 93
6.5 Mechanism Audit and Simulation Validity (E4) . . . 98
6.6 Answer to RQ2 . . . 109
6.7 Load-Aware Block Allocation (E5) . . . 110
6.8 Chapter Summary . . . 111

7 Discussion 112
7.1 Adequacy for the Intended Use . . . 112
7.2 Discovery Metrics and Simulation Behaviour . . . 112
7.3 Data-Aware Simulation in a Physical Logistics Process . . . 113
7.4 Threats to Validity . . . 113
7.5 Domain Knowledge and Stakeholder Review . . . 113
7.6 Implications for Practice . . . 114

8 Conclusion 115
8.1 Answers to the Research Questions . . . 115
8.2 Contributions . . . 116
8.3 Limitations . . . 116
8.4 Future Work . . . 116

A Observed Truck-Visit Behaviour 118
A.1 Arrival Pattern . . . 118
A.2 Congestion over the Day . . . 119
A.3 Delay Peaks . . . 121
A.4 Disruption Episodes . . . 122
A.5 Differences between the Partitions . . . 122
A.6 Bottlenecks at the Handover . . . 123

B Conformance of All Discovered Nets 125

C ProSiT Model Inventory 127
C.1 Discovery Settings and Versions . . . 127
C.2 Learned Decision Trees . . . 127
C.3 Calendars and Capacities . . . 128
C.4 Rule Register and Unobserved Sequences . . . 128
C.5 Discovered Models . . . 128

D Model Inspection and Reproduction 129
D.1 Unfiltered Training-Log Control Flow . . . 129
D.2 Executable Model State . . . 131

References 132

Listing of figures

2.1 A workflow net with a silent transition. Circles are places, white boxes visible transitions, and the black box a silent transition τ. The token in the source place i is a truck before the gate. The silent transition skips the yard stop, so the net also accepts a visit without any stop. 10

2.2 A real ProSiT tree: the arrival model learned from the CTB training partition (ProSiT 1.0.4, Section 5.8). The tree predicts the gap until the next truck arrives, in open minutes of the arrival calendar, from the hour and weekday of the previous arrival. Each leaf holds a fitted distribution. The simulator follows the conditions from the top to one leaf and draws the gap from that leaf. The means are those of the values the simulator draws. Sunday is closed in the arrival calendar, so no gaps are drawn there. 20

4.1 Steps of one simulated visit in the final model and where each value comes from. Blue: learned by ProSiT from the training log. Orange border: adaptation to the terminal (Section 4.5). Grey: structure and output. 36

4.2 Control flow of the simulation model: IM 0.0 restricted to rules 3 and 5 as a state machine (10 places, 54 transitions, no silent transitions). The labels show the stop guard of the simulator: n is the number of planned stops from the manifest and c the number of stops done; a yard transition may fire only if $c < n$, Gate Out only if $c = n$. 37

4.3 Arrival tree: minutes to the next Gate In by weekday (0 = Monday) and hour of the last arrival. Each leaf shows the training values (bars) and the fitted distribution (line), each on its own scale. 40

4.4 Routing tree of the transition `RMG_receive` at the first decision after Gate In: probability that it fires when enabled, by manifest attributes. 40

4.5 First stop of a visit by flow type: share in the training log and probability under the learned routing (normalised weights of the enabled transitions, averaged over 8,000 training manifests). 41

4.6 Waiting-time tree of block T13 (final model): minutes from a free lane to the start of the stop. All leaves on one scale. 41

4.7 Execution-time tree of RMG receives (final model). The first splits are on the number of other trucks at the block when the stop starts. 42

5.1 Flow of the experiments E1 to E3. Each stage reduces the set of candidate models and passes it on. E4 and E5 continue with the configuration selected in E3. 46

5.2 Repair pattern for rule 5: one exclusive choice across two parts of the net. The first choice leaves a token that records the decision (dashed places and arcs). `LL_delivery` can only fire if `LL_mixed` was skipped. The skip of the second choice exists twice, once for each token, so no control token remains at the end. `LL_receive` is connected like `LL_delivery`, and HO2 is repaired in the same way. Per handling area, the pattern adds two places and one silent transition. 58

5.3 Marker logic for rule 3 (at least one stop), shown for two optional stops in parallel. Gate In puts a token into *no stop yet*. Each stop exists twice: the first stop moves the token to *stop done*, a further stop only reads it (double arrow). Gate Out needs *stop done*, so a visit cannot leave without a stop. If both stops are skipped, however, the token stays in *no stop yet* and the visit can never finish. The complete construction therefore also forbids skipping the last open stop, with one more control place per stop (not shown). 58

6.1 Share of fitting visits and precision of the discovered nets on the training log, which is the basis for the selection. Left: all nets. Right: the nets with more than 91 % fitting visits and a precision above 0.7. Hollow markers: unsound nets, values for diagnosis only. The prefix tree accepts exactly the training sequences and serves as a reference. 74

6.2 The same nets on the test log (13 to 30 April 2026). The test log was not used for any decision. Precision hardly changes. IM 0.0 and ILP 1.00 still fit (almost) all visits, while the nets that accept few sequences lose more fitting visits than on the training log (IMf 0.3: 96.5 % → 94.9 %). 74

6.3 Which training sequences each sound candidate accepts. Top: share of the training visits per sequence (logarithmic scale), from the most to the least frequent sequence. Middle: number of stops of each sequence. Bottom: one row per net; a coloured cell means that the net accepts the complete sequence. The six most frequent sequences cover 88.5 % of the visits, so a net that accepts few sequences can still fit most visits. 76

6.4 Fitness–precision trade-off of the discovered nets on the validation partition. Left: all nets with an alignment; the dashed rectangle marks the zoomed region. Right: the candidates with more than 91 % fitting visits. Boxes summarise soundness, size and the coverage measures of Table 6.5. Hollow grey markers: the unsound Alpha nets. The prefix tree is a reference, not a candidate. 80

6.5 Selected net IM 0.0 (Inductive Miner without filtering, discovered in ProM on the full training partition): 28 places, 11 visible and 27 silent transitions (black). The silent AND-split after `Gate In` lets the branches occur in any order. Every branch can be skipped. 82
6.6 Repaired control-flow model `IM0_R5_R12` as a state machine (10 places, 54 transitions). Each place is a state of the visit. Repeated labels belong to different states. 87
6.7 Turnaround time, observed and simulated (final model; replayed visits and observed blocks; five seeds pooled). 105
6.8 Median turnaround time per Gate-In hour over the test period, observed and simulated with the same trucks (seed 42). 105
6.9 Load profiles, observed and simulated, in the test period (solid) and the training period (dotted); medians, means over five seeds. With three lanes, the states "4+ trucks at the block" and "3+ other trucks" cannot occur in the simulation. 106
6.10 Where the time of a visit is spent, observed and simulated (means). 106

A.1 Hourly arrivals (top) and median turnaround time by Gate-In hour (bottom) for the training and validation periods. Dots mark arrival peaks and delay peaks. Dashed lines show the thresholds of 165 arrivals and 37 minutes. The vertical line marks the start of the validation period. 119
A.2 Mean turnaround time by Gate-In hour, split into approach, stop intervals, transfer and exit. Training weekdays. 120
A.3 Median and P90 of the turnaround time by the number of trucks in the terminal at Gate In, in classes of 20. Training, classes with at least 50 visits. 120
A.4 Median stop interval by the number of trucks with an open interval at the same handling unit when the stop starts. Training. 121

D.1 Complete frequency directly-follows graph of the order-preserving CTB training log. Node labels report activity occurrences and arc labels report directly-follows counts. All 59 observed arcs are retained. The figure is a descriptive audit view and is not the executable simulation model. 130

Listing of tables

2.1 Simplified event log with two truck visits. Case attributes such as the number of planned stops are stored once per case and are not shown. 9

2.2 Components of a ProSiT simulation model (version 1.0.4). Times are counted in open minutes of the relevant calendar. 19

2.3 Discovery hyperparameters of ProSiT 1.0.4. 22

2.4 Research directions and the questions they leave open for the CTB case study. 24

3.1 Operational data sources used for event-log engineering. 27

3.2 Visit structure and turnaround time (TTT) by partition. Approach is the time from Gate In to the start of the first stop, exit the time from the end of the last stop to Gate Out, and transfer the time between stops. Because gate events are instantaneous, the four components add up to the TTT. . . . 31

3.3 Domain rules for truck visits. Source: domain knowledge of the author; confirmation by the terminal pending. The type decides where a rule can be enforced (Section 5.7). 34

4.1 Learned components of the final simulation model (ProSiT 1.0.4 on the repaired IM 0.0 state machine, training log). Waiting and execution time are the retrained components of the final configuration (Section 5.9.3); the inventory of every model is in Appendix C. 39

4.2 Discovery settings of the simulation model. 39

4.3 Capacities and calendars. The discovered resource calendars cover 122 to 128 of the 168 hours of a week; they are set to the full week so that trucks already inside are served to completion. The gate calendar still controls admission. . . 42

4.4 One simulated truck of the final model (configuration of E4, seed 42), step by step. The leaf of each tree follows from the features at that moment; the waiting and execution times are drawn from the distribution of that leaf; the waiting time includes any delay until a lane is free (capacity checked over the whole stop at its start). 43

4.5 Differences between the ProSiT simulation loop and the final model. 44

5.1 Experiments, varied factors and the research questions they address. 47

5.2 Visits per partition for the restricted and the expanded population. Partitions are chronological by Gate In. The validation period starts on 10 April 2026, 15:50, and the test period on 20 April 2026, 18:17. The 184 visits with unknown activity labels remain excluded. 52

5.3 Discovery configurations of E1, all run in ProM Lite 1.4 on the training partition of the expanded population (58,262 visits). The iDHM frequency threshold is relative to the number of visits: at 0.1, only the 7 of 60 directly-follows relations that occur at least 5,827 times are used. Because no data attributes were selected, iDHM discovers no conditional dependencies, and the conditional heuristic (C4.5 with F1 score, condition threshold 0.5) has no effect. . . 53

5.4 2×2 contrast for IM 0.0 in E3. All four cells use the same ProSiT settings and seeds. 59

5.5 Fixed simulation conditions in E3. 61

5.6 Measures of operational validity in E4. For each load profile in P4, the *profile gap* is the mean absolute difference of the simulated medians to the observed ones over the bins. 64

5.7 Mechanisms examined in E4. Observed values from the test log; the training log gives the same profiles within one to two minutes. 66

5.8 Configurations of E4. ProSiT: component as fitted in E3. No self-count: retrained without counting the stop in its own queue. Lane free: waiting measured from the moment a lane became free. Whole delay: waiting measured from enabling to start. Every configuration from b3 on, and the baseline a0, is also run with an exact count of the activities in progress (suffix x, see the design decisions). 68

6.1 Nets taken to the rule audit and repair (E2), with their values on the training log. Selection on training values only: sound, at least 90 % fitting visits and not dominated in fitting visits and precision. HM 0.8 is also non-dominated but lies between HM 0.7 and HM 0.9 and is left out. ILP 1.00 and IMf 0.3 are added as references for a permissive and a filtered net. 75

6.2 What the rare sequences and the sequences a net rejects stand for (training log, 59,139 visits, 82 sequences). Truck-minutes: sum of the turnaround times. Visits > P90: visits with a turnaround time above the training P90 of 64 minutes. Multi-block: visits to two or more RMG blocks (1.75 % of the visits). Mean TTT in minutes (all visits: 37). Again in test: sequences that occur again in the test log. 75

6.3 ProM replay of the validation partition (14,625 visits) on the five Inductive Miner nets. PM4Py returns the same number of fitting visits and the same total deviation cost (log plus model moves) for every net. Mean trace fitness as reported by ProM. The last column adds the PM4Py precision: the filtered nets reject 5.0 to 7.5 % of the visits for at most 0.045 more precision, and IMf 0.4 rejects more than a third. 77

6.4 Discovered nets on the training partition (58,262 visits) and the validation partition (14,625 visits). Fitting visits: share of visits whose alignment has no deviation. Fitness: alignment-based log fitness. IM: Inductive Miner, IMf with the noise threshold given; HM: iDHM with the dependency threshold given; ILP: ILP Miner with the filter threshold given (0.00 and 0.25 yield the same net). [a]Unsound, values reported as a diagnostic only. Alpha++ has no reachable final marking, so no alignment exists. [b]Reference net that accepts exactly the training sequences (Section 5.6.2). 78

6.5 Behaviour preserved by the sound nets and the prefix-tree reference. A sequence is accepted if the net replays it without deviation. Multi-block visits handle containers at more than one RMG block (1,008 training and 311 validation visits). 81

6.6 Rule audit and exact repair for rules 3 and 5 (E2). Rules violated: rules whose violation the language of the net permits, before → after the repair (6 (bound): more than four stops; enforced in the simulator by the stop guard, not in the net). Fitting visits (train / test) are the same before and after the repair. Same language: the unrepaired language written as a state machine, the control for the change of encoding. Sequences with 4 stops: distinct sequences the net permits; 5 occur in training. 83

6.7 Rule audit of the model languages. "Unbounded" means that loops allow arbitrarily many yard events. 84

6.8 Number of distinct sequences permitted by each net, by number of yard events. IM0 additionally permits the empty visit. 85

6.9 Repaired variants of IM0 on the expanded population. The control encodes the unrestricted IM0 language as a state machine. 86

6.10 The same rule restriction (R5, R12) applied to every sound base model. Permitted sequences are counted after the restriction. Validation visits accepted refers to the restricted model. 88

6.11 The four repaired nets taken to E3, ordered by precision. Left: conformance on the training log (E1, E2). Right: simulated visits in the test period with stop guard, means over five seeds. TVD: total-variation distance to the observed test log (0 = identical). RMG visits to 2 blocks: share of the visits with an RMG stop that use two different blocks. The turnaround time is far too long for every net; the cause lies outside the control flow (Section 6.5). 89

6.12 Effect of the rule treatment on the simulated visits (E3), test period, means over five seeds. Original net: IM 0.0 as discovered (parallel branches that silent transitions can skip). Repaired net: language restricted to rules 3 and 5, written as a state machine. Guard: data-aware stop guard for rule 6. Control: the unrepaired language written as a state machine. Overlapping stops: share of the visits with more than one stop. Unseen sequences: generated visits whose sequence never occurs in the training log. TVD: total-variation distance to the distribution of the observed test log (0 = identical). 90

6.13 Generated visits of the three control-flow candidates compared with the validation log. Means over five replications of 14,625 visits. Approach is Gate In to the start of the first stop, exit is the end of the last stop to Gate Out. 91

6.14 ProSiT's waiting-time target at the RMG blocks by the load of the block when the stop is enabled, training log. The target is measured from the moment the block was last completely free (single server). 99

6.15 Error attribution along the path of the truck (E4a–E4d), against the test log, means over five seeds. From b3x on, the observed demand and the observed blocks are used, so that only the queue and the service vary. The rows in italics are differences between real periods. Med., P90: median and P90 of the TTT; CTD: cycle-time distance; Appr., Stops: mean approach and stop time per visit (all in minutes). Top bl.: share of the RMG stops at the busiest block in %. Max: largest number of trucks at one block at once. Spear.: Spearman correlation of hourly arrivals and hourly median TTT. G-appr., G-stop, G-WIP: profile gaps for the approach by trucks at the block, the RMG stop duration by other trucks at the block and the TTT by trucks in the terminal (minutes). 100

6.16 Choice of the final configuration on the training period (E4f): replayed training visits with their observed blocks, compared with the training log, means over five seeds. Admissible: no rule violation and at most three trucks at a block at once. The configuration with the smallest sum of the profile gaps is chosen (←). Gaps and CTD in minutes. 102

6.17 Final configuration with each change reversed alone, with other block allocations, other arrival intensities and without replay (E4e), against the test log, means over five seeds. *Only the 2,393 visits of the first day, because the queue of the learned block choice grows without bound. Med., P90: median and P90 of the TTT; CTD: cycle-time distance; Appr., Stops: mean approach and stop time per visit (all in minutes). Top bl.: share of the RMG stops at the busiest block in %. Max: largest number of trucks at one block at once. Spear.: Spearman correlation of hourly arrivals and hourly median TTT. G-appr., G-stop, G-WIP: profile gaps for the approach by trucks at the block, the RMG stop duration by other trucks at the block and the TTT by trucks in the terminal (minutes). 103

6.18 Key values of the final simulation model against the observed logs (replayed visits and observed blocks, means over five seeds). Times in minutes; CTD: cycle-time distance; Spearman: hourly arrivals against hourly median TTT; Max at block: largest number of trucks at one block at once (the log records 4 because of short recording overlaps, Section 5.9.3). 105
6.19 Load-response profiles against the test log: medians in minutes, means over five seeds. –: fewer than 50 cases in the bin. With three lanes, the bins "4+ trucks at the block" and "3+ other trucks" cannot occur; in the log they are short recording overlaps. 107
6.20 Error of the mean turnaround time (simulated minus observed, minutes) and its parts; the four parts add up to the TTT error up to rounding. The stop error is split into a count part (stops per visit by area, i.e. routing) and a duration part (mean stop duration by area). Last row: training log minus test log. . . . 107
6.21 Acceptance criteria of Section 5.9.4 for the final configuration. Thresholds from the differences between real periods: Spearman of the weakest test week; profile gaps (approach / stop duration / TTT by trucks in the terminal, minutes) of the largest difference between weeks; CTD within the differences between weeks (3.1 minutes) or between periods (5.5 minutes). 108

A.1 Turnaround-time components in and outside peak hours. Visits are assigned to the hour of their Gate In. 122
A.2 Observed visits per partition. Delay-peak threshold (37 minutes) fixed on the training period. 123

B.1 Conformance of all discovered nets on the training log (2 March to 11 April 2026) and the test log (13 to 30 April 2026). Fitting visits: share of visits whose complete sequence the net accepts. Precision: escaping-edges precision on alignments with exact silent closure. Seq. accepted: training sequences accepted, of 82. Unseen test seq.: test sequences that do not occur in training and are accepted, of 16. Gen.: token-based generalisation on the training log; it is reported for completeness but not used for the selection, because it rewards nets that use few transitions often and barely separates the nets. Values of unsound nets are diagnostic only. The prefix tree is a reference that accepts exactly the training sequences. 126

Listing of acronyms

BPMN . . . . . Business Process Model and Notation

CHE . . . . . Container Handling Equipment

CRM . . . . . Customer Relationship Management

CTB . . . . . HHLA Container Terminal Burchardkai

DES . . . . . Discrete Event Simulation

DFG . . . . . Directly-Follows Graph

ERP . . . . . Enterprise Resource Planning

GOS . . . . . Gate Operating System

HHLA . . . . . Hamburger Hafen und Logistik AG

HO2 . . . . . Manual yard handling area

KPI . . . . . Key Performance Indicator

LL . . . . . Leerlager (empty-container yard)

MES . . . . . Manufacturing Execution System

OCR . . . . . Optical Character Recognition

PM . . . . . Process Mining

RMG . . . . . Rail-Mounted Gantry Crane

TAS . . . . . Truck Appointment System

TCS . . . . . Terminal Control System

TEU . . . . . Twenty-foot Equivalent Unit

TOS . . . . . Terminal Operating System

TTT . . . . . Truck Turnaround Time

VC . . . . . Van Carrier

WfMS . . . . . Workflow Management System

XES . . . . . eXtensible Event Stream

YMS . . . . . Yard Management System

1 Introduction

Container terminals connect maritime and hinterland transport networks. Higher container volumes and larger vessels require terminal operators to make many tactical and operational decisions under uncertainty. These decisions concern truck dispatching, yard-block allocation, resource use, congestion, equipment failures, and temporary infrastructure outages. Experiments in an operating terminal are often costly, disruptive, or not feasible. Simulation offers a way to assess decisions without interrupting terminal operations.

1.1 Motivation

To explain:

- Why terminals need decision support for truck handling, and why live experiments are not an option (one paragraph).
- Why simulation, and why building it from event logs (process mining) instead of by hand.
- What data-aware, white-box simulation adds (ProSiT): inspectable rules instead of black boxes.
- The gap: evidence comes mainly from business processes; physical logistics has capacity, congestion and space. Keep to about 1.5 pages in total.

Simulation is used for decision support in container-terminal planning and operations research. Its operational purpose is prospective: a terminal can compare how alternative process executions and operating conditions affect specific handling patterns before changing live op-

erations. For example, when high yard utilisation is forecast, a terminal may compare lower truck-slot release rates, alternative resource allocations, or policies that favour receive or delivery operations, and evaluate their effects on activity-specific service, pre-service delay, and total truck turnaround. Simulation can likewise assess infrastructure investments, scheduling approaches, truck appointment systems, and terminal redesigns before implementation. Traditional approaches mostly use manually developed Discrete Event Simulation (DES) models, agent-based models, or digital twins. Building and maintaining these models requires domain knowledge, manual work, and calibration [1, 2, 3, 4].

Process Mining research has studied how simulation models can be built from event logs. Earlier work showed that control-flow behaviour, activity durations, arrival patterns, resource allocations, and routing probabilities can be discovered from process data and used in executable simulation models [5]. *Data-aware process simulation* extends this idea. It uses context attributes, case information, process states, resource characteristics, and decision logic [6, 7, 8, 9].

Machine-learning and deep-learning methods can learn complicated behavior, but their results are often difficult to inspect, modify, and validate. Hybrid and white-box approaches combine interpretable models with data-driven behaviour discovery [10, 11]. ProSiT uses probabilistic white-box predictors to build simulation models that can be inspected and configured for what-if analysis [11].

Evidence on automatically discovered simulation models in logistics systems is still limited. Within the literature reviewed for this thesis, this intersection remains under-evaluated, with most studies still centred on business processes in healthcare, financial services, and administration [12, 8]. Container terminal truck operations involve physical resource constraints, congestion, infrastructure dependencies, spatial interactions, and changing process contexts. It is not yet clear how well a discovered data-aware simulation model can reproduce these conditions and support decisions. This thesis studies its fidelity, structural applicability, and limitations using event logs from real container terminal truck operations.

1.2 Problem Statement

To explain:

- The concrete operational problem at CTB: delay peaks in truck turnaround time. Use the observed numbers from Chapter 3 (10 % of training hours, 26 % of validation hours are delay peaks).

- The KPI: truck turnaround time (median, P90) and share of delay-peak hours. Say where the KPI comes from (HHLA practice, terminal contact).
- Why a simulator is needed to evaluate a procedural change against this KPI.
- The scientific problem: a discovered simulator can reproduce the visit sequences without reproducing the KPI and its response to load. Nobody has checked this layer by layer for a physical logistics process.
- Remove "TBC".

Container terminal operators make decisions about capacity, truck handling, yard use, policies, and infrastructure changes. Experiments in a real terminal are often not feasible or involve operational risk, so these decisions are often studied with simulation. TBC [2, 3].

Data-aware process simulation discovers simulation models from historical event logs. This may reduce modelling work and keep the model closer to the recorded process [13, 9, 8]. Before such a model can support decisions, it must reproduce observed operations, remain configurable, produce stable results, and represent the mechanisms through which a proposed intervention can affect the relevant KPI. A model intended to recommend a truck-slot rate under forecast congestion must, for example, propagate changed arrivals through resource contention and must keep receive and delivery effects distinguishable. Earlier studies have shown that data-aware simulation can be applied to business processes [8, 12]. Its use in logistics systems with congestion, competing resources, and context-dependent behaviour is less clear. This thesis evaluates the model's fidelity, use, and limitations for container-terminal truck operations.

1.3 Research Objective and Contributions

To explain:

- Objective in one sentence: investigate whether, and under which conditions, a data-aware process simulation learned from event logs can represent a physically constrained logistics process, using truck handling at CTB as the case. The aim is a scientific statement about the technique, not a recommendation for the terminal.
- Contributions as a short list, each pointing to a chapter: (C1) controlled control-flow selection with rule audit and exact repair, and evidence that coverage and the encoding, not precision, anticipate the simulated behaviour (Ch. 6/7); (C2) a mechanism-level validation of a ProSiT model for a capacity-limited logistics process: which assumptions of the technique break under physical limits, why, and what is needed to repair them (Ch. 6/7); (C3) a demonstration of what-if use under physical constraints: what the validated simulation can and cannot show about one

allocation rule, with its conditions of validity (Ch. 7); (C4) a reproducible pipeline with frozen inputs and documented runs.

- Say explicitly that this is an empirical case study, not a new algorithm. The limits of the technique and the approaches to overcome them are part of the result (Chapter 7).

1.4 Research Questions

To achieve the research objective, the thesis addresses the following research questions:

1. **RQ1:** [TODO: final wording] Which discovery and rule treatment yields a sound, executable control-flow model that preserves the observed truck-visit behaviour, and do conformance metrics anticipate its behaviour in simulation?

2. **RQ2:** [TODO: final wording] To what extent does the ProSiT simulation reproduce truck turnaround time and its response to load at the capacity-limited yard blocks, and which model components cause the remaining error?

3. **RQ3:** [TODO: final wording] To what extent can the simulation be used to analyse a load-aware block allocation for deliveries under the physical limits of the yard blocks, and what does it indicate about the effect on truck turnaround time?

To explain:

- Each RQ is answered by named experiments: RQ1 by E1–E3, RQ2 by E3–E4, RQ3 by E5 (Chapter 5).
- Wording to be agreed with the supervisor. Keep it exactly identical in Chapters 1, 6 and 9.

1.5 Thesis Outline

To explain:

- One or two sentences per chapter, in the final order: 2 Background and Related Work, 3 Data Preparation, 4 Methodology and Experimental Setup, 5 Results, 6 Discussion, 7 Conclusion. Mention the appendices (observed behaviour, ProSiT parameters and trees, reproduction).
- There are three RQs, not four; update the descriptions of Results and Conclusion accordingly.

Chapter 2 introduces the concepts that the thesis uses: truck operations in a container terminal and their key performance indicator, event logs, process discovery with Petri nets, domain

rules and model repair, conformance checking, business process simulation and the simulation framework ProSiT. Related work is discussed together with each concept. The chapter ends with the research gap.

Chapter 3 introduces HHLA Container Terminal Burchardkai and its data. It explains how records from several terminal systems are turned into one truck-visit event log, gives the key facts of this log and states the domain rules that every truck visit must satisfy.

Chapter 5 describes the five experiments: control-flow discovery (E1), rule audit and repair (E2), baseline simulation (E3), mechanism audit (E4) and what-if demonstration (E5), together with the outcome measures and the domain review.

Chapter 6 presents the results of the experiments in the same order.

Chapter 8 brings together the empirical findings and answers the four research questions. It evaluates what the case study demonstrates about the transfer of data-aware process simulation to container-terminal truck operations

The chapter concludes by outlining directions for future work, particularly concerning richer spatial and resource-state representations, alternative event-log designs, and further validation in other operational settings.

2 Background and Related Work

To explain:

- Target length about 15 pages. Only concepts that later chapters use. Related work is placed in the section where its topic is introduced; Section 2.7 collects the gap.
- Each section ends with one or two sentences on what it means for this thesis.

This chapter introduces the concepts that the thesis uses and the research related to each of them. It follows the path of the thesis: the terminal process and its key performance indicator (Section 2.1), event logs (Section 2.2), process discovery with Petri nets (Section 2.3), conformance checking (Section 2.4), business process simulation (Section 2.5) and the simulation framework ProSiT (Section 2.6). Section 2.7 states the research gap.

2.1 Container Terminal Truck Operations

To explain:

- About two pages. One small figure of a truck visit (Gate In, stops at RMG block / LL / HO2, Gate Out) would help. CTB numbers (blocks, lanes, volumes) stay in Chapter 3.

A container terminal connects ships, trains and trucks. Containers are stored in a yard between the waterside and the landside. In many terminals, including the one studied here, the yard consists of storage blocks. Automated rail-mounted gantry cranes (RMG) move contain-

ers inside a block and hand them over to trucks at the landside end of the block. Terminals often have further areas, for example a depot for empty containers and a manually operated area. Trucks enter and leave through a gate [2].

The truck visit A truck visit starts when the truck passes the entry gate (*Gate In*). The truck then drives to one or more handling areas. At each *stop*, a container is delivered to the terminal, received from it, or both. A visit that delivers and receives in one trip is called a double cycle or *mixed* visit. After the last stop, the truck leaves through the exit gate (*Gate Out*). Most visits have one stop. Visits with two or more stops are less frequent but take longer.

Physical constraints A truck visit is a physical process. Several constraints follow from this and hold for every visit, whether or not they are visible in the recorded data.

- A truck is one vehicle. Its stops happen one after another and never at the same time.
- A visit exists to move containers. A visit without any stop between the gates does not occur in practice.
- A storage block has a limited number of handover lanes and cranes. Trucks share them with each other and with the waterside and rail operations.
- A truck can carry a limited number of containers.

These constraints run through the whole thesis. A model that is learned only from event frequencies does not know them. Section 2.3.3 shows how such constraints can be written down as rules, and Section 2.6.5 explains why the simulator needs them explicitly.

Key performance indicators Truck operations are measured with key performance indicators (KPIs). The *truck turnaround time* (TTT) is the time from Gate In to Gate Out. Waiting times isolate the delay before a gate or a yard stop, and utilisation measures describe how much of the yard and the equipment is in use. TTT is the main measure of service quality for truck handling, because it is the time that trucking companies and cargo owners experience. It is one of the four main KPIs of the HHLA container terminals [4, 14]. The mean alone hides the long waits that cause complaints. This thesis therefore also reports the 90th percentile (P90) of TTT and the number of hours in which TTT is unusually high (Section 5.3).

Related work: simulation in container terminals Terminal managers decide about truck appointments, equipment, staff and yard operations while the terminal is running. Because the processes are connected, a local change can affect waiting times and service elsewhere. Simulation allows such changes to be tested without disturbing live operations. Discrete-event

simulation (DES) is the most common approach, because the terminal can be described by events such as arrivals, gate passages and container moves [1, 15, 16, 17]. Studies use simulation to compare equipment numbers and operating policies [18], to generate scenarios [19], and to evaluate truck appointment systems and double cycles [14, 4]. These models are built by hand. This needs domain knowledge, much manual work, and recalibration when operations change. A hand-built model may also describe how the process is meant to work rather than how it actually runs [5].

Related work: process mining in logistics Process mining has been applied to logistics, mostly to discover and analyse processes rather than to build executable simulation models [20]. Reconstructing a logistics process from several information systems requires care, because timestamps and identifiers from different systems may describe different operational stages [21]. Grobis and Ihlenfeldt use event data to prepare a simulation model for production logistics and split the time between two activities into transport and waiting [22, 23]. In a hospital, Vinci et al. discover a simulation model from an event log, check it against observed behaviour and only then use it to compare resource configurations [12]. This thesis follows the same order.

Implication for this thesis An adequate simulation of the truck process must reproduce TTT, its tail and its growth under load. It must also respect the physical constraints listed above, because a policy that changes the load on the blocks is only credible if the simulated trucks behave like real trucks.

2.2 Event Logs

Process mining works on *event logs*. An event is one recorded occurrence of an activity. Each event belongs to a *case*, the object whose process is followed. In this thesis, one truck visit is one case. Each event records at least the case, the *activity* (for example `Gate In` or `RMG_receive`) and a timestamp. An event log groups the events by case and keeps their order. The ordered list of activities of a case is its *trace*. Cases with the same trace share a *variant*. The set of variants is the observed language of the log [24]. Table 2.1 shows two simplified truck visits.

Timestamps Simulation needs more than the order of events. When an event has a *start* and a *complete* timestamp, their difference is its execution time. A third point in time, *enablement*, is the moment at which the process model allows the activity to begin. It is not recorded. It is reconstructed by replaying the log on the process model [5, 13, 11]. The time between enablement and start is called waiting time. In a physical process, this interval can contain queueing,

Case	Activity	Start	Complete	Resource
T001	Gate In	09:13	09:13	Gate
T001	RMG_receive	09:24	09:31	Block T12
T001	Gate Out	09:42	09:42	Gate
T002	Gate In	09:17	09:17	Gate
T002	LL_delivery	09:30	09:38	LL
T002	RMG_receive	09:44	09:52	Block T08
T002	Gate Out	10:03	10:03	Gate

Table 2.1: Simplified event log with two truck visits. Case attributes such as the number of planned stops are stored once per case and are not shown.

driving and other delays. The meaning of each timestamp must therefore be checked. In the terminal data, a stop starts when the truck reports ready in the handover lane. The stop interval can already contain waiting for the crane (Section 3.2.3).

Attributes Further attributes describe the case and its context, for example the number of containers, the planned stops or the load of the terminal at Gate In. A data-aware simulation may only use an attribute at a decision point if its value is known at that point. Otherwise the model uses information from the future, which is called leakage (Section 3.2.6).

Event logs are exchanged between tools in the XES format [24]. In this thesis, the same log is read by ProM [25] and by PM4Py [26].

2.3 Process Discovery and Petri Nets

Process discovery builds a process model from an event log [24, 27]. The model shows how the process was executed, not how it was planned. ProSiT, the simulator used in this thesis, needs the model as a Petri net. This section introduces Petri nets, the discovery algorithms compared in E1, and the use of rules to restrict a discovered model.

2.3.1 Petri Nets, Workflow Nets and Silent Transitions

A Petri net is a graph of *places* (circles) and *transitions* (boxes), connected by arcs [28, 24]. Formally, a labelled Petri net is a tuple $N = (P, T, F, \lambda)$. P is a finite set of places, T a finite set of transitions, and $F \subseteq (P \times T) \cup (T \times P)$ the set of arcs. The labelling function $\lambda : T \to A \cup \{\tau\}$ gives each transition either an activity of the event log or the label τ. The places with an arc to

a transition t are its input places •t. The places with an arc from t are its output places t•.

The state of a net is a *marking*: a number of tokens in each place. A transition is *enabled* when each of its input places holds a token. Firing it removes one token from each input place and adds one token to each output place. A choice is a place with several outgoing arcs: only one of the enabled transitions can take the token. Parallelism is a transition with several output places: it creates one token per branch, and the branches can then fire independently.

A *workflow net* has one source place i and one sink place o, and every node lies on a path from i to o. A run starts with one token in i and should end with one token in o. For a truck visit, the token in i is the truck before the gate and the token in o the completed visit. A workflow net is *sound* if three conditions hold [24]. First, from every reachable marking the final marking can still be reached (option to complete). Second, when o is marked, no other place holds a token (proper completion). Third, every transition can fire in some run (no dead transitions). Soundness matters for simulation, because the simulator executes the net and must bring every case to the end.

Transitions labelled τ are *silent*. They fire like other transitions but leave no event in the log. They express routing that has no recorded counterpart: skipping an activity, repeating a loop, or splitting and joining parallel branches. The *language* of a workflow net is the set of visible activity sequences of all complete runs. The net in Figure 2.1 accepts both `Gate In, RMG_receive, Gate Out` and `Gate In, Gate Out`.

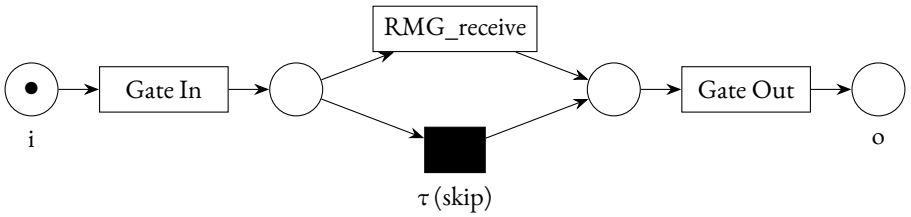


Figure 2.1: A workflow net with a silent transition. Circles are places, white boxes visible transitions, and the black box a silent transition τ. The token in the source place i is a truck before the gate. The silent transition skips the yard stop, so the net also accepts a visit without any stop.

Silent transitions are a means of representation. Two nets with very different numbers of silent transitions can have the same language. Discovery algorithms add them systematically. The Inductive Miner, for example, turns an optional activity into a silent skip and a parallel block into a silent split and join [29]. They have three consequences in this thesis. First, combinations of silent skips can admit behaviour that never occurs, such as the visit without a stop in Figure 2.1. Because silent paths leave no trace, this is easy to overlook. Second, alignments

(Section 2.4) must decide for every silent transition whether to fire it, which enlarges the search. Third, the simulator must decide them as well. It draws each step separately, without looking ahead (Section 2.6.3). A chain of likely skips can therefore produce a rare or impossible visit, and a silent parallel split can send one truck to two stops at the same time.

State machines A *state machine* is a Petri net in which every transition has exactly one input place and one output place [28]. A state machine workflow net therefore always holds exactly one token. Each place is a state of the process, and each transition a step from one state to the next. A state machine cannot express parallelism. For a truck, this is not a loss: its stops never run in parallel. Every finite set of sequences, and more generally every language accepted by a finite automaton, can be written as a sound state machine workflow net. Each automaton state becomes a place and each automaton edge a visible transition. This encoding is used for the repaired models in E2 (Section 5.7).

In such an encoding, the same activity label usually appears on several transitions, one for each state in which the activity can happen. These are called *duplicate labels*. The four discovery families compared in E1 create one visible transition per activity. For ProSiT the difference matters, because it learns one routing tree per transition (Section 2.6.1). With duplicate labels, the routing of an activity can therefore differ from state to state.

2.3.2 Discovery Algorithms

Discovery algorithms differ in how they turn observed sequences into a model. Experiment E1 compares four families.

- **Alpha family.** The Alpha Miner derives ordering relations (directly follows, causality, parallelism, choice) from the log and builds places from them [30]. Alpha+ and Alpha++ add short loops and non-local dependencies [31, 32]. The algorithms do not handle noise and do not guarantee a sound net.

- **Heuristics Miner.** The Heuristics Miner counts how often one activity follows another and keeps a dependency when its dependency measure exceeds a threshold [33, 34]. The interactive data-aware variant (iDHM) can also keep infrequent dependencies that depend on data, and converts the resulting causal net into a Petri net [35, 36]. The result is not guaranteed to be sound.

- **Inductive Miner.** The Inductive Miner splits the log recursively by sequence, choice, parallel and loop cuts. The result is a process tree, which is converted into a sound work-

flow net. The variant IMf first removes infrequent behaviour below a noise threshold [29].

- **ILP Miner.** Region-based miners add a place for every linear inequality that all prefixes of the observed traces satisfy. The ILP Miner finds these places with integer linear programming. A filter on the prefixes controls how much infrequent behaviour is kept [37].

Related work Benchmarks show that no algorithm is best on every log and every quality dimension [38]. Rehse et al. argue that discovery experiments need explicit assumptions and a controlled design: the same input, the same cohorts and the same implementation of each measure [39]. E1 follows this design. A further point concerns time. Two activities that occur in either order are not necessarily parallel; they may simply be ordered freely. Lifecycle-aware discovery uses start and complete timestamps to make this distinction [40]. For trucks it matters: two stops may occur in either order, but never at the same time.

2.3.3 Domain Rules and Model Repair

A discovered model generalises beyond the observed traces. Part of this generalisation can contradict knowledge that is known independently of the data, for example the physical constraints of Section 2.1. Such knowledge can be written down as rules.

Rules as automata Many rules on activity sequences can be written as Declare constraints, for example "Gate In occurs exactly once" or "mixed and delivery do not both occur at the same area" [41]. Every Declare constraint can be translated into a finite automaton that accepts exactly the sequences satisfying it [42, 43]. A sound workflow net has a finite reachability graph, so its language is also accepted by a finite automaton. Finite automata can be combined: the product of two automata accepts exactly the sequences that both accept. Running the net and the rule automata in one product therefore answers two questions exactly. Does the net accept a sequence that breaks a rule? And which sequences of the net satisfy all rules? The product follows the same idea as the synchronous product used for alignments [44]. A net combined with declarative constraints in this way corresponds to a hybrid process model [45].

Not every rule is a rule on sequences. "Stops never overlap" is a rule on timestamps. "The number of stops equals the number of planned stops" depends on a case attribute. Such rules cannot be written in the untimed language of a net. In data-aware Petri nets they become *guards*: conditions on data that a transition must satisfy before it may fire [46].

Related work: domain knowledge in discovery and repair Schuster et al. classify how domain knowledge enters process discovery [47]. Knowledge can be used before discovery, during discovery, or *a posteriori* on a discovered model. It can be explicit or come from user feedback, and it can be applied automatically or interactively. Explicit knowledge is written in one of two kinds of formalism. An *imperative* formalism, such as a Petri net, follows a closed-world assumption: only the behaviour it describes is allowed. A *declarative* formalism, such as Declare, follows an open-world assumption: everything is allowed that does not break one of its constraints [47]. This thesis uses both. The discovered net is imperative, the domain rules are declarative, and the repair keeps only the behaviour that both allow. Schuster et al. also classify approaches by the formalism of the output model. Here the output must be a sound workflow net, because ProSiT executes Petri nets and learns its routing per transition (Section 2.6).

Greco et al. enforce precedence constraints during discovery [48]. Norouzifar et al. integrate rules into the cuts of the Inductive Miner [49]. Dixit et al. work a posteriori: they edit a discovered model at random and keep the candidates that satisfy user-specified Declare constraints best [50]. Revoredo discusses how domain knowledge can guide the repair of a process model [51].

Classical model repair changes a model so that it can replay every trace of a log while staying close to the original [52, 53]. This adds behaviour. The opposite direction removes behaviour that the log does not support, for example by adding non-local places to a Petri net [54] or by refining process trees [55]. Vinci and de Leoni repair a model for simulation: they simulate it, find behaviour that is missing in the generated log and add it [56]. Restricting behaviour that the simulator generates too often is named as future work.

Implication for this thesis The repair in E2 uses explicit, declarative domain knowledge a posteriori and is fully automatic. It restricts the discovered language to the sequences that satisfy the rules and writes the result as a sound state machine. Rules that depend on timestamps or attributes are not forced into the net. They are handled as guards in the simulator (Section 2.6.5).

2.4 Conformance Checking

Conformance checking compares a log with a model [57, 44]. Four quality dimensions are common [58]. *Fitness* measures how much of the observed behaviour the model can replay. *Precision* penalises behaviour that the model allows but the log does not show. *Generalisa-*

tion rewards a model that is likely to accept new, unseen traces of the same process. *Simplicity* prefers small models. The dimensions pull in different directions. A model that accepts every sequence has perfect fitness and poor precision; a model that lists every observed trace has perfect precision and poor generalisation.

Alignments An alignment relates the trace of a case to a complete run of the model. It is a sequence of moves of three types. A *synchronous move* consumes an event and fires a transition with the same label. A *log move* consumes an event that the model cannot follow. A *model move* fires a transition without a matching event. Model moves on silent transitions are free. The other deviations have a positive cost. An optimal alignment $\gamma^\star$ of a trace σ with a net N is

$$\gamma^\star = \arg\min_{\gamma \in \Gamma(\sigma, N)} c(\gamma), \tag{2.1}$$

where $\Gamma(\sigma, N)$ is the set of all alignments and c the total cost [57, 59]. Optimal alignments are usually computed with the A* search algorithm on the product of the trace and the net. With an admissible heuristic, the first complete alignment found is optimal [59].

Fitness Trace fitness is one minus the cost of the optimal alignment divided by the cost of the worst case, in which every event is a log move and the model runs its shortest path alone. Log fitness averages over all traces. A trace *fits* if its optimal alignment has no deviation.

Precision Escaping-edges precision is based on alignments [60, 61]. For every prefix of an aligned trace, it compares the activities that the model allows next with the activities that the log shows after the same prefix. An activity that the model allows but the log never shows after that prefix is an *escaping edge*. Precision is one minus the share of escaping edges, weighted by how often each prefix occurs. Precision measures have known weaknesses. Their value depends on how silent transitions are handled after a prefix, so two nets with the same language can receive different values. Different tools implement this step differently. ProM and PM4Py can therefore report different precision for the same net and log [46]. Section 5.6.3 states which implementation is used and why.

Generalisation Token-based generalisation counts how often each transition is used in the replay. A transition that fires many times contributes little uncertainty; a transition that fires rarely contributes much [58]. The measure looks at transitions, not at sequences. A model that allows many unobserved sequences through frequently used transitions can still score high.

Coverage Fitness gives partial credit. A trace with one deviation among six events still reaches a trace fitness of about 0.8, and log fitness averages such values over all traces. Coverage measures ask a stricter question about whole cases: does the model accept the complete sequence, yes or no? This thesis uses three of them.

- **Share of fitting cases.** The share of traces whose optimal alignment has no deviation. It is the simplest fitness measure [24] and counts every case, so frequent sequences weigh more.

- **Share of accepted sequences.** The share of the distinct sequences (variants) of the log that the model accepts, each counted once. This is recall on the level of the language: the log is read as a set of sequences, not as a collection of cases. Entropy-based recall extends the same idea to models with infinitely many sequences [62].

- **Coverage per group.** The share of fitting cases within a group of cases, for example the visits that use two RMG blocks. A dominant variant can hide that a smaller but regular group is not accepted [49].

On a hold-out log, a fourth view checks generalisation directly: how many of the sequences that occur in the hold-out log but not in the training log does the model accept?

When cases follow a long-tailed distribution, the measures can differ a lot. A model that accepts only the few frequent sequences fits almost all cases but accepts only a small share of the sequences. For a simulator the unweighted view matters as well, because a sequence that the model does not accept can never be generated.

High coverage is not the same as overfitting. A model *overfits* if it allows only the observed sequences and fails on new ones; it *underfits* if it allows much more than the log shows [24]. Precision measures the second error. The first can only be seen on a hold-out log, which is why the fourth view above matters. Noise filtering, as in the Inductive Miner with filtering, removes rare behaviour because it treats it as noise [29]. Whether rare behaviour is noise or regular behaviour cannot be decided from its frequency alone. It is a question of the data quality and of the domain.

Implication for this thesis Fitness, precision and coverage compare the language of a model with the observed traces. Coverage shows which observed behaviour a simulator built on the model could never generate. None of the measures says how often a simulator will generate each permitted sequence. A path that is permitted but never observed may be rare in the language and still frequent in the simulation. This is why the selected models are also simulated

(E3).

2.5 Business Process Simulation

To explain:

- One to one and a half pages. Concepts only; the measures and replications used are defined in Chapter 5.

Business process simulation (BPS) runs an executable model of a process to study its behaviour without disturbing the real system [63]. A BPS model has five parts [63, 5]:

- an *arrival process* that decides when new cases start;
- a *control-flow model* that decides which activities can follow, together with *routing* probabilities or rules at each choice;
- *resources* with their capacity and working calendars;
- *time models* for the execution and waiting times of activities;
- *case attributes* that describe each case.

Most BPS tools use discrete-event simulation: time jumps from one event to the next. Because arrivals, routing and durations are drawn at random, one run is one possible history. Several runs with different random seeds (*replications*) show how much the results vary [64]. Simulation is mainly used for *what-if analysis*: the same model is run with and without a change, for example more resources or another allocation rule, and the KPIs are compared [63].

Related work: discovering simulation models Rozinat et al. showed early that all five parts can be estimated from an event log [5]. Camargo et al. automate this and search for the configuration whose generated log is most similar to the real log [13, 65]. López-Pintado and Dumas show that modelling resources individually, with their own calendars and performance, brings the simulated cycle times closer to the real ones [66, 67]. Stochastic process models add probabilities to the paths of a model [68, 69]. Deep-learning generators can learn more complex dependencies, but they are hard to inspect and to change for a scenario [70].

Related work: data-aware simulation A classical BPS model uses the same routing probability for every case. A *data-aware* model lets routing, times or resource choice depend on data, for example case attributes or the current workload. Mannhardt et al. define stochastic

Petri nets whose transition weights depend on data [7]. De Leoni et al. add the history of the case to this *process state* and show that the best state depends on the process [8]. López-Pintado et al. also model how attributes change during a case [9]. RIMS calls predictive models at run time, so that they can use the current state of the simulation [10]. ProSiT, introduced in Section 2.6, applies data awareness to all parts of the model.

Validation A simulation model is valid only for an intended use [71]. Sargent separates *conceptual validity* (are the assumptions reasonable for the purpose?), *verification* (is the model implemented correctly?), *data validity* and *operational validity* (is the output accurate enough for the purpose?). For discovered BPS models, operational validity is usually checked by comparing logs generated by the model with a real log that was not used for discovery. Chapela-Campa et al. recommend separate measures for the control flow, the timing and the congestion of the generated logs, because one aggregate score hides which part of the model is wrong [72, 73]. Özdemir et al. add two cautions [74]: a later test log mixes the error of the model with changes of the process between the periods, and distances based on the Wasserstein distance can hide temporal patterns. Law recommends *trace-driven* comparisons, in which the model is fed with the observed inputs of the comparison period, for example the observed arrivals, so that model and system face the same demand [64]. A comparison with a historical log shows that the model reproduces the current process. It does not show that the model predicts the effect of a change correctly. Evidence for a what-if use needs more: plausible mechanisms, a model that responds to load as the real system does, and the judgement of people who know the process. Such *face validation* by domain experts is a recognised technique [71]. A review of 105 studies on simulation and process mining found that 36 report no validation at all [75].

Replications A model is run N times with different seeds. For a measure with replication values $x_1, \ldots, x_N$, the mean $\bar{x}$ is reported with the interval $\bar{x} \pm t_{0.975,N-1}\, s/\sqrt{N}$, where s is the standard deviation across replications [64]. This interval describes the random variation of the simulator for one fitted model. It does not cover the uncertainty of the fitted parameters or of the model structure.

2.6 ProSiT

To explain:

- Core section, three to four pages. Describes ProSiT 1.0.4 as implemented (`prosit-pm`); every statement must match the code. The values chosen for CTB are in Section 5.8, the full trees in

Appendix C.

ProSiT (*PROcess SImulation Tool*) discovers a data-aware simulation model from a Petri net and an event log [11, 76]. Every part of the model is learned as a *probabilistic decision tree*: a decision tree whose leaves hold a probability or a distribution instead of a single value. The model is called white-box, because every learned rule can be read and changed. This section describes what ProSiT learns, how a tree is built and used, how a simulated case runs, and where the framework stops.

2.6.1 Inputs and Learned Components

ProSiT needs a sound workflow net and an event log with start and complete timestamps, a resource per event and case attributes. It first aligns every trace with the net. The alignment tells, for every event, which transition fired, which transitions were enabled at that moment and since when. From this replay, ProSiT builds one training table per component. Table 2.2 lists the components.

A few components need a short explanation.

- **Arrivals.** The gaps between consecutive case starts are computed in open minutes of the arrival calendar. The calendar marks a weekday and hour as open if enough arrivals fall into it. It decides only *whether* cases can arrive; the tree decides *how often*.

- **Routing.** For every transition, ProSiT collects all moments in the replay at which it was enabled. The class is 1 if it fired and 0 if another transition fired. The tree then predicts the probability that the transition fires, given the case attributes and how often each activity has occurred so far. Silent transitions get a routing tree like visible ones.

- **Capacity.** By default every resource handles one activity at a time. With *multitasking* enabled, a resource whose activities overlap in at least a given share of cases gets a capacity of the largest observed overlap plus one.

- **Waiting time.** The waiting time of an event is the time between its start and the later of two points: the moment the transition was enabled, and the moment the resource became free. Waiting caused by a busy resource is therefore not part of the target. The simulator produces it through the capacity check.

Component	Learned model	Predicts	Features
Arrivals	one regression tree	gap until the next case starts	hour and weekday of the previous arrival
Arrival calendar	table	open or closed per weekday and hour	–
Case attributes	observed tuples or one distribution per attribute	attributes of a new case	–
Routing	one classification tree per transition	whether the transition fires when it is enabled	case attributes, number of times each activity has occurred so far
Resource choice	one classification tree per resource	whether the resource is chosen	activity, case attributes, resources used so far (optional: workload, queue length)
Capacity	number per resource	how many activities can run at once	–
Waiting time	one regression tree per resource	time from "enabled and resource free" to start	activity, hour, weekday, case attributes, activity counts (optional: workload, queue length)
Execution time	one regression tree per activity	time from start to complete	resource, hour, weekday, case attributes, activity counts
Resource calendars	table per resource	open or closed per weekday and hour	–

Table 2.2: Components of a ProSiT simulation model (version 1.0.4). Times are counted in open minutes of the relevant calendar.

2.6.2 Probabilistic Decision Trees

A decision tree splits the training data step by step on one feature at a time, for example "hour $\leq$ 21". Each split is chosen so that the groups become as different as possible in the target (CART). The groups at the end are the *leaves*. In ProSiT, a routing leaf stores a probability. A time leaf stores a distribution: ProSiT fits a fixed value, a normal, exponential, lognormal, gamma and uniform distribution to the values in the leaf and keeps the one closest to the data in Wasserstein distance. Values drawn from it are limited to the smallest and largest observed value.

Model selection A tree is not always better than one distribution for all cases. ProSiT therefore chooses by five-fold cross-validation. It tries every tree depth from one to `max_depth_tree` and every minimum leaf size in `min_samples_leaf_cv`, and compares them with a single distribution without splits. For routing, the score is the log loss of the predicted probabilities.

For time models, version 1.0.4 uses the continuous ranked probability score (CRPS). For an observed value y and a predicted distribution F,

$$\mathrm{CRPS}(F, y) = \mathbb{E}\,|X - y| - \tfrac{1}{2}\,\mathbb{E}\,|X - X'|, \tag{2.2}$$

where X and X′ are independent draws from F [77]. The first term rewards distributions close to the observation, the second rewards spread where the data are spread. A lower score is better. Version 1.0.3 used the Wasserstein distance between training and test values inside each leaf. That criterion measures only how stable a leaf is, so it almost always chose a single distribution. This thesis uses version 1.0.4 (Section 5.8.2).

Example Figure 2.2 shows the arrival tree that ProSiT learned from the training data of this thesis. Read from the top: if the previous truck arrived on a weekday between 5 and 21 h, the gap to the next truck is drawn from a normal distribution with a mean of 0.65 minutes, about 90 trucks per hour. At night, the mean gap is about two minutes. On Saturday after 10 h, it is 13 minutes. The tree has depth three, so it cannot separate the morning ramp-up, the daytime plateau and the evening; the leaf from 5 to 21 h pools them. Whether this is good enough is tested in E4a (Section 5.9.4).

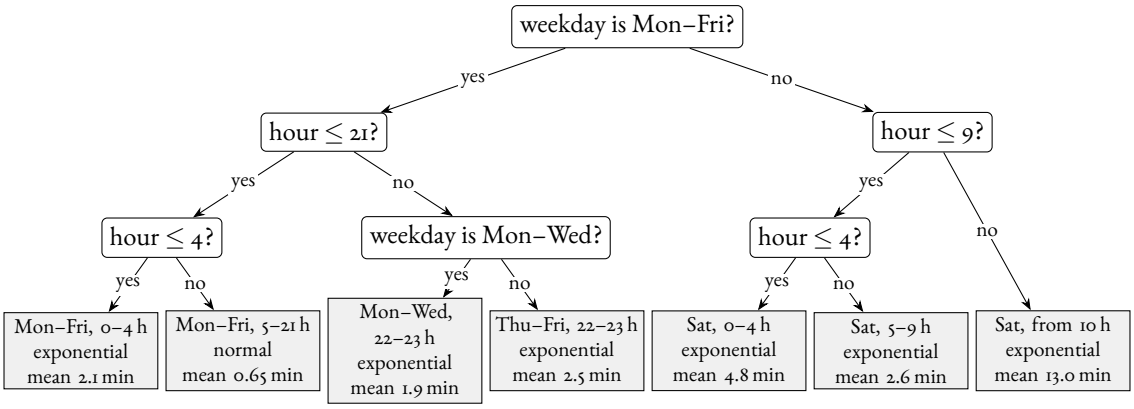


Figure 2.2: A real ProSiT tree: the arrival model learned from the CTB training partition (ProSiT 1.0.4, Section 5.8). The tree predicts the gap until the next truck arrives, in open minutes of the arrival calendar, from the hour and weekday of the previous arrival. Each leaf holds a fitted distribution. The simulator follows the conditions from the top to one leaf and draws the gap from that leaf. The means are those of the values the simulator draws. Sunday is closed in the arrival calendar, so no gaps are drawn there.

2.6.3 Execution of a Simulated Case

The simulator processes all cases in one loop, always continuing with the case whose next step is enabled earliest. A case runs as follows.

1. **Arrival.** The start time is the start of the previous case plus a gap drawn from the arrival tree, counted in open minutes of the arrival calendar.

2. **Attributes.** The case receives its attributes, in empirical mode as one complete tuple of a randomly drawn training case.

3. **Enabled transitions.** The token sits in the source place. A transition is enabled if all its input places hold a token. Its enabling time is the latest time stamp of these tokens.

4. **Routing.** Each enabled transition gets the probability from its routing tree, given the case attributes and the activities so far. The simulator draws one transition in proportion to these weights. Silent transitions are drawn in the same way and take no time.

5. **Resource.** For a visible transition, the candidate resources are those that executed this activity in the log. Of these, the ones with free capacity at the enabling time are kept, and one is drawn from the resource trees. If none is free, the resource that becomes free first is taken and the start moves to that moment.

6. **Times.** A waiting time is drawn from the tree of the resource (zero for the first activity of a case). The start is the enabling time plus the waiting time; the complete time is the start plus an execution time drawn from the tree of the activity. Both are counted in open minutes of the resource calendar.

7. **Token update.** The tokens move to the output places and carry the complete time. Steps 3 to 7 repeat until the final marking is reached.

The generated log contains, for each event, the case, activity, resource, enabling, start and complete time and the case attributes. The firing sequence of each case is stored as well.

2.6.4 Hyperparameters

Table 2.3 lists the hyperparameters of the discovery. The values used in this thesis and the reasons are given in Section 5.8.

Parameter	Default	Meaning
`max_depth_tree`	5	Largest tree depth tried in cross-validation. A value below 1 switches all trees off (one distribution or probability per component).
`min_samples_leaf_cv`	50, 100, 200	Minimum leaf sizes tried in cross-validation.
`use_workload_features`	off	Adds the current workload and queue length of a resource as features of the resource choice and the waiting time. Execution times do not use them.
`enable_multitasking`	off	Allows a capacity above one (see above).
`multitasking_thr`	0.05	Share of cases with overlapping activities above which multitasking is assumed.
`attribute_mode`	distribution	*distribution*: each attribute drawn on its own; *empirical*: complete observed attribute tuples.
Arrival calendar thresholds	0.1, 0.7	Minimum confidence of a slot and minimum share of arrivals the open slots must cover.
Resource calendar thresholds	0.1, 0.1, 0.4	Minimum confidence, support and participation of a resource slot.
`random_state`	72	Seed of the discovery.

Table 2.3: Discovery hyperparameters of ProSiT 1.0.4.

2.6.5 Evidence and Limitations

The ProSiT paper evaluates the framework on five case studies with a temporal 80/20 split and five simulated logs per model. ProSiT reaches an accuracy comparable to a black-box baseline while its rules stay readable [11]. The authors also stress a boundary: the trees capture associations in the training data, not causes. A readable rule shows which association the simulator uses. Whether it holds after an intervention needs other evidence.

For this thesis, seven properties of the implementation matter.

1. **No hard rules.** A transition can fire whenever its input places hold tokens and its weight is above zero. ProSiT has no guards. A rule that the net does not encode, such as "at least one stop" or "as many stops as planned", is followed only as often as the routing trees happen to respect it.

2. **No look-ahead.** Each routing draw is local. A sequence of likely steps can lead to a visit that never occurs in the data.

3. **Parallel branches run in parallel.** A silent parallel split enables several stops at the same time, and the simulator can start them at the same time.

4. **Capacity at enabling time.** The capacity of a resource is checked when the transition is enabled, not over the interval in which the activity actually runs.

5. **Load enters through few features.** Workload features act on the resource choice and the waiting time only. Execution times do not react to the load of the terminal.

6. **The queue feature counts the activity itself.** In training, the queue length of a resource counts the activities that are enabled and not yet started. An activity that waited is therefore counted in the queue of the resource that performed it, but not in the queues of the other candidates. In the simulation the activity is not yet in any queue when the resource is chosen.

7. **Waiting is measured against one server.** The waiting time is the time from the moment the resource was last completely free, that is, from the latest end of all its activities in progress at the enabling time. For a resource that serves several cases side by side, a lane may already be free much earlier.

The first three properties are the reason why the control-flow model matters more for a simulator than for a descriptive model. This thesis addresses them with the rule repair of E2 and with a data-aware *stop guard*: before each routing draw, transitions that would break a data rule get the weight zero (Section 5.8). This is the guard of a data-aware Petri net (Section 2.3.3), evaluated inside the ProSiT loop. Properties four to seven concern the physical constraints of the blocks and are examined in E4 (Section 5.9).

2.7 Research Gap and Positioning

To explain:

- About one page. End with the research questions of Chapter 1.

Table 2.4 summarises what the reviewed research contributes and which question remains open for a terminal like CTB.

Three gaps follow. First, discovered models are usually selected by fitness and precision. These measures do not show what a simulator does with behaviour that the model permits but the log never shows, and physical rules are not part of the discovered model. Second, data-aware simulation discovery has been evaluated mainly on administrative and healthcare processes. Its behaviour in a physical logistics system with shared capacity and congestion is largely

Research direction	Main contribution	Open question for CTB
Terminal simulation [18, 4]	Hand-built models that compare operating policies on KPIs such as TTT.	Can a model discovered from data support the same kind of comparison?
Discovery benchmarks [38, 39]	Compare fitness, precision and size of discovered models.	Which model is adequate when it is *executed*, not only described?
Domain knowledge and repair [47, 50, 56]	Use rules in discovery or repair; repair for simulation adds missing behaviour.	Can physical rules remove impossible behaviour exactly, and which rules belong in the net and which in the simulator?
Data-aware simulation [7, 9, 11]	Routing and times depend on data and process state; evaluated on business processes.	Does it reproduce a physical process with shared capacity and congestion?
Simulation validation [72, 73, 71]	Multi-perspective comparison of generated and real logs.	Does the model respond to load like the terminal, and do domain experts find it plausible?

Table 2.4: Research directions and the questions they leave open for the CTB case study.

untested. Third, validation of discovered simulation models focuses on similarity to a historical log. Whether the model responds to load like the real system, which is needed before a what-if analysis, is rarely tested, and the judgement of domain experts is rarely documented.

This thesis addresses these gaps in a case study of truck operations at HHLA Container Terminal Burchardkai. It does not propose a new discovery algorithm or a new simulator. It compares discovery algorithms for their use in simulation (E1), restricts the selected models with physical rules (E2), measures how the control-flow model changes the simulated visits (E3), audits the simulation mechanisms and their response to load (E4), and evaluates one operational policy only if the model passes these checks (E5).

3 Data Preparation

To explain:

- Roadmap: environment, data sources, event-log construction, context attributes, key facts of the event log, domain rules. Keep the chapter short; details go to Appendix A.
- This chapter only describes the data and the observed system. It does not fit models.

This chapter describes the operational context and the pipeline that turns raw operational records into a simulation-ready event log. The resulting event log is the input for Chapters 5 and 6.

Section 3.1 introduces HHLA Container Terminal Burchardkai (CTB), the truck-handling operations and the data sources. Section 3.2 describes how the pipeline reconstructs lifecycle timestamps and derives the case-level context required by ProSiT [11]. Section 3.3 gives the key facts of the resulting event log, and Section 3.4 the domain rules that every truck visit must satisfy. The use of the train/validation/test partitions is described in Section 5.2.

3.1 Case Study Environment

To explain:

- Keep it short and concrete (the committee criticised too much port context in the defense).

3.1.1 HHLA Container Terminal Burchardkai

To explain:

- Terminal key facts, the truck process, the handling areas (RMG blocks, LL, HO2) and the gate.
- Replace "operational experts confirmed by the exact source (author knowledge or documented terminal review with date).

This case study concerns the HHLA Container Terminal Burchardkai (CTB) in the Port of Hamburg. CTB handles approximately 2.6 million TEU (Twenty-foot Equivalent Units) annually across 315 hectares [78].

Truck process. External trucks visit CTB to deliver or receive containers. At *Gate In*, the truck is registered at the entry gate. It then drives to the assigned yard position, where an RMG or other handling equipment loads or unloads the container. After all planned operations are completed, the truck leaves through *Gate Out*.

The CTB environment in this project has several resource constraints. Located between the waterside and the landside, 22 RMG storage blocks (T06–T27) serve both sides. Operational experts from the terminal confirmed two constraints used in this study: the activities of one truck cannot be executed in parallel, and at most three automated rail-mounted gantry cranes can work concurrently at one block. The CTB has two specialised areas: an empty-container depot (called "MT" or "LL", from the German *Leerlager*) and the manually operated HO2 yard area. The terminal gate consists of 16 lanes, split between incoming and outgoing trucks. The number of operated gate lanes varies by day, shift and terminal resource plan.

3.1.2 Data Sources

To explain:

- Keep the table. Add the observation period and what each system records about a truck visit.

The data come from several systems used in daily terminal operations. These systems were developed over time and record different parts of the process. A Terminal Operating System coordinates them. Table 3.1 summarises the four sources used in this thesis.

These systems generate independent data pools with different identifier schemes, requiring a data-integration procedure described in Section 3.2. Historical data from March and April 2026 (approximately eight weeks) were extracted. The raw extract covers approximately 104,000 truck visits, 113,466 container move events and 336 yard-state snapshots (8 weeks $\times 7$ days $\times 6$ snapshots per day). The move-event count refers to the raw report-system extract. It

Data source	Data provided
ITS schedule	Truck-visit schedules and timestamps for gate entry, handling readiness, order completion and gate exit.
Report system	Stage-level execution data, gate-entry records, exchange-area assignments, container transactions and carrier identifiers.
Yard management system	Four-hour snapshots of storage-block capacity, occupancy and utilisation.
Container master data	Container size, type, loading status, hazardous and reefer indicators, dwell time and inbound/outbound carriers.

Table 3.1: Operational data sources used for event-log engineering.

is not the denominator used later for the cleaned and cross-system-matched schedule records.

3.2 Event-Log Construction

To explain:

- The pipeline from four source systems to one truck-visit event log, as one figure (figures/schemas/event_log_engineeri
  and a short walk-through.
- Every step that changes the population must be counted (attrition table: raw visits, after cleaning, after matching, final).

The Truck schedule serves as the baseline for the data assembly.

In a first step, the raw schedule is cleaned from false or incomplete entries. That includes missing timestamps and total visit times of over 12 hours, as those indicate system errors.

We are matching container details to each recorded transatction to give each visit with its operations complexity. Using ProSiT as the simulation framework in this thesis, we are then aggregating the individual container details to a case level complexity. Container attributes, like dangerous goods, fill status etc. are transformed into a complexity code system attributing each visit with a complexity score and container counts and trafic direction.

Then the schedules attributes and timestamps are transformed into an event log style. Then the system variables are added by the initial timestamp of the process, aka the moment, the truck enters the terminal. The demand of the gate at moment of entry, as well as the demand at the planned transaction points are calculated.

To include the terminals utilization into the process behaviour, the utilization of each of

the yard sections and blocks is calculated and added to each visit.

[TODO: Decided: no target features; multiblock visits are restored (expanded population). Describe this in Cleaning and Population.]

3.2.1 Visit Reconstruction

To explain:

- Case notion: one completed truck visit from Gate In to Gate Out; the key that links the systems (schedule UID).
- How container transactions are matched to visits, and what happens to unmatched records.

3.2.2 Activities and Event Order

To explain:

- The eleven generic labels (table): Gate In, Gate Out, receive/delivery/mixed at RMG, LL and HO2. Why block names are not part of the label.
- Event order from case:event:order (schedule sequence), not from timestamps, and why.

3.2.3 Timestamp Semantics

To explain:

- A stop interval runs from readiness in the handover lane to fulfilment. It can contain waiting for the crane, so it is not pure service time.
- Gate events are instantaneous; therefore turnaround time decomposes exactly into approach, stops, transfer and exit.
- Minute resolution of all source timestamps; known imprecision at LL and HO2.

Order of stops and imprecise LL timestamps The stops of a visit never overlap in the data: in no training or test visit does a stop start before the previous stop has ended. This supports domain rule 4 (Section 3.4). The gate events are less clean. In 6 of the 59,139 training visits and 5 of the 32,014 test visits, an `LL_delivery` stop ends about two minutes after the truck has passed Gate Out. Such a stop cannot be physical; the timestamps of the empty-container depot are known to be imprecise. The event log used for discovery orders the events of a visit by the recorded sequence (`case:event:order`), in which Gate Out is always last, so these timestamps do not affect it.

They do matter for discovery methods that read start and end timestamps. The lifecycle variant of the Inductive Miner [40] treats overlapping intervals as concurrency. Without a noise filter, the six overlapping training visits, 0.01 % of the training log, were enough for it to place Gate Out in parallel with the stops: the resulting net accepts a stop after Gate Out, which breaks rule 2. For this experiment, the end of a stop is therefore capped at the Gate Out time of its visit. With this correction the lifecycle variant returns a net with exactly the same language and the same concurrency as the Inductive Miner without lifecycle information (Section 5.6). Lifecycle-based discovery thus depends on reliable start and end timestamps: a few imprecise timestamps at one area are enough to produce a physically impossible model.

3.2.4 Resource Identities

To explain:

- Resource = RMG block (T06–T27), LL, HO2, gates. One generic label can occur at different blocks.
- Physical capacities and their source: three handover lanes and up to three cranes per block shared with waterside and rail; four lanes at LL. Name the source (author knowledge, terminal confirmation with date).

3.2.5 Cleaning and Population

To explain:

- Cleaning rules (missing timestamps, visits over 12 hours, unknown activity labels: 184 visits) with counts.
- The population correction: visits with several RMG blocks were first excluded for a target feature, then restored, because the exclusion removed a regular pattern (Chapter 7, E1). Final counts per partition.

3.2.6 Context Enrichment

To explain:

- For every attribute: definition, source, and whether it is known at Gate In (planned manifest) or reconstructed. This answers the leakage question.
- Note that historical demand and utilisation values are case-constant in the simulation and do not react to simulated load.

To enable data-aware simulation, the event log is enriched with contextual attributes describing the operational state at the time of process execution. These attributes are organised into five categories.

Information Available at Truck Entry The scheduling record contains the complete planned visit manifest before the truck enters the terminal. The number of planned stops, the numbers of delivery and receive transactions, the planned target area and the resulting final visit-complexity score are known at Gate In. The pipeline reproduces this entry-time information by aggregating all rows of the same `FAHRPLAN_UID`. It does not infer these attributes from later realised durations or completion outcomes. `visit_complexity` is a deterministic summary of the planned stop/container manifest (visit size, container-handling complexity and mixed-flow complexity), not an ex-post performance measure. Using these variables as case attributes does not leak the simulated future provided that the same planned manifest is supplied for a prospective run.

The implementation detail that the target block is reconstructed from the planned stop's area-specific activity label can otherwise look like look-ahead: after genericising `T06--T27` activities, the reconstructed block is copied into `primary_target_area`. Semantically, however, the source is the planned `ANLAUFPUNKT_HALTESTELLE` attached to the visit order and available at entry, not the later activity-completion timestamp. This distinction defines the deployment contract for all entry-known case attributes.

Container Features Container-related context is aggregated to case level so that visit properties can serve as case-level attributes in ProSiT. The resulting `n_containers` attribute counts the unique containers handled during the visit. The binary attributes `has_hazardous` and `has_reefer` are aggregated using a logical OR across the complete manifest. The `full_ratio` records the share of loaded containers, while `container_complexity_mean` and `container_complexity_su` describe the average and cumulative handling complexity of the visit.

Yard Features Yard state features capture the storage capacity and utilisation status of different terminal areas at truck arrival (Gate In event). These features are matched by an exact equality join on the minute-resolved Gate In timestamp. The joined snapshot provides the overall `gate_utilization`, the aggregated `rmg_utilization`, and the area-level `vc_utilization` and `mt_utilization`. For single-block visits, `target_utilization` gives the utilisation of the assigned RMG block.

Demand Features Demand features quantify operational load pressure at different terminal areas over a rolling 60-minute window. The general `gate_demand` counts trucks pro-

cessed across the terminal during the preceding hour. The `rmg_demand`, `vc_demand` and `mt_demand` attributes separate this load by operational area. For single-block visits, `target_demand` records the demand for the assigned RMG block.

3.3 Event Log Overview

This section gives the key facts of the event log that later chapters use. The full description of the observed behaviour, with the arrival pattern, congestion over the day, delay peaks, disruption episodes and the differences between the partitions, is in Appendix A. Thresholds are fitted on the training partition and applied unchanged to the validation partition. The test partition is not used.

3.3.1 Visit Structure and Turnaround Time

Table 3.2 summarises both partitions. Nine out of ten visits have a single yard stop. Visits with three or more stops are rare. Receive and delivery visits are similarly frequent, and about one visit in seven is mixed. About a quarter of the visits do not use an RMG block. Visits with two different RMG blocks make up 1.7 % of the training visits.

Measure	**Training**	**Validation**
Visits	58,262	14,625
Visits with 1 / 2 / $\geq$3 stops	91.3 / 8.6 / 0.2 %	89.8 / 10.1 / 0.1 %
Delivery / receive / mixed visits	38.6 / 48.2 / 13.2 %	42.2 / 43.4 / 14.5 %
No RMG / one block / several blocks	22.5 / 75.7 / 1.7 %	27.1 / 70.8 / 2.1 %
TTT mean / median / P90 [min]	36.5 / 28.5 / 64	42.2 / 34 / 78
Median TTT delivery / receive / mixed	23 / 30 / 41	28 / 35 / 48
Median TTT one block / several blocks	29 / 46.5	33 / 50
Mean approach / stops / transfer / exit	10.9 / 12.0 / 1.1 / 12.5	13.2 / 13.7 / 1.5 / 13.6

Table 3.2: Visit structure and turnaround time (TTT) by partition. Approach is the time from Gate In to the start of the first stop, exit the time from the end of the last stop to Gate Out, and transfer the time between stops. Because gate events are instantaneous, the four components add up to the TTT.

The turnaround time depends on the type of visit. Receive visits take longer than delivery visits, and mixed visits take longest. A visit with two RMG blocks takes about 17 minutes longer than a single-block visit. The decomposition shows that the stop intervals account for only about one third of the mean TTT. Approach and exit together account for about two thirds. Since a stop interval begins when the truck reports ready in the handover lane, it can already contain waiting for the crane. Driving and any queueing before the truck reports ready fall into the approach.

3.3.2 Load and Delay

Trucks arrive from Monday to Friday, with few visits on Saturdays and none on Sundays. On weekdays, between 126 and 152 trucks per hour arrive from 7:00 to 16:00. Hours with at least 165 arrivals, the 90th percentile of the training hours, are called *arrival peaks*.

The turnaround time grows with the number of trucks in the terminal, and the growth is non-linear (Figure A.3). Up to about 100 trucks, the median TTT rises slowly from 23 to 34 minutes. Between 140 and 160 trucks it reaches 43 minutes, and above 160 trucks the P90 exceeds two hours.

Hours in which the median TTT is at least 37 minutes, the 90th percentile of the training hours, are called *delay peaks*. They make up 10.2 % of the training hours and 26.5 % of the validation hours. Delay peaks and arrival peaks rarely coincide. The additional time in delay peaks lies mainly in the approach to the first stop and in the exit, not in the stop intervals. A few disruption days, such as 23 March 2026, show very long exit times at normal arrival rates. They are reported separately but never removed.

The validation period is more congested than the training period, with a daily median TTT of 32 against 29 minutes (Section A.5). The set of activity sequences, however, hardly changes: only six validation visits follow a sequence that does not occur in training. A model fitted on the training period is therefore expected to underestimate the validation TTT, independently of its control flow.

3.3.3 Implications for the Simulation

The observed behaviour defines what an adequate simulation must reproduce for the purpose of this thesis.

1. The weekday and hourly arrival profile, including arrival peaks above 165 trucks per hour.

2. The stop structure and the turnaround time by visit type.

3. The increase of the turnaround time with the number of trucks in the terminal, and of the stop interval with the occupancy of the unit.

4. The growth of approach and exit time under load, since these components carry most of the delay in delay-peak hours.

5. Delay peaks at a frequency comparable to the observed one, excluding disruption episodes, which are reported separately.

These requirements are the basis of the comparison measures in Chapter 5.

3.4 Domain Rules

To explain:

- Add per rule the share of training and validation visits that satisfy it (audit level 1 in E2).
- Open points: repeated mixed events at the same block; eligible blocks for deliveries.

The physical constraints of Section 2.1 are written down as seven rules (Table 3.3). They are based on the domain knowledge of the author. They have not yet been confirmed independently by the terminal. Confirmation is requested in part B of the domain review (Section 5.5). Until then the rules are working assumptions. Rule 7 is tentative, because the truck type and its capacity are not recorded.

For LL and HO2, the area is the handling unit, so rule 5 can be read from the activity labels. For RMG, the unit is the block, which the generic activity label does not show. The RMG part of rule 5 therefore needs the block of each stop, which is the resource of the event.

3.5 Chapter Summary

To explain:

- One paragraph: the event log, the population, the observed delay pattern and bottleneck, and the rules. Hand over to the methodology (Chapter 5).

#	Rule	Type	Needs
1	Gate In occurs exactly once and is the first event.	sequence	activity labels
2	Gate Out occurs exactly once and is the last event.	sequence	activity labels
3	A visit has at least one yard stop between Gate In and Gate Out.	sequence	activity labels
4	The stops of a visit never overlap in time.	time	start and complete timestamps
5	A mixed stop handles all moves of the visit at its unit. The visit has no separate delivery or receive at the same unit.	sequence (LL, HO_2); data (RMG)	activity labels; block of each RMG stop
6	The number of stops equals the number of planned stops in the manifest, at most four.	data; the bound is a sequence rule	planned stops (case attribute)
7	The load of the truck stays within its capacity after every stop (tentative).	data	container counts and truck type (not recorded)

Table 3.3: Domain rules for truck visits. Source: domain knowledge of the author; confirmation by the terminal pending. The type decides where a rule can be enforced (Section 5.7).

4

Model Construction and Simulation

This chapter describes the simulation model: how it is built from the event log, what it has learned, and how it produces one simulated truck visit. The experiments that selected and tested its parts are described in Chapter 5; their results are in Chapter 6. The model described here is the final configuration of E4 (Section 5.9.4).

4.1 From Event Log to Executable Model

The model has two parts. The *control flow* says which sequences of stops a truck may perform. It is a Petri net discovered from the training log and repaired with domain rules (E1, E2). The *simulation parameters* say how often each path is taken, which block a stop goes to, and how long trucks wait and are served. ProSiT learns them from the same training log [11]. A small number of changes adapt the simulation loop to the physical limits of the terminal (Section 4.5).

Figure 4.1 shows how these parts produce one simulated visit. A truck arrives and receives its manifest. It then moves through the control flow one step at a time. At each step, the learned routing gives every possible next step a probability, the stop guard removes the steps that would break the planned number of stops, and one step is drawn. For a yard stop, the block, the waiting time and the execution time follow. The visit ends with Gate Out.

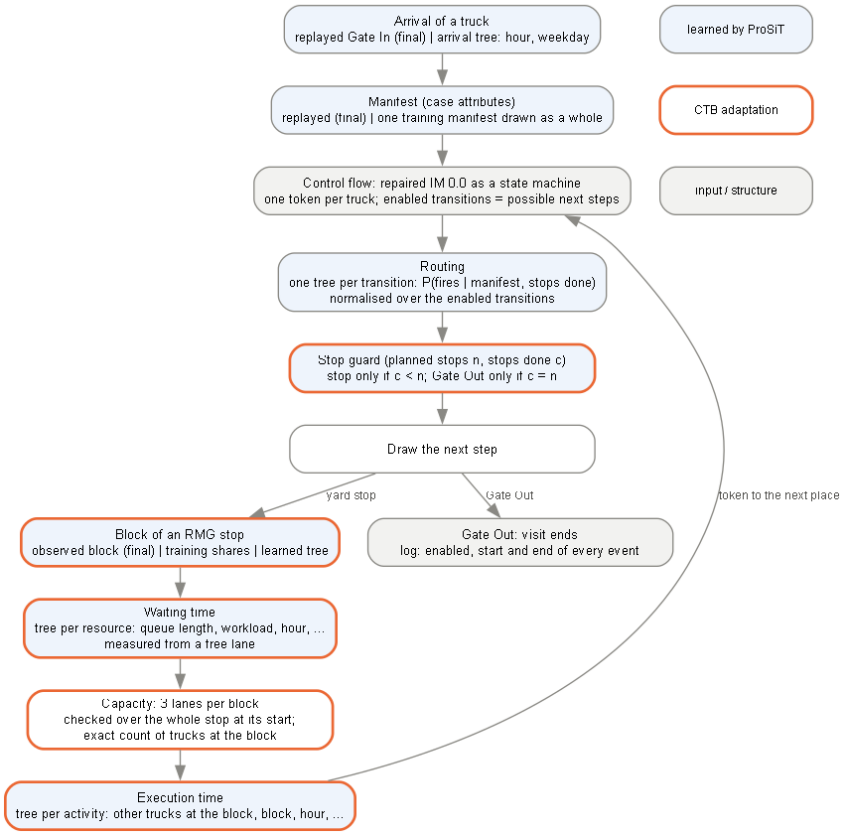


Figure 4.1: Steps of one simulated visit in the final model and where each value comes from. Blue: learned by ProSiT from the training log. Orange border: adaptation to the terminal (Section 4.5). Grey: structure and output.

4.2 Control Flow

The control flow is a workflow net: a Petri net with one start and one end place, in which every truck visit is one token that moves from Gate In to Gate Out (Section 2.3.1). ProSiT executes the net, so it must be sound: from every reachable state the visit can still end, and no token is left behind.

The net used in the simulation is the unfiltered Inductive Miner net IM 0.0, restricted exactly to two domain rules and written as a state machine (Figure 4.2). Its selection is described in E1 and E2 (Sections 5.6 and 5.7). It accepts all 82 sequences of the training log and has ten places and 54 visible transitions. Every place is a state of the visit, and every transition one possible next step. A state machine is a Petri net in which every transition has exactly one input and one output place [28]. Several transitions can carry the same activity label, which the labelled

nets of ProSiT allow.

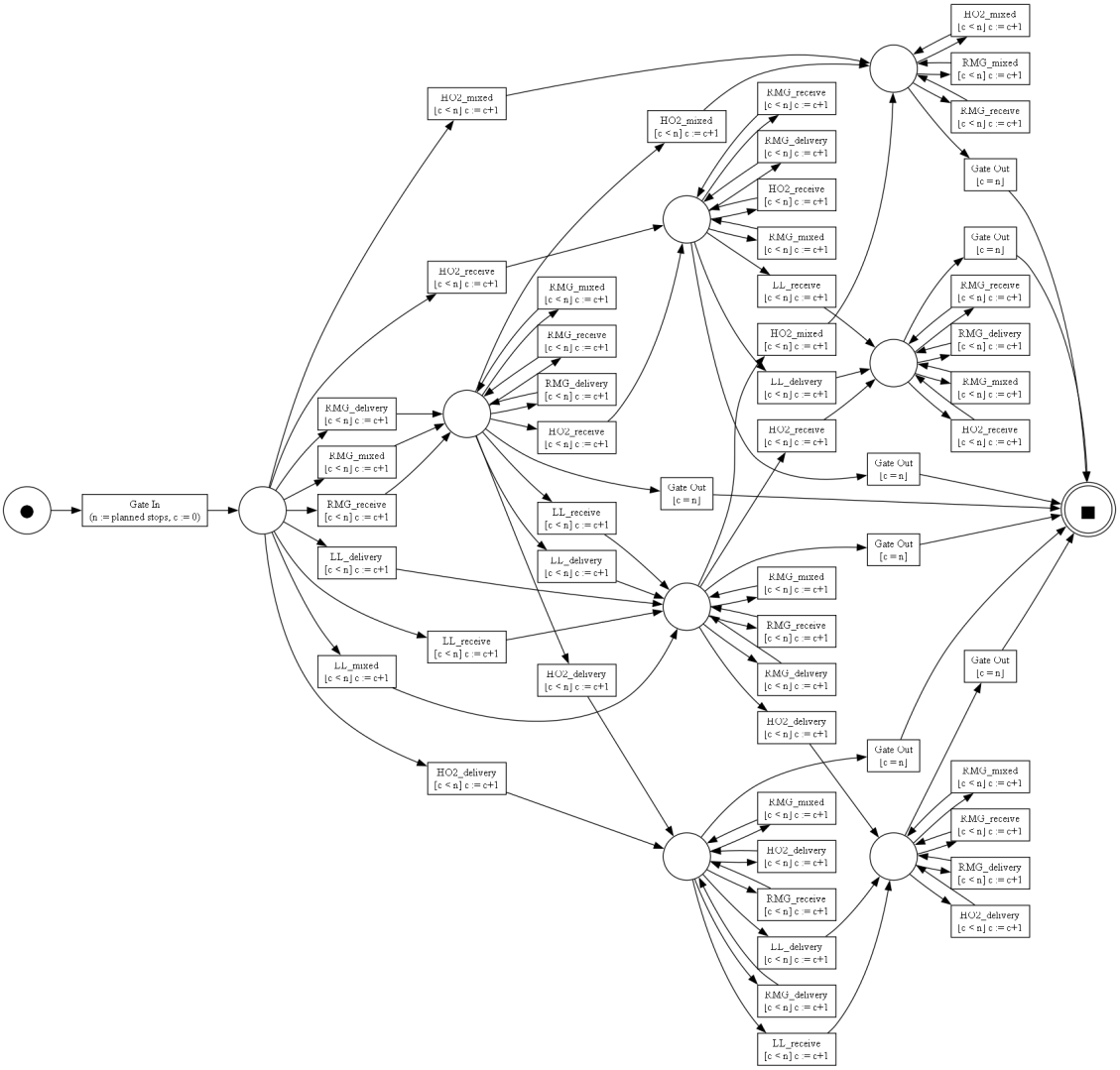


Figure 4.2: Control flow of the simulation model: IM 0.0 restricted to rules 3 and 5 as a state machine (10 places, 54 transitions, no silent transitions). The labels show the stop guard of the simulator: n is the number of planned stops from the manifest and c the number of stops done; a yard transition may fire only if $c < n$, Gate Out only if $c = n$.

Two properties of this form matter for the simulation. First, a truck holds one token, so two of its stops can never be scheduled at the same time (rule 4). Second, every decision is one choice among the visible next steps at one place. ProSiT gives every transition a weight that depends on the case attributes and on the activities done so far, but not on the marking of the net [8, 11]. In a state machine, each transition has one input place, so its weight automatically refers to one state of the visit. In the original Inductive Miner net, the free order of the stops is

expressed by parallel branches that can be skipped by silent transitions, and the skip decisions are then taken independently of each other. Decision points are places with several outgoing arcs, and silent transitions are a known difficulty for learning them [5]. The consequences for the simulated visits are analysed in E3 (Section 6.4).

The stop guard is the only data condition of the model. It is not part of the net, because ProSiT has no guards; it is applied in the simulation loop before every routing draw. Together with the state machine it forms a simple data Petri net with the planned stops n and a counter c. From every reachable state, a visit can still end with exactly n stops, for every n from one to four, so the net with the guard is sound in the data-aware sense [79].

4.3 Learned Components

ProSiT learns every simulation parameter as a probabilistic decision tree [11]. The training log is first aligned with the net, so that every event is assigned to a transition. For each component, a tree is then trained on the features that are known at the moment the value is needed, for example the manifest of the visit, the hour, or the number of trucks at a resource. Each leaf holds a fitted distribution (exponential, normal, lognormal, gamma, uniform or a constant) and the sample of training values it was fitted on. In the simulation, the features of the current situation select one leaf, and the value is drawn from its distribution, truncated to the range observed in the leaf. ProSiT 1.0.4 chooses by cross-validation between a tree (depth one to three, at least 50, 100 or 200 cases per leaf) and a single distribution, using the continuous ranked probability score. Table 4.1 lists the components; the settings are in Table 4.2.

Arrivals and manifests. In generative runs, the time to the next arrival is drawn from a tree on the weekday and hour of the last arrival (Figure 4.3). On weekdays between 5 and 21 hours, the next truck follows after 0.5 minutes on average; at night and at weekends, the mean gap is 2 to 35 minutes. The manifest of a generated visit is one training manifest drawn as a whole, so that its attributes stay consistent. In the final configuration, both are replayed from the period that is simulated: every simulated truck arrives at the Gate-In time of one observed truck and carries its manifest (Section 5.9.1).

Routing. Every transition with alternatives has its own tree; it predicts the probability that the transition fires when it is enabled. The weights of the enabled transitions are then normalised. Of the 54 transitions, 38 have a tree and 16 a constant weight, mostly those without alternatives. The trees split on the manifest. Figure 4.4 shows the tree for an RMG receive

Component	Model	Target	When it is used	Fitted (models / trees / median leaves; main split features)
Arrival times	Regression tree, distribution per leaf	Minutes to the next Gate In	Arrival tree (generative runs); replayed arrivals in the final configuration	1 / 1 / 7; hour, weekday
Case attributes	Joint empirical sample	Manifest of a visit	Gate In: one training manifest drawn as a whole (replayed manifest in the final configuration)	9 attributes, 59,139 training manifests
Routing	Classification tree per transition	Probability that an enabled transition fires	Every step of the token game; weights normalised over the enabled transitions; stop guard masks forbidden steps	54 / 38 / 5; planned stops, visit complexity, receives, full ratio
Block choice	Classification tree per block	Probability of a block	Every RMG stop (generative runs); observed blocks in the final configuration	26 / 22 / 5; queue length, workload, reefer, visit complexity
Waiting time	Regression tree per resource	Minutes from enabling (lane free) to start	After the resource is chosen; capacity checked over the whole stop at its start	26 / 25 / 6; queue length, workload, hour, weekday
Execution time	Regression tree per activity	Minutes from start to completion	After the start time is known	11 / 8 / 7; other trucks at block, hour, block T18, visit complexity

Table 4.1: Learned components of the final simulation model (ProSiT 1.0.4 on the repaired IM 0.0 state machine, training log). Waiting and execution time are the retrained components of the final configuration (Section 5.9.3); the inventory of every model is in Appendix C.

Setting	Value	Reason
ProSiT version	prosit-pm 1.0.4	Selects between tree and single distribution by CRPS.
`max_depth_tree`	3 (cross-validated 1–3)	Readable trees.
`min_samples_leaf_cv`	50, 100, 200	Default grid.
`attribute_mode`	empirical	Manifests are drawn as a whole, so they stay consistent.
`use_workload_features`	on	Queue length and workload of a resource as features.
`enable_multitasking`, threshold	on, 0.05	A block serves several trucks at once.
Case attributes	9 manifest attributes	Known at Gate In: flow type, containers, planned stops, deliveries, receives, hazardous, reefer, full ratio, visit complexity.
`random_state`	42	Reproducibility.
Replications	seeds 42–46 (E5: 42–51)	Random variation of the simulator.

Table 4.2: Discovery settings of the simulation model.

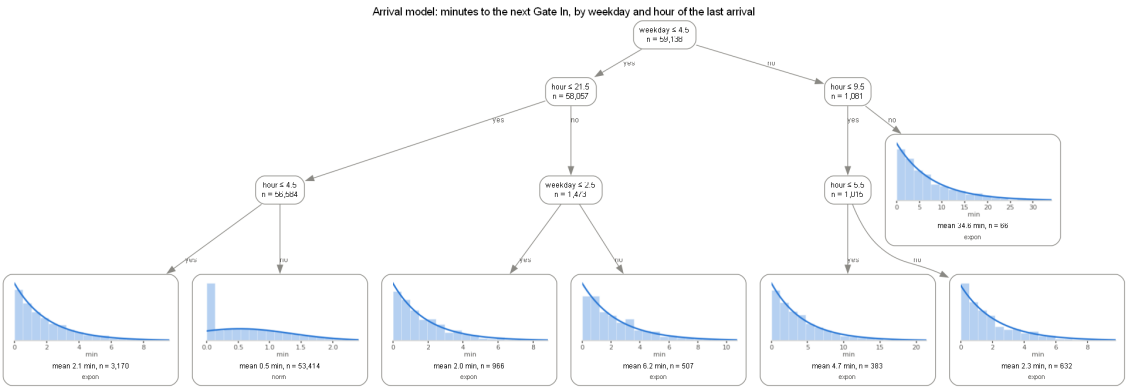


Figure 4.3: Arrival tree: minutes to the next Gate In by weekday (0 = Monday) and hour of the last arrival. Each leaf shows the training values (bars) and the fitted distribution (line), each on its own scale.

as the first stop: for a receive visit with one planned stop and mostly full containers, its probability is 0.88; for a delivery visit it is practically zero. Figure 4.5 compares the first stop that the learned routing chooses with the training log, by flow type: the shares agree within one percentage point.

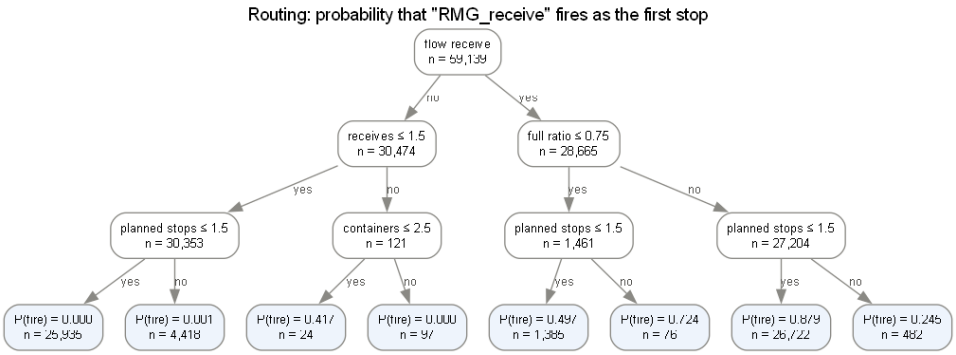


Figure 4.4: Routing tree of the transition `RMG_receive` at the first decision after Gate In: probability that it fires when enabled, by manifest attributes.

Block choice. ProSiT learns one tree per block for the probability that a stop goes there, with the queue length and the workload of the block as features. These trees learned that trucks prefer the block with the longer queue, because the training data count a stop in the queue of its own block (E4, Section 6.5). The final configuration therefore uses the observed block of each stop; generative runs draw the block from the training shares.

Waiting time. One tree per resource predicts the time from the moment a stop can start until it starts. In the final model, this time is measured from the moment a lane of the block

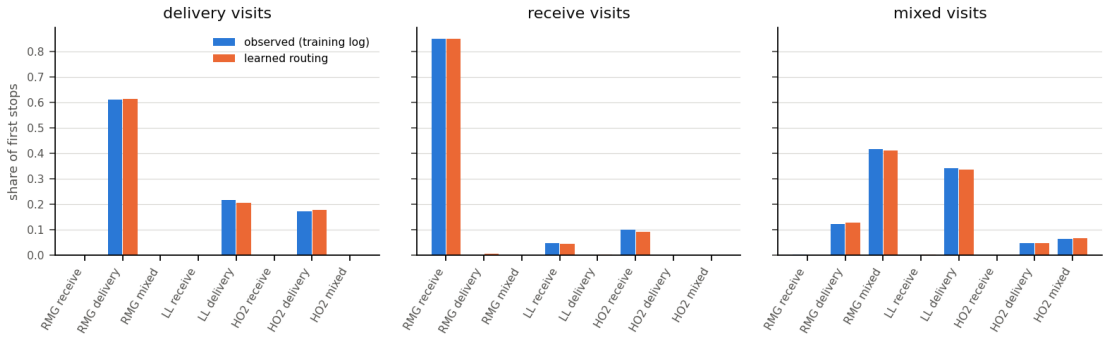


Figure 4.5: First stop of a visit by flow type: share in the training log and probability under the learned routing (normalised weights of the enabled transitions, averaged over 8,000 training manifests).

was free, not from the moment the whole block was free (Section 5.9.3). The trees split mainly on the queue length and the workload of the resource. At block T13 (Figure 4.6), the expected waiting time is 7 minutes with at most one truck in the queue, 12 to 28 minutes with two to six, and more than 50 minutes with seven or more.

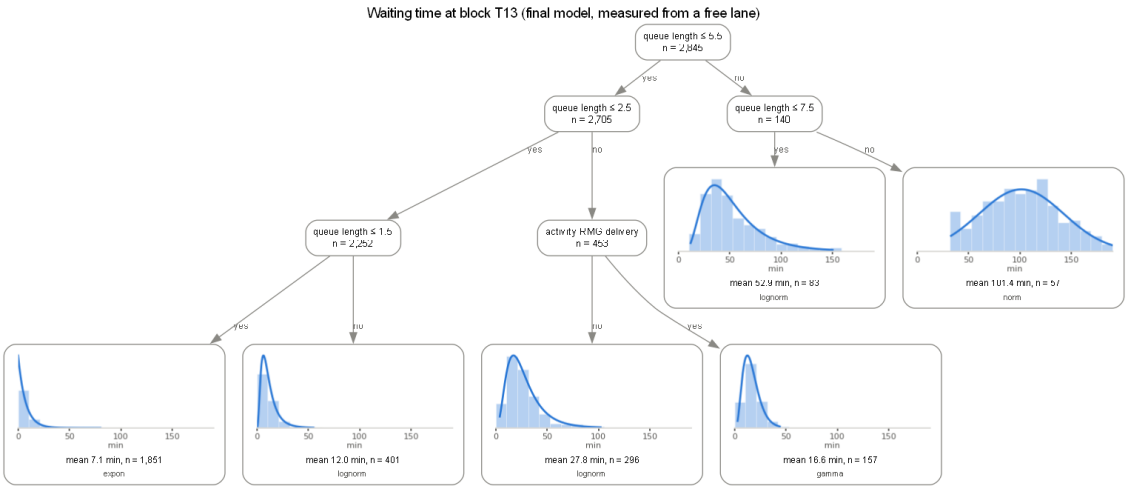


Figure 4.6: Waiting-time tree of block T13 (final model): minutes from a free lane to the start of the stop. All leaves on one scale.

Execution time. One tree per activity predicts the duration of a stop. In the final model, it has one more feature: the number of other trucks at the block when the stop starts (Section 5.9.3). For RMG receives this feature makes the first two splits (Figure 4.7). Without reefer containers, a receive takes 7.5 minutes on average when the truck is alone at the block,

11.7 minutes with one other truck, 16.0 with two and 20.8 with three; block T18 is slower. The tree for RMG deliveries has the same structure.

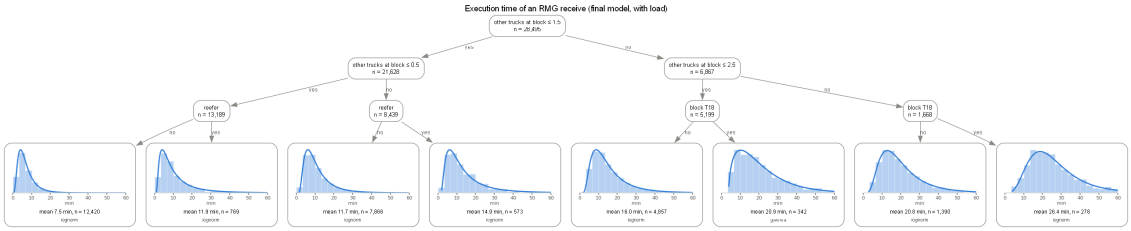


Figure 4.7: Execution-time tree of RMG receives (final model). The first splits are on the number of other trucks at the block when the stop starts.

Calendars and capacities. Table 4.3 lists the capacities and calendars. The capacity of LL and HO2 is discovered from the number of trucks served at the same time; the RMG blocks have three lanes.

Resource	Capacity	Active hours per week (discovered)	Note
Gate In, Gate Out	1	127 / 128	Gate events are instantaneous.
LL (empty-container depot)	23	127	Discovered by multitasking.
HO2 (handling area)	14	126	Discovered by multitasking.
RMG blocks (22)	3	122–168	Three lanes per block (domain); the discovered values were 4 or 5.

Table 4.3: Capacities and calendars. The discovered resource calendars cover 122 to 128 of the 168 hours of a week; they are set to the full week so that trucks already inside are served to completion. The gate calendar still controls admission.

4.4 Execution of a Simulated Visit

The simulation processes the visits in the order of their arrival. For each visit, the loop repeats four steps until Gate Out: determine the enabled transitions of the current state; compute their routing weights and apply the stop guard; draw one transition; for a yard stop, choose the block, determine the start (waiting time, then the earliest time at which fewer than three trucks are at

the block during the whole stop) and draw the execution time. Every event is written with its enabling, start and end time; times are floored to the minute, as in the source systems.

Table 4.4 follows one simulated truck of the final model through these steps, with the values the simulator used and the leaf of each tree that produced them.

Step	Value used by the simulator	Source
Arrival	Gate In Fri 17 Apr 14:46 (replayed from the test log)	–
Manifest	process flow type = receive, n containers = 1, n stops = 1, n receives = 1, has reefer = 0	replayed with the visit
Routing, first decision	RMG receive 0.93; HO2 receive 0.04; LL receive 0.02	one tree per enabled transition, normalised; stop guard: Gate Out masked (0 of 1 stops done)
Drawn step	RMG receive	–
Block	T24 (observed block of the visit)	final configuration: observed blocks
Load at enabling	2 trucks in progress, 1 waiting	recomputed from the simulated log
Waiting time	11 min (enabled 14:46, start 14:57); leaf mean 7.6 min, lognorm	tree of block T24: queue length $\leq$ 2.5, queue length $\leq$ 1.5, workload $\leq$ 2.5
Execution time	31 min (end 15:28); leaf mean 16.0 min, lognorm	tree of RMG receive: other trucks at block $>$ 1.5, other trucks at block $\leq$ 2.5, block T18 no
Exit	Gate Out 15:45; turnaround time 59 min	stop guard: all planned stops done

Table 4.4: One simulated truck of the final model (configuration of E4, seed 42), step by step. The leaf of each tree follows from the features at that moment; the waiting and execution times are drawn from the distribution of that leaf; the waiting time includes any delay until a lane is free (capacity checked over the whole stop at its start).

4.5 Adaptations to the Terminal

The simulation loop of ProSiT is not re-implemented. Its source is read at run time, a few marked lines are changed by exact text replacement, and each change can be switched off. With all changes switched off, the loop produces exactly the same log as ProSiT. Table 4.5 lists what each change does and why it is needed; the evidence is in E3 and E4.

4.6 Chapter Summary

The simulation model combines a repaired control flow, written as a state machine with one data guard, and the probabilistic trees that ProSiT learns for routing, arrivals, blocks, waiting and execution times. Its adaptations to the terminal keep the physical limits: one stop of a truck at a time, as many stops as planned, and at most three trucks at a block. How well the model reproduces the terminal is reported in Section 6.5.

Part	ProSiT 1.0.4	Final model	Why
Control flow	discovered net as given	IM 0.0 restricted to rules 3 and 5, as a state machine	no physically impossible visits; one token per truck (E2, E3)
Number of stops	no guard	stop guard: as many stops as planned	rule 6; ProSiT has no guards (E3)
Resource calendars	discovered (122–128 of 168 hours)	full week for trucks already inside	a stop is not interrupted at the end of a shift
Timestamps	continuous	floored to the minute	as in the source systems
Arrivals and manifests	arrival tree, random training manifest	replayed (final); arrival tree (generative)	separates the model error from the demand (E4)
Block choice	learned trees (queue feature counts the stop itself)	observed block (final); training shares (generative)	the learned rule sends trucks to the longest queue (E4)
Capacity	discovered 4–5; checked when a stop is enabled	3 lanes; checked over the whole stop at its start	at most three trucks at a block (E4)
Waiting time	measured from a completely free block	measured from a free lane	otherwise the learned waiting time falls with the load (E4)
Execution time	no load feature	other trucks at the block as a feature	stops take longer when the block is busy (E4)
Counting	stops at the first finished activity in the schedule	every activity counted	13–14 % of the counts were too low (E4)

Table 4.5: Differences between the ProSiT simulation loop and the final model.

5 Methodology and Experimental Setup

This chapter describes how the claims of this thesis are tested. It states which experiment answers which research question, which inputs are held fixed, which factor is varied, and how the outcome is measured. The concepts behind the methods (Petri nets, rules, conformance measures and ProSiT) are introduced in Chapter 2. The results follow in Chapter 6 in the same order.

5.1 Overview of the Experiments

Table 5.1 maps the experiments to the research questions, and Figure 5.1 shows how E1 to E3 hand their results on. E1 and E2 concern the untimed control-flow model. E3 runs the resulting candidates in the simulator under identical conditions. E4 and E5 build on the configuration selected in E3.

To explain:

- Extend the flow figure by E4 (mechanism audit, acceptance for the intended use) and E5 (what-if demonstration).

5.1.1 Goals of the Experiments

Each experiment has one goal and one deliverable.

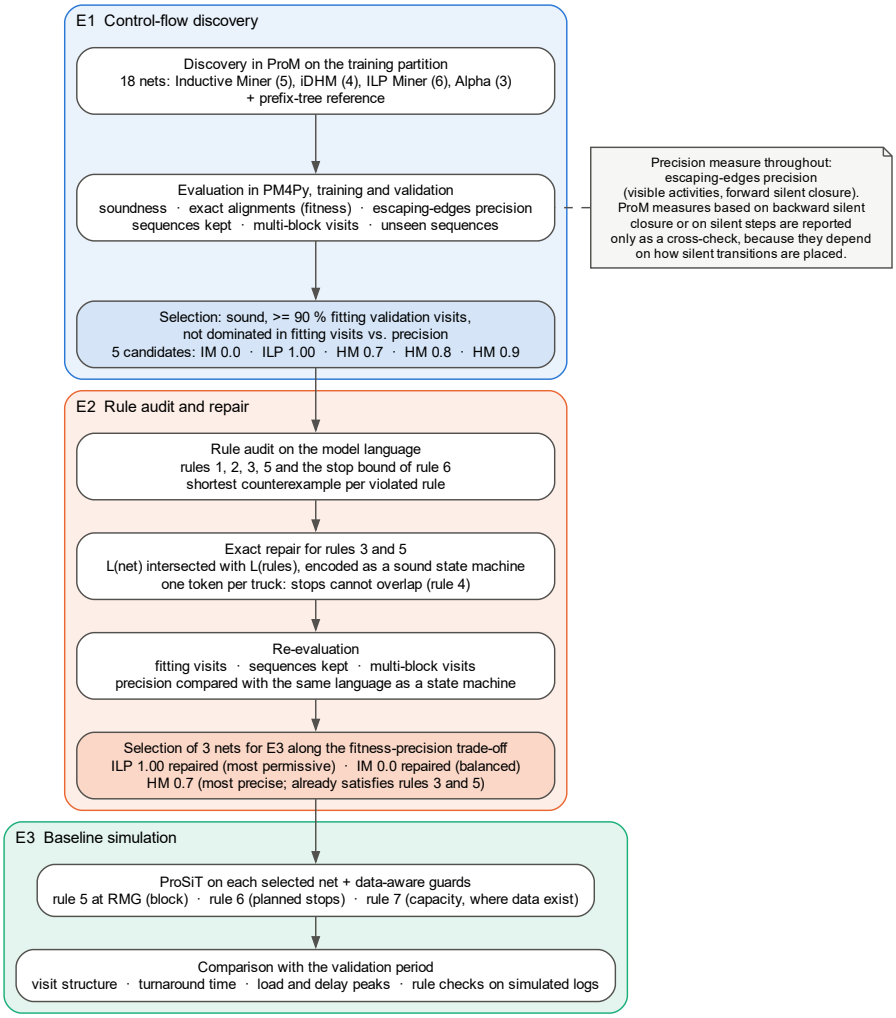


Figure 5.1: Flow of the experiments E1 to E3. Each stage reduces the set of candidate models and passes it on. E4 and E5 continue with the configuration selected in E3.

ID	Experiment	Varied factor and fixed conditions	RQ
E1	Control-flow discovery	Discovery algorithm and parameter. Fixed training log, activity abstraction, event order and evaluation cohorts.	RQ1
E2	Domain-rule audit and targeted repair	Rules enforced on the discovered language. Fixed: the five candidates of E1.	RQ1
E3	Baseline simulation of control-flow candidates	Control-flow model, and for one net the use of repair and guard (2×2). Fixed ProSiT version, hyperparameters, training log, calibration, seeds and horizon.	RQ1, RQ2
E4	Mechanism audit and simulation validity	One component at a time, in the order a truck meets them: demand, block allocation, queue at the block, service under load. Oracle and repaired variants; fixed fitted model, seeds and period.	RQ2
E5	What-if demonstration	Block allocation rule for deliveries (S1), three eligible sets, four robustness runs. Fixed final configuration of E4, replayed trucks, ten paired seeds.	RQ3

Table 5.1: Experiments, varied factors and the research questions they address.

E1 Identify the tested discovery configurations that yield a sound workflow net accepting all three visit groups of the validation log (no RMG, one block, several blocks). Report their fitness, precision and size. Deliverable: the candidates for E2.

E2 Determine which domain rules each candidate violates. Remove these violations without losing any observed sequence that satisfies the rules. Deliverable: repaired, sound models and their rule-compliance evidence.

E3 Measure how the control-flow model alone changes the generated visit structure and turnaround time, with all other simulation conditions fixed, and separate the effect of the repair from that of the data-aware guard. Deliverable: the control-flow model used in E4 and E5.

E4 Quantify how much of the remaining deviation between simulated and observed turnaround time is caused by the demand, the block allocation, the queue at the block and the service time, and whether the simulated turnaround time responds to load at the blocks as

observed. Deliverable: the final simulation configuration, the attribution of the error to components and the decision whether E5 is admissible.

E5 Show what the simulation of E4 can and cannot say about a load-aware block allocation for deliveries: the direction of the effect on turnaround time, the mechanism, who gains and who loses, and whether this holds under the weaknesses found in E4. Deliverable: a conditional what-if statement with its conditions of validity, not a recommendation for the terminal.

E1 to E3 answer RQ1. E3 and E4 answer RQ2. E5 answers RQ3. Every selection uses the training log only; the test log reports the selected configurations.

The experiments do not aim to rank discovery algorithms in general, to optimise the allocation rule, or to show that the simulator is valid for purposes other than the turnaround and peak measures defined in Section 5.3.

5.1.2 Simulation-Informed Model Selection

The control-flow model is not selected on discovery metrics alone. A discovered model is a language of permitted activity sequences. Discovery metrics measure how well this language matches the recorded sequences. A simulator, in contrast, executes the model. It draws routing decisions, assigns resources and schedules time. Behaviour that is permitted by the language but never observed can therefore become frequent in the generated log. For this reason the selection procedure has three stages, applied in the order of E1, E2 and E3.

The first stage screens the discovered models for soundness, coverage of the observed visit groups and precision (E1). The second stage expresses operational requirements as explicit domain rules and checks whether each model's language violates them (E2). The third stage executes the remaining candidates in ProSiT and compares the generated visits with the validation log (E3). A model is selected only if it passes all three stages.

The second and third stages are needed because a sound net with high fitness and precision can still permit behaviour that a truck cannot perform. Two cases are relevant here: a visit without any yard stop, and two stops of the same visit executed at the same time. Both are domain rules (rules 3 and 4 in Table 3.3). E2 checks whether the model language permits them and repairs the model. E3 measures how often they occur in the generated log. For one net, E3 crosses repair and guard in a 2×2 design, so that the contribution of each can be seen.

5.2 Data Partitions and Their Use

To explain:

- One temporal split at Monday 13 April 00:00 (weekly closure). The training log fits every model (discovery, ProSiT parameters, peak thresholds) and is used for every selection (E1, E2, the choice of the final configuration in E4). The test log (13–30 April) only reports the selected configurations; E4 also replays its arrivals and manifests.
- Table: which experiment reads which partition (E1–E5 against train / test).
- Honest disclosure: data from the test period were used in earlier analyses by the author (old split), so the test log is a *retrospective hold-out*, not an untouched confirmatory test.
- Distribution shift: the test period is more congested than the training period (median TTT 33 vs. 29 min, P90 75 vs. 64 min, 25.7 % vs. 10.4 % delay-peak hours, cycle-time distance 5.5 min; Section A.5 still shows the old split). Absolute errors therefore mix model error and drift. Model variants are compared with each other on the same period, and E4 runs the final configuration also on the training period.
- Boundary handling: Gate-In hours that span the train/validation boundary are excluded from hourly statistics (or assigned by rule), stated once here.

5.3 Outcome Measures and Peak Definitions

To explain:

- Primary KPI: turnaround time (TTT) per visit, Gate Out minus Gate In. Reported as median and P90, overall and by flow type. Why P90: delay peaks are a tail phenomenon.
- TTT decomposition (approach, stops, transfer between stops, exit) as diagnostic measures, with the formula. Note that gate events are instantaneous, so the decomposition is complete.
- Arrival peak: Gate-In hour with at least the training P90 of arrivals (165 trucks/h). Delay peak: Gate-In hour with at least 20 visits and median TTT at least the training P90 (37 min). Both thresholds are frozen from training and reused unchanged in every experiment, including the policy scenarios, so that improvement is not normalised away.
- Attribution rule: hourly TTT is attributed to the hour of Gate In (arrival cohort), not to the hour in which the delay occurs.
- Disruption episodes (e.g. 23 March): defined by a rule fixed in advance (delay peak without arrival peak, exit-time dominated). Reported separately *and* included in the all-visits result. Never removed to improve the fit.

- Throughput and backlog: visits completed per day, visits unfinished at the end of the horizon, used as guardrails.

5.4 Simulation Runs, Replications and Uncertainty

To explain:

- Horizon definition: fixed number of visits from the first validation Gate In (current) vs. fixed calendar window. State which one and why. Initial work in progress at the start (empty terminal at 15:50) and how the first hours are treated (warm-up exclusion or reported).
- Unfinished visits at the end of the horizon: counted and reported, never silently dropped from TTT statistics.
- Replications: number per configuration (5 for E3/E4, 10 for E5), seeds, and how the number was chosen (precision of the median and P90 across replications).
- Paired comparison: the same seeds for baseline and scenario. Note that equal seeds do not guarantee common random numbers in ProSiT.
- Reporting: mean over replications with min–max, or a t-based interval over replication means. Replications quantify stochastic variation only, not uncertainty in the fitted parameters.
- Event order in generated logs: reconstructed from start timestamps. Firing sequences are exported for new runs.
- Run hygiene: one new run directory per configuration, hashed inputs, recorded versions (details in the appendix).

5.5 Domain Review

To explain:

- Purpose: an independent check of data meaning, rules and simulation plausibility, which the author's own domain knowledge cannot provide.
- Instrument: structured questionnaire with five parts. A: data semantics. B: rules, including the unobserved two-stop sequences. C: expected values, collected *before* seeing results. D: assessment of the simulation output. E: realistic control levers.
- Use of answers: B confirms or changes the rule register (E2). C provides acceptance tolerances for E3/E4 that were not chosen by the author after seeing the results. D is face validation of E3. E selects the lever for E5.

- Documentation: reviewer role, date, model version shown, answers, disagreements and resulting changes. Summary confirmed by the reviewer.
- Limitation: one reviewer is expert opinion, not proof of validity.

5.6 Control-Flow Discovery (E1)

E1 asks which discovery configuration produces a sound control-flow model that reproduces the recorded truck visits without permitting much additional behaviour. It is designed as a controlled comparison following the recommendations of Rehse et al. [39]: every candidate is fitted on the same cases and evaluated on the same cohorts with the same implementation of each metric.

5.6.1 Discovery Input

The input is the truck-visit event log of Chapter 3. Each case is one completed visit from Gate In to Gate Out. The yard events use eleven generic labels: `Gate In`, `Gate Out`, and `receive`, `delivery` or `mixed` at the three handling areas RMG, LL and HO2. Block identities such as `T21` are kept in the resource attribute and are not part of the activity label.

The sequence of events within a case is taken from the explicit order attribute `case:event:order`. The log is not re-sorted by timestamp. Start and readiness timestamps are not used to infer concurrency during discovery. Before discovery, the training and validation partitions are frozen. The preparation step rejects missing labels, duplicate order positions within a case, cases that appear in more than one partition and partitions that are not chronological by Gate In. It stores SHA-256 hashes of every input. Seven unit tests cover these checks. The XES files used in ProM are written from the frozen partitions by a script that records their hashes, re-reads each file, checks that it reproduces the partition sequence by sequence and checks that OpenXES, the XES library of ProM, imports it. The reserved test partition is not read in E1, E2 or E3.

Population correction. An initial version of the log excluded visits that handled containers at more than one RMG block, because a case-level target-block feature cannot describe several blocks. E1 was first run on this restricted population. The analysis of its results (Section 6.1) showed that the exclusion removes a regular operational pattern. The log was therefore rebuilt with these visits restored, and E1 was repeated on the expanded population. All earlier split memberships were kept. Restored visits were assigned by their Gate In time with the existing boundaries. Table 5.2 lists both populations.

Population	Training	Validation	Test (reserved)
Restricted (at most one RMG block)	57,254	14,314	17,892
Expanded (multiblock visits restored)	58,262	14,625	18,266
of which multiblock visits	1,008	311	374
of which without RMG activity	13,122	3,964	5,117
Distinct activity sequences (expanded)	80	59	–

Table 5.2: Visits per partition for the restricted and the expanded population. Partitions are chronological by Gate In. The validation period starts on 10 April 2026, 15:50, and the test period on 20 April 2026, 18:17. The 184 visits with unknown activity labels remain excluded.

5.6.2 Candidate Algorithms and Parameter Grid

Four discovery families were compared. They rest on different construction principles, so that the comparison covers more than one way of trading fitness against precision [38]. The Inductive Miner recursively splits the log into sequence, choice, parallel and loop operators and guarantees a sound, block-structured net. Its variant Inductive Miner – infrequent (IMf) filters behaviour below a noise threshold at each split. At threshold 0.0 nothing is filtered and every training visit fits the net [29]. This configuration is called IM 0.0 in the following. The interactive Data-aware Heuristic Miner (iDHM) builds a causal net from dependency measures on directly-follows frequencies and converts it into a Petri net [35, 36]. It was used with the dependency and binding heuristics of the Flexible Heuristics Miner [34]. The Alpha algorithm and its extensions Alpha+ and Alpha++ derive places directly from the ordering relations of the log and have no noise handling [30, 31, 32]. The ILP Miner constructs places as solutions of integer linear programmes over the prefix-closed log, based on the theory of regions [37]. Its sequence encoding filter removes infrequent branches of the prefix tree: a branch is kept only if its frequency reaches at least $(1 - t)$ times that of its most frequent sibling, where t is the filter threshold. Unlike the noise threshold of IMf, a higher threshold therefore filters less; $t = 1$ keeps all behaviour. Split Miner was not part of the comparison.

All nets were discovered in ProM Lite 1.4 [25] on the complete training partition of the expanded population, with the activity label (`concept:name`) as classifier, and exported as PNML files with initial and final marking. iDHM was run without data attributes, so its nets contain control flow only. Table 5.3 lists the configurations. They are a bounded exploration, not an exhaustive tuning. The grid sizes differ between families. Conclusions therefore concern the tested configurations and not the algorithm families in general.

Family	ProM plugin (package)	Settings	Nets
Inductive Miner	Mine Petri net with Inductive Miner (InductiveMinerDeprecated 6.10.72), variant IMf	noise threshold 0.0, 0.1, 0.2, 0.3, 0.4	5
Heuristics	Interactive Data-aware Heuristic Miner (DataAwareCNetMiner 6.9.193); dependency heuristic: Flexible Heuristics Miner; bindings heuristic: nearest activity (FHM)	frequency 0.0 with dependency 0.7, 0.8 and 0.9; frequency 0.1 with dependency 0.9; bindings 0.1; "accepted connected" on (default); long-term dependencies and "all tasks connected" off; no data attributes; Petri net output	4
ILP Miner	ILP-Based Process Discovery (HybridILPMiner 6.10.154), advanced configuration; LP solver LPSolve 5.5	sequence encoding filter threshold 0.0, 0.25, 0.55, 0.75, 0.95, 1.0; p/t net; objective: Parikh values weighted by absolute frequencies; constraints: theory of regions, empty after completion, no trivial regions; sink place added; two variables per event; one place per causal relation, causal graph from the Midi miner with default settings	6
Alpha	Alpha Miner (AlphaMiner 6.9.78)	variants Alpha, Alpha+, Alpha++	3

Table 5.3: Discovery configurations of E1, all run in ProM Lite 1.4 on the training partition of the expanded population (58,262 visits). The iDHM frequency threshold is relative to the number of visits: at 0.1, only the 7 of 60 directly-follows relations that occur at least 5,827 times are used. Because no data attributes were selected, iDHM discovers no conditional dependencies, and the conditional heuristic (C4.5 with F1 score, condition threshold 0.5) has no effect.

Prefix reference. In addition to the miners, a prefix-tree reference model is built [80]. Every observed prefix of a training sequence becomes one place, and every observed extension one transition. The language of this net equals exactly the set of training sequences. It is not a simulation candidate. It shows how well the validation visits can be explained by doing nothing more than memorising the training sequences, and thus calibrates the precision values of the discovered models.

5.6.3 Evaluation Protocol

All nets are evaluated with PM4Py 2.7.22.3 [26] on the complete training log and the complete test log (Section 5.2). The evaluation is scripted. Each metric runs in a separate process with a time limit of 1,800 seconds. A failure or timeout is recorded with its status and never replaced

by a score of zero.

1. **Soundness.** Each net is checked with WOFLAN [81]. Only sound workflow nets are executable in ProSiT without deadlocks or leftover tokens.

2. **Fitness.** Alignments are computed with the state-equation A* search of PM4Py [59, 57]. A log or model move on a visible transition costs 10,000, a move on a silent transition 1 and a synchronous move 0. Fitness counts only the visible deviations and therefore equals fitness under the standard cost function. The small cost of silent moves only selects, among equally good alignments, the one with the fewest silent moves. The search is exact. Fitness and precision are defined on optimal alignments, and for candidate nets with at most 45 transitions and visits of at most six events the exact search is fast. Techniques that give up optimality to scale to large models [82] are therefore not needed. Every alignment must end in the final marking. A visit *fits* if its alignment contains no deviation. The share of fitting visits and the alignment-based log fitness are reported.

3. **Precision.** Escaping-edges precision on alignments [60, 61]. For every prefix of a log trace, the visible activities that the net enables after the prefix, including those reached through silent transitions, are compared with the activities that follow the prefix in the log. Each prefix is weighted by its frequency. Precision is computed separately on the training and the test log. PM4Py's helper that follows silent transitions after a prefix can miss silent paths and overstated the precision of IM 0.0. It is replaced by an exhaustive search over all markings reachable by silent transitions. The values were checked against an independent implementation of the same definition (identical for all nets) and against ProM's ETConformance plugin, which gives the same values for nets without silent transitions.

4. **Coverage** (Section 2.4). The share of fitting visits; the share of the distinct training sequences that the net accepts; the share of fitting visits per visit group (without RMG, one RMG block, two RMG blocks); and, on the test log, the number of sequences that do not occur in training and that the net accepts. Because the visits follow a long-tailed distribution, log fitness and the share of fitting visits are dominated by a few frequent single-stop sequences; the share of accepted sequences and the coverage per group show what a net loses in the tail. For the visits that a net rejects, the share of the truck-minutes (sum of the turnaround times), of the visits above the training P90 turnaround time and

of the multi-block visits is reported as well. This states the lost behaviour in terms of the KPIs that the simulation has to reproduce (Section 5.3).

5. **Generalisation.** Token-based generalisation [58] on the training log. It is reported for completeness but not used for the selection: it rewards nets that use few transitions often and barely separates the nets. Generalisation is checked directly on the test log instead.

6. **Structure.** Places, visible and silent transitions, and arc-degree simplicity.

For unsound nets, alignment values are reported as a diagnostic only, and only if the final marking is reachable. ProM's own replayer and its precision plugins serve as cross-checks (Section 6.1). No automatic winner is computed. All selection decisions use training values; the test log is only reported. A net is eligible for E2 if it is sound and fits at least 90 % of the training visits. Among the eligible nets, those that are not dominated in the share of fitting visits and precision are candidates. The coverage measures are reported for every candidate and show which observed behaviour it rejects. Two nets are added as references: the most permissive net that accepts all sequences, and a filtered Inductive Miner net.

5.7 Domain-Rule Audit and Targeted Repair (E2)

E2 asks which behaviour without justification the five E1 candidates permit, and whether it can be removed without losing observed behaviour. Discovery algorithms do not know what a truck can physically do. A model can reach high fitness and precision and still allow a visit without any stop, or two stops of the same truck at the same time. The inputs are the five nets chosen in E1 (Section 5.6.3): the candidates IM 0.0, HM 0.7 and HM 0.9, and the two references ILP 1.00 (most permissive) and IMf 0.3 (filtered).

5.7.1 Rule Types and Where They Are Enforced

The rules are listed in Table 3.3 (Section 3.4). They are of three types, and each type is handled where it can be expressed exactly (Section 2.3.3).

- **Sequence rules** speak only about activity labels: rules 1, 2 and 3, the LL and HO2 part of rule 5, and the upper bound of four stops in rule 6. They are checked and enforced on the language of the net.

- **The time rule** (rule 4, stops never overlap) speaks about timestamps. It cannot be checked on the untimed language. A state machine holds one token per visit, so the

simulator cannot start two stops of the same truck at once. Rule 4 is therefore enforced by the encoding of the repaired net and checked in the generated log.

- **Data rules** depend on attributes: the RMG part of rule 5 (same block), the equality of rule 6 (as many stops as planned) and rule 7 (load capacity). They are guards in the simulator or are checked in the generated log (Section 5.8).

5.7.2 Audit at Three Levels

Each rule is checked at three levels, because conformance at one level does not imply conformance at the next.

1. **Recorded log.** Does every training and test visit satisfy the rule? A violation here would question the rule or the data.

2. **Model language.** Does the net permit a sequence that breaks the rule? Each sequence rule is written as a small automaton over activity labels (a monitor). The reachability graph of the net, with silent steps resolved, is explored together with each monitor. If the net can complete a run that the monitor rejects, the shortest such sequence is reported as a counterexample. The check is exact for bounded nets, including nets with loops [43, 44]. In addition, the number of distinct permitted sequences is counted for one to four stops and compared with the observed sequences.

3. **Generated log.** Does the simulation produce visits that break the rule? This level is part of E3 and is the only level for rule 4 and the data rules.

5.7.3 Targeted Repair

A candidate that breaks a sequence rule is repaired by restricting its language to the sequences that satisfy the rule. The repair has three steps.

1. The reachability graph of the net, with silent steps resolved, is combined with the monitors of the selected rules. Only sequences that the net and all selected rules allow remain.

2. States with the same possible continuations are merged (automaton minimisation). This gives the smallest automaton with the same language.

3. The automaton is written as a Petri net in the simplest way: each state becomes one place and each edge one visible transition. The result is a state machine workflow net without silent transitions (Section 2.3.1).

The repair is exact. Every sequence of the original net that satisfies the rules is kept, and no other sequence is added. The result is sound by construction. The procedure runs automatically and writes a new PNML file; the original net is not changed. It is closest to Dixit et al. [50], who edit a discovered model until it satisfies Declare constraints. The difference is that the restricted model is built exactly instead of being searched among random edits. In the classification of Schuster et al. [47], the repair uses explicit, declarative knowledge a posteriori and is fully automated.

The repair is applied for rule 3 and the LL and HO2 part of rule 5. Rules 1 and 2 hold in every candidate. The bound of rule 6 is broken by every candidate with a loop. It is not encoded in the net, for two reasons. The stop guard in E3 enforces the stronger equality with the planned stops, which implies the bound. And encoding the bound would make the state machine much larger, so ProSiT would learn each routing tree from fewer observations.

What the repair means in Petri net terms The same restriction can also be built by hand on the original net. Two local patterns are enough (Figures 5.2 and 5.3). They show what the exact repair does, and they make the repaired behaviour readable in ProM or WoPeD.

Two patterns repair the net locally instead of replacing it by a state machine. Rule 5 links two exclusive choices in different parts of the net by a decision token (Figure 5.2). Places that connect distant choices in this way are known as non-local or long-distance dependencies [32, 54]. Rule 3 requires that at least one of several independent optional stops occurs. This is an inclusive choice, the multi-choice pattern [83], and in a plain Petri net it needs additional control places. A marker records whether a stop has happened (Figure 5.3). The marker alone can block a visit in which every stop is skipped, so the complete construction also forbids skipping the last open stop. Both repaired nets were checked against the exact repair: they accept exactly the same sequences, are sound and hold at most one token per place.

Control for the change of representation The state machine encoding changes two things at once: the language and the representation. Precision depends on the representation as well (Section 2.4). To separate the two, the unrepaired language of each candidate is also written as a state machine. The precision of a repaired net is compared with this *same-language* state machine, not with the original net.

Acceptance and selection A repaired net must be sound and must keep its coverage (Section 2.4): every training sequence that it accepted before and that satisfies the rules is still accepted, and the share of fitting training and test visits does not change. Its precision is compared with the same-language state machine. Four repaired nets are taken to E3: ILP 1.00 (most

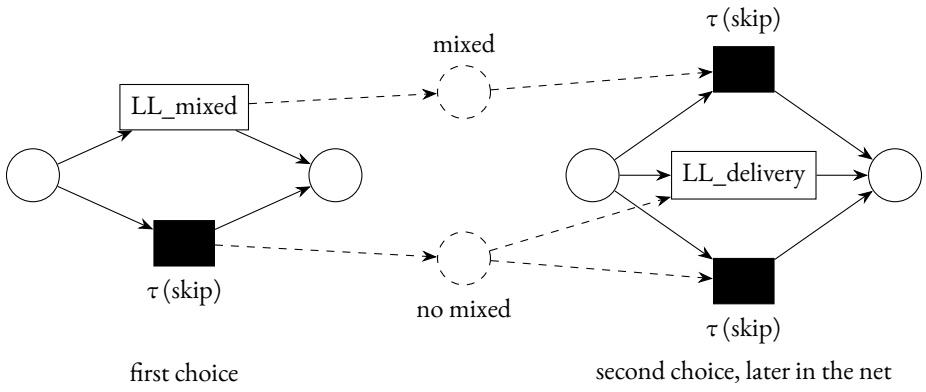


Figure 5.2: Repair pattern for rule 5: one exclusive choice across two parts of the net. The first choice leaves a token that records the decision (dashed places and arcs). `LL_delivery` can only fire if `LL_mixed` was skipped. The skip of the second choice exists twice, once for each token, so no control token remains at the end. `LL_receive` is connected like `LL_delivery`, and HO2 is repaired in the same way. Per handling area, the pattern adds two places and one silent transition.

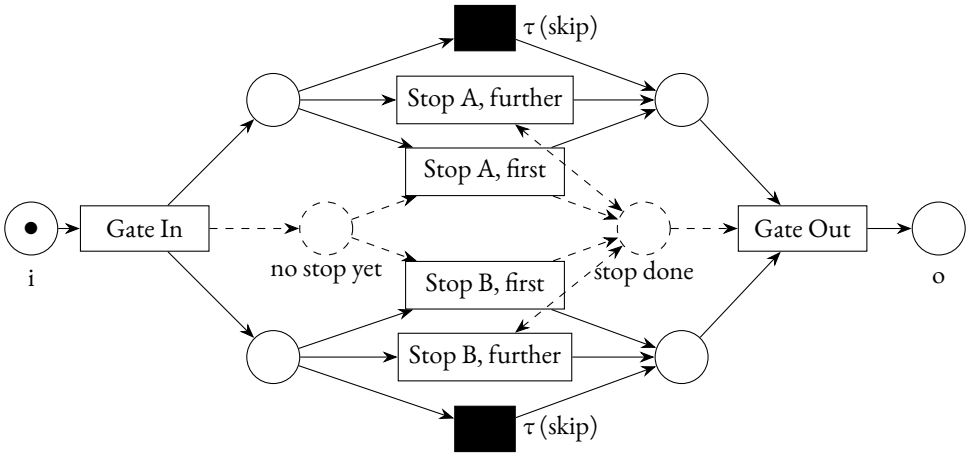


Figure 5.3: Marker logic for rule 3 (at least one stop), shown for two optional stops in parallel. Gate In puts a token into *no stop yet*. Each stop exists twice: the first stop moves the token to *stop done*, a further stop only reads it (double arrow). Gate Out needs *stop done*, so a visit cannot leave without a stop. If both stops are skipped, however, the token stays in *no stop yet* and the visit can never finish. The complete construction therefore also forbids skipping the last open stop, with one more control place per stop (not shown).

permissive), IM 0.0 (all observed sequences with the highest precision), HM 0.7 (most precise net with at least 99 % fitting visits) and IMf 0.3 (high precision with low coverage). HM 0.9 is left out because it is close to HM 0.7 and accepts fewer sequences. Taking IMf 0.3 to E3 tests whether a net with high precision but low coverage is a good basis for simulation.

5.8 Baseline Simulation of Control-Flow Candidates (E3)

E3 asks how the choice of control-flow model affects the generated visits when everything else in the simulation is held constant. It also establishes the baseline against which the mechanisms in E4 are analysed.

5.8.1 Candidates and Contrast

Three control-flow models are simulated. All three are state machines without silent transitions.

- **ILP 1.00 repaired** (12 places, 82 transitions): the most permissive net that keeps all observed sequences.
- **IM 0.0 repaired** (10 places, 54 transitions): the balanced candidate.
- **HM 0.7** (9 places, 29 transitions): the most precise candidate. It already satisfies rules 3 and 5, so the repair does not change its language; it is used in its state machine encoding.

The title of this thesis puts the physical constraints at the centre. To show what they contribute, IM 0.0 is also simulated in a 2×2 design (Table 5.4). One factor is the net: the original Inductive Miner net (28 places, 38 transitions, 27 of them silent) or the repaired state machine. The other factor is the stop guard: off or on. The naive cell, original net without guard, is what a direct use of discovery and ProSiT produces.

	Guard off	**Guard on**
Original IM 0.0	naive: discovery and ProSiT as they are	rules only as data guards
Repaired IM 0.0	rules only in the net	rules in the net and as guards (main configuration)

Table 5.4: 2×2 contrast for IM 0.0 in E3. All four cells use the same ProSiT settings and seeds.

5.8.2 ProSiT Configuration and CTB Adaptations

ProSiT learns its parameters separately for each net, with the settings in Table 5.5. Identical settings therefore do not imply identical fitted routing, timing or resource models. E3 compares the complete fitted simulators.

The validation log is used only to set the number of simulated visits and the start time. It is not used to fit any parameter.

Stop guard ProSiT has no guards (Section 2.6.5). The stop guard adds one data-aware guard to the simulation loop. Before each routing draw, it compares the number of stops the visit has done with the number of planned stops from the manifest, which is known at Gate In. While stops are missing, Gate Out gets the weight zero. Once all planned stops are done, every yard transition gets the weight zero. Silent transitions are not affected. If all enabled transitions then have weight zero, Gate Out is fired when the planned stops are done (forced exit). Otherwise the learned weights are used, and the case is counted as a fallback. The guard leaves the net and the learned models unchanged. With the guard switched off, the loop reproduces ProSiT exactly.

To explain:

- Add a worked example of one simulated truck with the real fitted values of IM 0.0 repaired: arrival leaf, attributes, routing weights at each step (with and without guard), chosen block, waiting and execution draws, resulting timestamps. Every step must match the implementation (Section 2.6.3). This answers the committee's request for concrete parameters.
- Full trees and calendars in Appendix C.

5.8.3 Comparison Measures

The generated logs are compared with the test log on three groups of measures. The coverage of each net (E1) sets a limit for this comparison: a sequence that the net rejects can never appear in the generated log.

1. **Visit structure.** Share of visits without a yard stop (rule 3), share of multi-stop visits with overlapping stops (rule 4), violations of rule 5, distribution of the number of stops and agreement with the planned stops (rule 6), share of visits whose sequence never occurs in training, and share of RMG visits that use more than one block.

2. **Turnaround time.** TTT is reported as mean, median and P90, in total and by flow type (delivery, receive, mixed). It is decomposed into the approach from Gate In to the start

Element	Setting	Reason
Software	ProSiT 1.0.4, PM4Py 2.7.22.3, Python 3.11	Version 1.0.4 selects trees by CRPS; 1.0.3 almost always chose single distributions (Section 2.6.2).
Training data	training partition, 58,262 visits	Test partition not read.
`max_depth_tree`	3 (default 5)	Trees stay readable; cross-validation can still choose depth 1 or 2.
`min_samples_leaf_cv`	50, 100, 200	Default.
`attribute_mode`	empirical	Keeps consistent manifests, e.g. flow type, numbers of stops, deliveries and receives. Independent draws would create impossible visits.
Case attributes	flow type, containers, planned stops, deliveries, receives, hazardous and reefer flags, full ratio, visit complexity, demand and utilisation at Gate In	All known at Gate In (Section 3.2.6).
`use_workload_features`	on	Delay grows with load (Section 3.3), so resource choice and waiting time should see the workload.
`enable_multitasking`, threshold	on, 0.05	A block serves several trucks at once. Without multitasking every resource has capacity one.
RMG capacity	at most 3 per block	Three handover lanes per block; the discovered capacities were 4 or 5.
Arrivals	native arrival tree on hour and weekday	The learned profile is part of the baseline; alternatives follow in E4a.
Resource calendars	open for the full week	Trucks already inside are served to completion.
Timestamps	floored to the minute	As in the source systems.
Simulation loop	native ProSiT loop: learned block choice, capacity checked at enabling	Changes to the loop are tested in E4c.
Stop guard	on (off in two cells of Table 5.4)	Enforces rule 6 and thereby rule 3 (see below).
`random_state`	42	Fixed for reproducibility.
Replications	seeds 42–46, 14,625 visits each, from the first validation Gate In	Same number of visits as the validation partition.

Table 5.5: Fixed simulation conditions in E3.

of the first stop, the stop intervals, the transfer between stops, and the exit from the end of the last stop to Gate Out.

3. **Delay peaks.** Share of Gate-In hours with at least 20 visits whose median TTT is at least 37 minutes, and the number of hours with at least 165 arrivals. Both thresholds are the 90th percentiles of the training hours (Section 5.3).

For the stop guard, the number of changed draws, forced exits and fallbacks per run is reported.

The simulated log records start and completion timestamps. The order of events within a simulated visit is reconstructed from the start timestamps; for equal timestamps the completion time decides. Each measure is reported as the mean over the five replications with its minimum and maximum. Five replications quantify the random variation of the simulator. They do not capture uncertainty in the fitted parameters.

5.8.4 Additional Checks and Selection Criterion

To explain:

- Validator self-check: apply the evaluation script to the observed validation log itself. All rule-violation shares must be zero and all statistics must equal the directly computed values.
- Block-level checks: load per RMG block (observed vs. simulated), repeated visits to the same block, and the RMG part of rule 5 (mixed followed by delivery or receive at the *same* block) in the generated log.
- Selection criterion, fixed before reading the results: the selected candidate must produce no violations of rules 3, 4 and 5 and the smallest distance to the observed stop distribution. TTT is reported but not used for selection, because it is dominated by arrivals and timing (see E4).
- Adequacy tolerances for TTT taken from the domain review (part C6), e.g. median within ±x min, P90 within ±y min.

5.9 Mechanism Audit and Simulation Validity (E4)

E4 answers RQ2: to what extent does the ProSiT simulation reproduce the turnaround time of trucks and its response to load at the capacity-limited yard blocks, and which model components cause the remaining error? The base is the main configuration of E3: IM 0.0 repaired, stop guard, manifest attributes, ProSiT 1.0.4 with trees and workload features.

A simulation model is valid only for an intended use [71]. Here the intended use is E5, which moves trucks from busy to less busy blocks. A model can only show the effect of this

change if trucks wait longer and are served more slowly when a block is busy, as at the terminal (Section 3.3). E4 therefore checks two things: whether the simulated turnaround time has the right level, and whether it is produced by the right mechanism under load.

5.9.1 Evaluation Design

The evaluation follows the three levels of Sargent [71] (Section 2.5).

1. **Verification.** Does the simulator do what it should? The modified simulation loop must reproduce ProSiT exactly when all changes are switched off. A retrained component must be identical to the fitted one when its training data are not changed. Replayed arrivals must be exact, and the generated logs must respect the capacity and the domain rules. The number of activities in progress at a resource, which ProSiT uses for the workload features and for the capacity check, is recomputed from the generated log and compared with the value the simulator used.

2. **Conceptual validity.** Are the assumptions of each component plausible for the terminal? Every learned model is inspected (split features, leaves), and every mechanism of the simulation loop is compared with the physical process (Table 5.7).

3. **Operational validity.** Is the output accurate enough for the intended use? Generated logs are compared with the observed test log on four perspectives (Table 5.6).

Reference values A distance between a generated and an observed log has no meaning on its own. Two observed periods also differ. The differences between real periods are therefore used as the reference: between the three weeks of the test period, the cycle-time distance is 1.7 to 3.1 minutes and the median TTT differs by up to 3 minutes; between the training and the test period, the cycle-time distance is 5.5 minutes, and median and P90 differ by 4 and 11 minutes. The load profiles of P4 are much more stable: the profile gaps between real periods are at most 2.0 minutes for the approach and 1.0 minute for the stop duration. The load response is thus a stable property of the terminal, while the level of the turnaround time follows the demand of the period.

Two weaknesses of the usual comparison Özdemir et al. point out two weaknesses of comparing a generated log with a later test log [74]. First, the comparison mixes the error of the model with the change of the process between the periods. Second, distances based on the Wasserstein distance can hide temporal patterns. E4 addresses both. Most configurations replay the observed Gate-In times and manifests of the test period (*conditional replay*), so that

Perspective	Measures	Why
P1 Turnaround time	TTT median, P90 and mean; cycle-time distance (CTD, Wasserstein-1 in minutes); median TTT by flow type	The KPI of the thesis [73].
P2 Where the time is spent	Mean approach (Gate In to first stop), stop time, transfer between stops and exit; the four means add up to the mean TTT	Locates the error in the visit.
P3 Physical constraints	Trucks in the terminal; share of RMG stops at the busiest block and correlation of block shares with the observed ones; share of RMG stops that start with three or more other trucks at the block; largest number of trucks at one block at once; visits without stop, overlapping stops, visits with more than four stops	The constraints that the title refers to.
P4 Response to load	Spearman correlation of hourly arrivals and hourly median TTT; median TTT by trucks in the terminal; median approach to an RMG first stop by the trucks at that block at Gate In; median RMG stop duration by the other trucks at the block at its start; share of delay-peak hours	The property that E5 needs.
Standard BPS measures	3-gram distance of the activity sequences (NGD-3) and case-arrival distance (CAR)	Comparability with the ProSiT evaluation [11, 73].

Table 5.6: Measures of operational validity in E4. For each load profile in P4, the *profile gap* is the mean absolute difference of the simulated medians to the observed ones over the bins.

the simulated trucks are the observed ones and the demand is not part of the error. The final configuration is also run on the training period, which separates the error of the model from the change between periods. And the medians, quantiles and load profiles of P1 and P4 are reported next to the Wasserstein-based distances.

5.9.2 Error Attribution by Component Substitution

A truck meets the components of the simulator in a fixed order: the demand (arrival time and manifest), the block allocation, the queue at the block (capacity and waiting time) and the service (execution time); the exit follows. The error of a component is measured by replacing it and observing how much of the error disappears. Two kinds of replacement are used.

- **Oracle.** The component is replaced by the observed values of the test period: replayed arrivals and manifests, or the observed block of each RMG stop. This is trace-driven simulation [64]. It shows the error that remains if the component were perfect.

- **Repair.** The component stays learned, but a documented defect of its training data or of the simulation loop is corrected.

The components are examined in the order in which the truck meets them. Each step keeps the components before it at their oracle values, so that only one component varies at a time. The generative alternatives, for example learned instead of observed blocks, are reported as deviations from this chain. After the final configuration has been chosen, each of its changes is reversed alone (*leave one out*), which shows whether every change is needed. Finally, the error of the mean TTT is split exactly into the errors of approach, stop time, transfer and exit.

Retraining protocol Retrained components use ProSiT's own feature table, discovery functions, hyperparameters and random state. Only the stated change of the training data differs. As a check, every component is first retrained on unchanged data; it must equal the fitted component exactly.

5.9.3 Mechanisms and Design Decisions

Table 5.7 lists the mechanisms that are examined, how ProSiT implements them and why a variant is tested.

Design decisions

- **One fitted model.** All configurations start from the same fitted model. A configuration replaces single components, so that differences come from the replaced component and not from a new fit.

- **Observed blocks for the queue and service steps.** The share of each block changes from week to week, so no model learned on the training period can reproduce the block loads of the test period. To judge the queue and the service at a block, the observed block of each stop is used. The error of the block allocation itself is measured separately (E4b).

- **Where the queue comes from.** A queue can arise in the simulation, when a stop has to wait for a free lane, or it can be part of the learned waiting time. Using both counts the waiting twice. The two consistent combinations are compared: waiting from the moment a lane became free together with capacity at start, and the whole delay before the stop without capacity limit. The second treats waiting like an extraneous delay that the model learns from data [84].

- **Number of lanes.** A block has three lanes and serves at most three trucks at once. This is a physical limit. The log nevertheless shows RMG stops that start while three other stops at the same block are still in progress (5.6 % of the RMG stops in training, 8.1 %

Component	ProSiT 1.0.4	Terminal	Variants tested
Demand	Arrival gap drawn from a tree on hour and weekday of the previous arrival; manifest drawn as a random training tuple, independent of the time	Strong daily profile; up to 255 arrivals per hour	Hour-of-week tree from training; replay of the observed arrivals and manifests (oracle)
Block allocation	Learned choice among free blocks, with the queue length of each block as feature. In training, a stop that starts after it was enabled is counted in the queue of its own block, so the label enters the feature	The block follows the container location and the yard plan; the share of each block changes from week to week (correlation 0.27 to 0.62 between weeks)	Learned choice retrained without this count; training shares of the blocks; observed block (oracle)
Capacity	Checked at the enabling time	A block serves at most three trucks at once (three lanes). The log shows a fourth truck at 8.1 % of the RMG stops, but only for a short overlap (see the design decisions)	Checked over the whole stop interval at its actual start, with three lanes; four lanes only as a sensitivity run
Waiting	Waiting time measured from the moment the block was last completely free (single server)	Several trucks are served side by side; the approach grows from 6 to 16–24 minutes with three or more trucks at the block	Waiting measured from the moment a lane became free, with capacity at start (queue arises in the simulation); or the whole time from enabling to start, without capacity limit (queue learned from data)
Service	Execution time without load features	The RMG stop takes 6, 9, 14 and 20 minutes (median) with 0, 1, 2 and 3 other trucks at the block	Execution time with the number of other trucks at the block as an additional feature

Table 5.7: Mechanisms examined in E4. Observed values from the test log; the training log gives the same profiles within one to two minutes.

in the test period). For these stops, the first of the other three ends after a median of 3 minutes (in 68 to 73 % of the cases within 5 minutes). The fourth truck is therefore already registered at the block while it waits for a lane: the recorded stop interval is the time at the block, not the time in the lane. The simulation uses three lanes. Four lanes, as the log suggests, are run only as a sensitivity check and are not admissible. For the same reason, the load profiles of P_4 are compared only for the states that three lanes allow (up to two other trucks at the start of a stop, up to three trucks at the block at Gate In).

- **Load in the service time.** At the terminal, part of the time at the block is spent waiting for the crane, which serves several trucks. The stop interval therefore grows with the number of trucks at the block. ProSiT does not offer load features for execution times. The extension adds one feature, the number of other stops in progress at the block when the stop starts, both in training and in the simulation. With capacity at start, the simulated count is at most the number of lanes minus one.

- **Exact counting.** ProSiT counts the activities in progress at a resource by walking backwards through its schedule and stops at the first activity that has already ended. This is exact only if the schedule is sorted by end time. The simulation loop stores activities in the order in which they are processed, so some counts are too low. In training, ProSiT counts exactly. The verification of E4 found this after the first runs; all configurations from E4b on are therefore also run with an exact count (marked x), and the rule for the final configuration is applied to these runs.

- **Unchanged parts.** Routing, stop guard, case attributes, calendars and the exit are not changed. The exit does not depend on the load (12.4 minutes on average in both periods).

5.9.4 Experiments

Table 5.8 lists the configurations. All use five seeds (42–46) and simulate the 32,014 visits of the test period from its first Gate In, except the runs of E4f, which simulate the 59,139 visits of the training period. A configuration whose queue grows without bound is simulated only for the first day of the test period (2,393 visits) and compared with the observed visits of that day.

Choice of the final configuration The rule is fixed before the results are read. As in E1 and E2, the selection uses training data only: the configurations of E4c and E4d (with exact counting) are also run on the training period, with the replayed training visits and their

Run	Demand	Blocks	Capacity	Waiting	Service	Question
E4a Demand						
a0	ProSiT tree	learned	3, enabling	ProSiT	ProSiT	E3 baseline
a1	hour of week	learned	3, enabling	ProSiT	ProSiT	Does a realistic arrival profile help?
a2	replay	learned	3, enabling	ProSiT	ProSiT	Error without demand error
E4b Block allocation						
b1	replay	learned, no self-count	3, enabling	ProSiT	ProSiT	Is the self-count the cause of the concentration?
b2	replay	training shares	3, enabling	ProSiT	ProSiT	Load-independent allocation
b3	replay	observed	3, enabling	ProSiT	ProSiT	Error without allocation error
E4c Queue at the block						
c1	replay	observed	3, start	ProSiT	ProSiT	Capacity over the whole stop
c2	replay	observed	3, start	no self-count	ProSiT	Consistent queue feature
c3	replay	observed	3, start	lane free	ProSiT	Queue arises in the simulation
c4	replay	observed	4, start	lane free	ProSiT	Four lanes, as in the log (sensitivity)
c5	replay	observed	none	whole delay	ProSiT	Queue learned from data (sensitivity)
E4d Service under load						
d3–d5	as c3–c5				with load	Does load-dependent service reproduce the stop durations?
E4e Final configuration (chosen by the rule below)						
e–	each change of the final configuration reversed alone					Is every change needed?
eblk	final configuration with training shares or a learned block choice					What is lost without the observed blocks?
e×	final configuration with arrivals compressed or stretched (intensity 0.8, 1.2)					Does TTT rise with load?
eG	final mechanisms with hour-of-week or ProSiT arrivals and training block shares					Fully generative accuracy
E4f Training period (in-sample)						
T	the admissible c1x–c3x and d3x on the training period (replay of the training visits)					Selection of the final configuration
fo	final configuration on the training period					Model error or change between periods?

Table 5.8: Configurations of E4. ProSiT: component as fitted in E3. No self-count: retrained without counting the stop in its own queue. Lane free: waiting measured from the moment a lane became free. Whole delay: waiting measured from enabling to start. Every configuration from b3 on, and the baseline a0, is also run with an exact count of the activities in

observed blocks, and compared with the training log. The test log is only reported. A configuration is admissible if it produces no rule violations and never more than three trucks at a block at once. Among the admissible ones, the configuration with the smallest sum of the three profile gaps of P4 is chosen; ties are broken by the cycle-time distance. The load response comes first because it is what E5 needs and because it is stable between periods. The first version of this rule allowed four trucks at a block, as observed in the log. It was corrected to three after the domain confirmed the limit and before the training runs were read; the configurations with four lanes or without a capacity limit are therefore not admissible and are not run on the training period.

Acceptance criteria for the intended use Fixed before the results are read, with thresholds taken from the differences between real periods:

1. **Physical admissibility:** no rule violations; at most three trucks at a block at once.

2. **Load response:** Spearman correlation of hourly arrivals and median TTT at least 0.49 (the lowest weekly value of the test period); profile gaps at most 2.0 minutes for the approach, 1.0 minute for the stop duration and 5.75 minutes for the TTT by trucks in the terminal (the largest gaps between real weeks).

3. **Extreme condition:** TTT rises when the arrival intensity rises (0.8, 1.0, 1.2).

4. **Level:** cycle-time distance at most 3.1 minutes (within the variation between weeks) or at most 5.5 minutes (within the change between periods); reported for the conditional and the fully generative configuration.

E5 is admissible for a quantitative estimate if criteria 1 to 3 hold for the final configuration under replay; otherwise it is run only as a what-if demonstration (Section 5.10). Criterion 4 states how far the absolute values can be trusted.

5.10 What-If Demonstration (E5)

E5 answers RQ3: to what extent can the simulation be used to analyse a load-aware block allocation for deliveries under the physical limits of the yard blocks, and what does it indicate about the effect on truck turnaround time? The purpose is not a recommendation for the terminal. E5 shows what the simulation validated in E4 can and cannot say about one change of the process.

Status after E4 The final configuration of E4 passed the physical and the extreme-condition criterion, but missed the load-response criterion narrowly (Section 5.9.4). By the rule fixed in E4, the simulation is therefore not admissible for a quantitative estimate of a policy effect. E5 is run as a *what-if demonstration*: it states the direction of an effect and the mechanism behind it, and only if both are robust to the weaknesses found in E4. Effect sizes are reported as values of the model under the conditions of E4, not as estimates for the terminal.

Scenario S1 The observed waiting before the first RMG stop grows from 6 to 16 minutes when three trucks are already at the block, while other blocks often have free lanes (Section 3.3). The storage block of an export delivery is assigned by yard planning, whereas a receive must go to the block where its container is. S1 therefore moves only RMG deliveries. When a delivery becomes possible, the simulator counts the stops at its block that are in progress or already scheduled. If this number is three or more, all lanes are taken, and the delivery is moved to the eligible block with the fewest such stops below three; ties go to the nearest block. If no block is eligible, the delivery stays. Receives and mixed stops keep their observed block. The rule draws no random numbers. Three variants differ in the eligible blocks: the observed block ± 1, ± 2, or any block (upper bound).

Fixed conditions The baseline is the final configuration of E4 on the test period: replayed arrivals and manifests, observed blocks, capacity at start with three lanes, lane-free waiting, load-dependent service and exact counting. In the baseline, S1 only records the deliveries it would move (dry run); as a verification, the baseline log must equal the E4 log for the same seed. Baseline and variants use the same ten seeds (42–51). Equal seeds give common random numbers only until the first moved stop, so all differences are computed per seed and then averaged.

Measures

- **Z1 Effect.** Paired difference (S1 minus baseline) of the mean, median and P90 of the TTT of all visits, of the visits with a moved delivery, and of the visits that arrive with 100 or more trucks in the terminal; mean over the ten seeds with a 95 % t-interval.

- **Z2 Mechanism.** Approach to the first RMG stop, share of RMG stops that start with two other trucks at the block (a full block), and waiting for a free lane per RMG stop.

- **Z3 Distribution.** Paired change of the TTT per visit, by group: visits with a moved delivery, other visits at the blocks that gave deliveries away, visits at the blocks that received them, and all other visits; and by flow type.

- **Z4 Costs and physical limits.** Share of RMG deliveries moved, distance in block numbers, largest number of trucks at a block (at most three) and rule violations. Not modelled: driving distance, later rehandling and the constraints of the yard plan.

The share of delay-peak hours is reported but not interpreted, because the simulation produces almost none (E4).

Robustness (Z5) The ±2 variant is repeated, each time with its own baseline, under four changes that address the weaknesses of E4: (a) service without load, the lower bound of the slowdown at a busy block; (b) a stronger slowdown, in which the service model sees one more truck whenever at least one other truck is at the block; (c) arrivals compressed to an intensity of 1.2, where the simulated level matched the observed one; (d) fully generative arrivals and blocks (hour-of-week arrivals and training block shares).

Rules for the claims Fixed before the runs:

1. A direction is stated only if the 95 % interval of the paired difference for the affected visits excludes zero in the main variants and the sign is the same in all four robustness runs. If the sign changes, the finding is that the simulation is not adequate for this question.

2. A mechanism is stated only if the approach and the share of RMG stops that start at a full block change in the expected direction.

3. Interpretation of the variants: an effect already at ±1 or ±2 means that a local rule is enough in the model; an effect only with any block means that it needs unlimited flexibility; no effect means that the model shows no lever; better affected visits but worse other visits mean a shift of waiting, not a reduction.

5.11 Threats Addressed by the Design

To explain:

- Short table: threat, design response, remaining limitation. Discussed further in Chapter 7.
- Representation vs. language change in the repair: control model with the same language as a state machine (E2).
- Information leakage through case attributes: only attributes from the planned visit manifest known at Gate In.
- Double counting of waiting: the queue either arises in the simulation (lane-free waiting with capacity at start) or is learned (whole delay without capacity), never both (E4).

- Selection on the test log: every selection uses the training log; the test log only reports the selected configuration; rules fixed before the results were read (E4: one documented correction of the capacity rule).
- Drift between periods: model variants compared on the same period.
- Single seed or few replications: replication counts and intervals.
- Author as domain source: external domain review.

5.12 Chapter Summary

To explain:

- E1–E3 select an executable control flow in three stages. E4 explains the remaining error and decides whether a quantitative policy study is admissible. E5 demonstrates what-if use with one lever, fixed demand and robustness runs, and states only what is robust. All thresholds and criteria are fixed before the corresponding results are read.

6 Results

This chapter reports the results in the order of the experiments in Chapter 5. Section 6.1 compares the discovered control-flow models (E1). Section 6.2 audits the selected model against the domain rules and evaluates its repair (E2). Section 6.3 executes the candidates in ProSiT and compares the generated visits with the validation log (E3). Each section ends with a short conclusion that states what the evidence supports and what it does not.

6.1 Discovery Comparison and Model Choice (E1)

To explain:

- OUTDATED: the text of this section after the figures still describes the old split (training, validation and test partitions, 80 training sequences). Rewrite it with the split at 13 April (Section 5.2). Already current: Figures 6.1, 6.2 and 6.3, Tables 6.1, 6.2 and B.1, and the summary in Section 6.4.

The eighteen nets of Table 5.3 were discovered in ProM on the training partition of the expanded population (58,262 visits, 80 distinct activity sequences). They were evaluated with PM4Py as described in Section 5.6.3 on the training partition and on the validation partition (14,625 visits, 59 sequences, six of which never occur in training).

ProM's own replayer serves as a cross-check of the alignments. It was run on the validation partition for the five Inductive Miner nets with the standard cost function and penalised improper completion, using the splitting replayer, an A* search that guarantees optimal align-

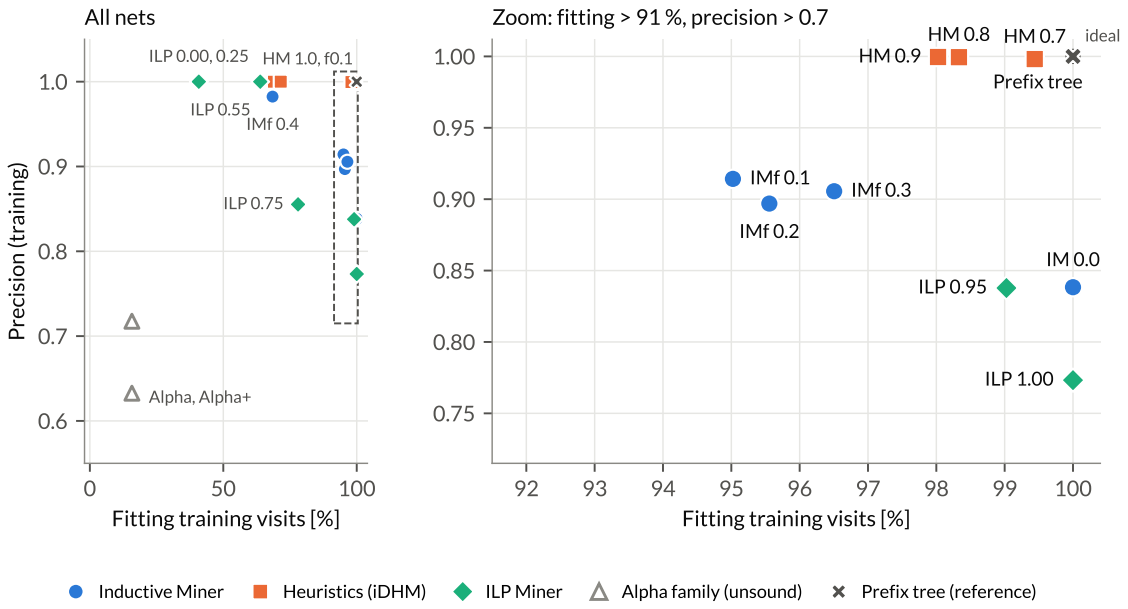


Figure 6.1: Share of fitting visits and precision of the discovered nets on the training log, which is the basis for the selection. Left: all nets. Right: the nets with more than 91 % fitting visits and a precision above 0.7. Hollow markers: unsound nets, values for diagnosis only. The prefix tree accepts exactly the training sequences and serves as a reference.

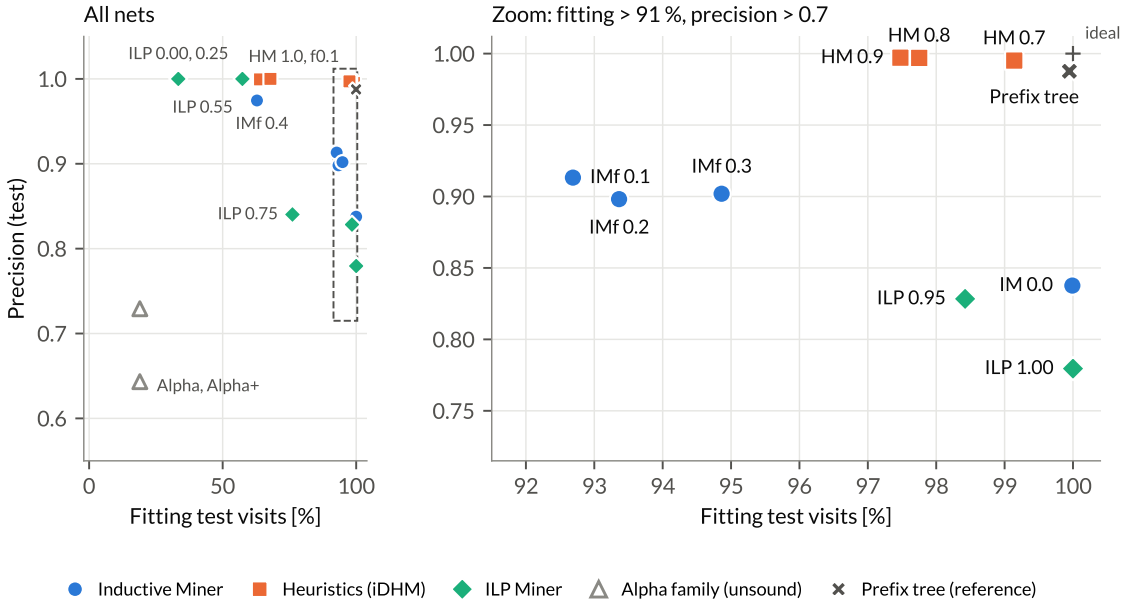


Figure 6.2: The same nets on the test log (13 to 30 April 2026). The test log was not used for any decision. Precision hardly changes. IM 0.0 and ILP 1.00 still fit (almost) all visits, while the nets that accept few sequences lose more fitting visits than on the training log (IMf 0.3: 96.5 % → 94.9 %).

Net	Fitting visits [%]	Precision	Seq. accepted	Multi-block accepted [%]	Reason
IM 0.0	100.00	0.838	82 of 82	100.00	non-dominated; the only net that accepts all training sequences with precision above 0.8
HM 0.7	99.44	0.998	38 of 82	99.22	non-dominated; most fitting visits among the Heuristics nets
HM 0.9	98.02	1.000	27 of 82	98.45	non-dominated; most precise sound net with at least 90 % fitting visits
ILP 1.00	100.00	0.773	82 of 82	100.00	reference: most permissive net (accepts every sequence of stops); dominated by IM 0.0
IMf 0.3	96.50	0.906	19 of 82	1.16	reference: filtered Inductive Miner; dominated by the Heuristics nets

Table 6.1: Nets taken to the rule audit and repair (E2), with their values on the training log. Selection on training values only: sound, at least 90 % fitting visits and not dominated in fitting visits and precision. HM 0.8 is also non-dominated but lies between HM 0.7 and HM 0.9 and is left out. ILP 1.00 and IMf 0.3 are added as references for a permissive and a filtered net.

			Share of the training ... [%]			Mean	Again
Visits	Seq.	visits	truck-minutes	visits > P90	multi-block	TTT	in test
Sequences seen once	21	0.04	0.09	0.24	0.78	91	7 of 21
Sequences with fewer than 10 visits	47	0.19	0.42	1.23	2.62	82	25 of 47
All but the 15 most frequent sequences	67	1.23	2.00	4.02	10.95	59	44 of 67
Rejected by IMf 0.3	63	3.50	5.46	9.99	98.84	57	41 of 63
Rejected by HM 0.9	55	1.98	2.89	4.73	1.55	54	38 of 55
Rejected by HM 0.7	44	0.56	0.96	2.17	0.78	62	29 of 44
Rejected by IM 0.0	0	0.00	0.00	0.00	0.00	–	–

Table 6.2: What the rare sequences and the sequences a net rejects stand for (training log, 59,139 visits, 82 sequences). Truck-minutes: sum of the turnaround times. Visits > P90: visits with a turnaround time above the training P90 of 64 minutes. Multi-block: visits to two or more RMG blocks (1.75 % of the visits). Mean TTT in minutes (all visits: 37). Again in test: sequences that occur again in the test log.

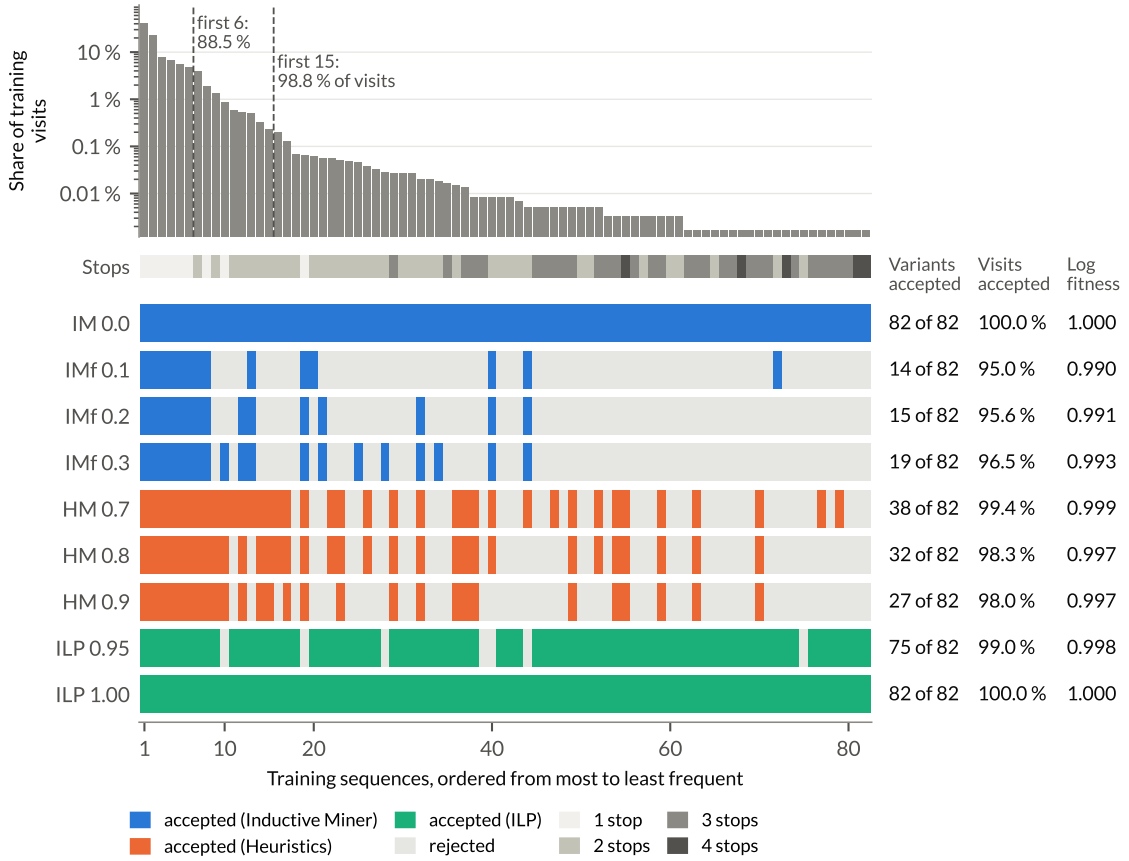


Figure 6.3: Which training sequences each sound candidate accepts. Top: share of the training visits per sequence (logarithmic scale), from the most to the least frequent sequence. Middle: number of stops of each sequence. Bottom: one row per net; a coloured cell means that the net accepts the complete sequence. The six most frequent sequences cover 88.5 % of the visits, so a net that accepts few sequences can still fit most visits.

ments and is the default of ProM's replay plugin [85, 86] (Table 6.3). ProM and PM4Py return the same number of fitting visits and the same total deviation cost for every net, and the same mean trace fitness to the two decimals that ProM reports. They differ only in how equally cheap deviations are split between log and model moves, and in the number of silent moves, which cost nothing in ProM.

ProM's precision plugins compute measures that differ from the escaping-edges precision used here. *Measure Precision/Generalization* (package PNetAlignmentAnalysis) reports lower values: 0.664 instead of 0.897 for IMf 0.2 on the training partition. By default, the plugin counts every distinct sequence once and averages a ratio per event. It also counts an activity as enabled if it becomes enabled after silent transitions are fired backwards, provided their output

Net	Fitting visits ProoM	share	Log / model moves ProoM	PM4Py	Mean trace fitness	Precision (PM4Py)
IM 0.0	14,624	99.99 %	1 / 0	1 / 0	1.00	0.869
IMf 0.1	13,535	92.55 %	1,105 / 0	881 / 224	0.99	0.914
IMf 0.2	13,667	93.45 %	966 / 0	742 / 224	0.99	0.896
IMf 0.3	13,897	95.02 %	735 / 0	735 / 0	0.99	0.900
IMf 0.4	9,249	63.24 %	1,181 / 4,730	1,177 / 4,734	0.93	0.973

Table 6.3: ProM replay of the validation partition (14,625 visits) on the five Inductive Miner nets. PM4Py returns the same number of fitting visits and the same total deviation cost (log plus model moves) for every net. Mean trace fitness as reported by ProM. The last column adds the PM4Py precision: the filtered nets reject 5.0 to 7.5 % of the visits for at most 0.045 more precision, and IMf 0.4 rejects more than a third.

places are marked, whether or not they occurred in the trace. This re-opens choices that the visit has already passed. Before `Gate Out`, for example, IMf 0.2 is credited with ten enabled activities instead of one. This step penalises silent transitions rather than additional behaviour. The precision of the Multi-perspective Process Explorer [46] weights by frequency and has no backward step, but it treats every silent transition as a step of its own. It reports 0.958 for IMf 0.2 and 0.612 for IM 0.0. For IM 0.0, the value depends on the alignment: the same measure, computed on two other optimal alignments that contain the same 749,230 moves in a different order, gives 0.534 and 0.394. In nets with silent transitions in concurrent branches, this measure therefore reflects where the alignment places the silent moves rather than the behaviour of the net. The escaping-edges precision avoids both effects because it compares visible activities only. All precision values in this thesis are computed with PM4Py, and values of different tools are not mixed.

6.1.1 Overall Comparison

Table 6.4 reports soundness, size, fitness and precision. Figure 6.4 places every net in the plane of fitting validation visits and validation precision.

None of the three Alpha nets is sound. Alpha++ cannot reach its final marking at all, and Alpha and Alpha+ accept only 16 % of the visits. By the protocol they are not executable in the simulator and are not considered further. All Inductive Miner, Heuristics and ILP nets are sound.

Without filtering, IM 0.0 accepts every training visit and all but one validation visit. Its validation precision is 0.869. Filtering trades this coverage for little precision. With thresholds

Net	Sound	Places / trans. (silent)	Fitting visits train	Fitting visits val.	Fitness val.	Precision train	Precision val.
IM 0.0	yes	28 / 38 (27)	100 %	99.99 %	0.9998	0.903	0.869
IMf 0.1	yes	10 / 17 (6)	95.07 %	92.55 %	0.9851	0.914	0.914
IMf 0.2	yes	7 / 15 (4)	95.58 %	93.45 %	0.9870	0.897	0.896
IMf 0.3	yes	11 / 18 (7)	96.51 %	95.02 %	0.9901	0.906	0.900
IMf 0.4	yes	11 / 17 (6)	68.48 %	63.24 %	0.9338	0.982	0.973
HM 0.7	yes	22 / 45 (34)	99.45 %	99.02 %	0.9983	0.998	0.994
HM 0.8	yes	22 / 42 (31)	98.34 %	97.69 %	0.9961	0.999	0.997
HM 0.9	yes	22 / 37 (26)	97.98 %	97.29 %	0.9955	1.000	0.998
HM 0.9, freq. 0.1	yes	10 / 11 (6)	71.62 %	66.65 %	0.9066	1.000	1.000
ILP 0.00, 0.25	yes	6 / 5 (2)	40.64 %	34.86 %	0.7935	1.000	1.000
ILP 0.55	yes	6 / 6 (2)	63.84 %	58.42 %	0.8770	1.000	1.000
ILP 0.75	yes	8 / 7 (2)	78.16 %	73.80 %	0.9481	0.855	0.842
ILP 0.95	yes	12 / 11 (2)	99.05 %	98.37 %	0.9967	0.838	0.805
ILP 1.00	yes	24 / 13 (2)	100 %	100 %	1.0000	0.773	0.747
Prefix tree[b]	yes	164 / 242 (80)	100 %	99.96 %	0.9999	1.000	0.969
Alpha[a]	no	4 / 11 (0)	15.91 %	16.05 %	0.8625	0.633	0.612
Alpha+[a]	no	5 / 11 (0)	15.91 %	16.05 %	0.8625	0.718	0.693
Alpha++	no	22 / 11 (0)	–	–	–	–	–

Table 6.4: Discovered nets on the training partition (58,262 visits) and the validation partition (14,625 visits). Fitting visits: share of visits whose alignment has no deviation. Fitness: alignment-based log fitness. IM: Inductive Miner, IMf with the noise threshold given; HM: iDHM with the dependency threshold given; ILP: ILP Miner with the filter threshold given (0.00 and 0.25 yield the same net). [a]Unsound, values reported as a diagnostic only. Alpha++ has no reachable final marking, so no alignment exists. [b]Reference net that accepts exactly the training sequences (Section 5.6.2).

0.1 to 0.3, precision rises to at most 0.914, while 5.0 to 7.5 % of the validation visits are rejected. The effect is not monotone: IMf 0.2 is less precise than IMf 0.1. At 0.4, precision reaches 0.973, but more than a third of the visits are rejected.

The Heuristics nets reach almost perfect precision (0.994 to 0.998) and still accept 97 to 99 % of the validation visits. On these two measures, HM 0.7 is better than every filtered Inductive Miner net. The net with frequency threshold 0.1 contains only 5 of the 11 activities (the gates, `RMG_delivery`, `RMG_receive` and `LL_delivery`) and rejects every visit with any other stop.

For the ILP Miner, a higher filter threshold keeps more behaviour. At 0.00 and 0.25 the miner returns the same net, which permits only the most frequent sequence, `Gate In > RMG_receive > Gate Out` (40.6 % of the training visits). At 0.55 the net adds `RMG_delivery` as an alternative stop. These nets reach a precision of 1.000 because they permit only one or two sequences. Without filtering, ILP 1.00 accepts every training and every validation visit, including the visit that IM 0.0 rejects. Its validation precision of 0.747 is the lowest among the sound nets. At 0.95, 1.6 % of the validation visits are rejected, and precision rises only to 0.805.

In Figure 6.4, only ILP 1.00, IM 0.0 and the Heuristics nets lie on the front of non-dominated nets: ILP 1.00 and IM 0.0 with the highest fitness, the Heuristics nets with the highest precision. ILP 1.00 is ahead of IM 0.0 by a single validation visit and behind it by 0.122 in precision.

The prefix tree only memorises the 80 training sequences. It accepts 99.96 % of the validation visits and rejects exactly the six visits whose sequence never occurs in training. Its precision is 1.000 on the training partition by construction and 0.969 on the validation partition, where only 53 of the 80 training sequences recur. Together with ILP 1.00, it marks the range for nets that keep every training sequence: 0.969 if nothing else is permitted, 0.747 if every sequence of stops is permitted. IM 0.0 lies between the two at 0.869. The Heuristics nets are more precise than the prefix tree, which is only possible because they reject observed sequences.

For all sound nets with more than 90 % fitting visits, the share of fitting visits drops by at most 2.6 percentage points from training to validation. The validation period contains few new sequences: 53 of its 59 sequences also occur in training.

6.1.2 What the Aggregate Measures Hide

The share of fitting visits weights each sequence by its frequency. A net can therefore reject many observed sequences and still fit 99 % of the visits. Table 6.5 counts the distinct sequences that each net accepts and the acceptance of multi-block visits, the visit group that the restricted population had excluded (Section 5.6.1).

The Heuristics nets reach their precision by excluding observed behaviour. HM 0.7 accepts

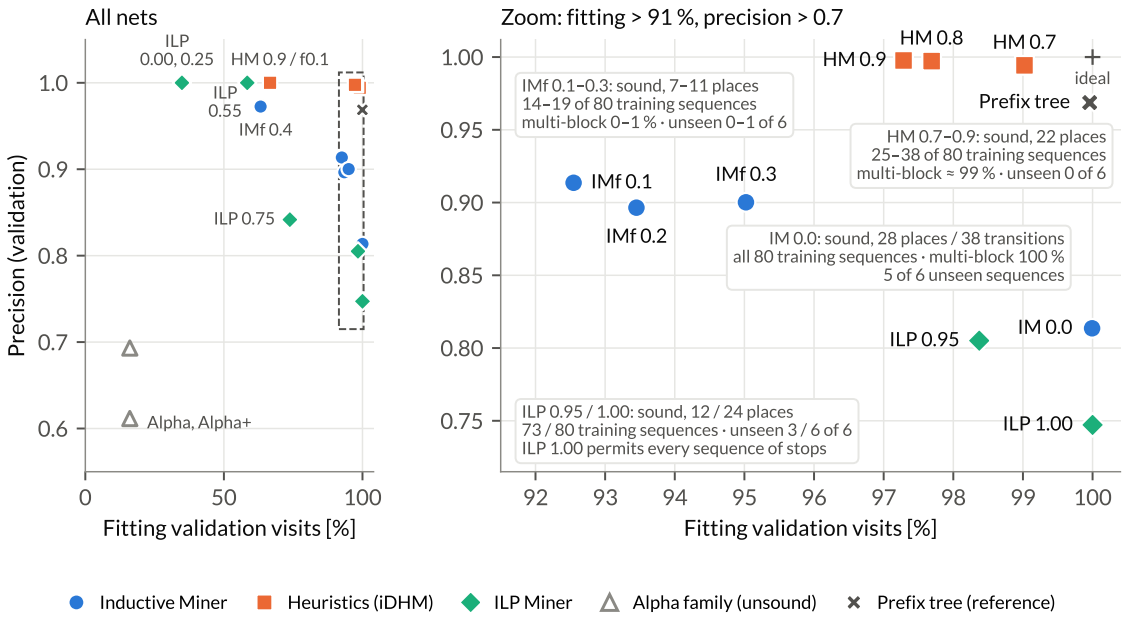


Figure 6.4: Fitness–precision trade-off of the discovered nets on the validation partition. Left: all nets with an alignment; the dashed rectangle marks the zoomed region. Right: the candidates with more than 91 % fitting visits. Boxes summarise soundness, size and the coverage measures of Table 6.5. Hollow grey markers: the unsound Alpha nets. The prefix tree is a reference, not a candidate.

99.45 % of the training visits but only 38 of the 80 training sequences. The 42 rejected sequences cover 322 visits, and all of them contain at least two stops. Of the 13 pairs of stops that occur in both orders in training, HM 0.7 accepts nine in one order only and four in neither order. Examples are `RMG_delivery > LL_delivery` (40 visits) and `RMG_receive > LL_receive` (36 visits). The Heuristics nets accept none of the six validation sequences that are new in this period. A net of this kind can only reproduce the most frequent order of every combination of stops.

The filtered Inductive Miner nets lose a regular operational pattern. IMf 0.1 accepts none of the multi-block visits, IMf 0.2 to 0.4 accept about 1 %. The most frequent multi-block sequence, `Gate In > RMG_delivery > RMG_receive > Gate Out`, is a double cycle whose two containers are stored in different blocks (799 training visits). With 1.7 % of the training visits, the multi-block group falls below every tested noise threshold. These visits are also the longest ones, with a median turnaround time of 50 minutes in validation against 33 minutes for visits with one RMG block. IMf 0.1 additionally rejects the single-stop visit `Gate In > HO2_mixed > Gate Out` (486 training visits).

IM 0.0 keeps every training sequence and every multi-block visit, and it accepts five of the

Net	Training sequences accepted (of 80)	Validation sequences accepted (of 59)	Unseen validation sequences accepted (of 6)	Multi-block visits accepted, train / val.
IM 0.0	80	58	5	100 / 100 %
IMf 0.1	14	13	0	0 / 0 %
IMf 0.2	15	16	1	1.2 / 1.0 %
IMf 0.3	19	19	1	1.2 / 1.0 %
IMf 0.4	7	7	0	1.2 / 1.0 %
HM 0.7	38	28	0	99.4 / 99.0 %
HM 0.8	32	24	0	98.9 / 98.7 %
HM 0.9	25	19	0	98.6 / 98.7 %
HM 0.9, freq. 0.1	3	3	0	0 / 0 %
ILP 0.00, 0.25	1	1	0	0 / 0 %
ILP 0.55	2	2	0	0 / 0 %
ILP 0.75	21	14	0	97.6 / 98.1 %
ILP 0.95	73	50	3	100 / 100 %
ILP 1.00	80	59	6	100 / 100 %
Prefix tree	80	53	0	100 / 99.4 %

Table 6.5: Behaviour preserved by the sound nets and the prefix-tree reference. A sequence is accepted if the net replays it without deviation. Multi-block visits handle containers at more than one RMG block (1,008 training and 311 validation visits).

six new validation sequences. The one rejected validation visit, `Gate In > HO2_mixed > RMG_delivery > Gate Out`, follows a sequence that never occurs in training.

ILP 1.00 keeps all observed behaviour as well, but only because it permits every sequence of stops between `Gate In` and `Gate Out`. Counted on its reachability graph, it accepts 9^k sequences with k stops, that is, 6,561 sequences with four stops. IM 0.0 accepts 2,009 sequences with four stops, and three occur in training. ILP 1.00 is therefore a flower model for the stops. Its precision of 0.747 is the value that a net reaches when it keeps the gates but imposes no constraint on the stops.

Precision is weighted by frequency as well. On the training partition, IM 0.0 is as precise as the filtered nets when weighted by visits (0.903 against 0.897 to 0.914). With every training sequence counted once, its precision falls to 0.665, while IMf 0.1 to 0.3 reach 0.874 to 0.895

and the Heuristics nets 0.970 to 0.989. The additional behaviour of IM 0.0 is therefore concentrated in the rare sequences with several stops. The filtered nets are not penalised in the same way for the sequences they reject, because these enter the measure only through their alignments.

6.1.3 The Selected Net

Figure 6.5 shows IM 0.0. After `Gate In` and an optional `LL_mixed`, a silent AND-split opens five branches: `RMG_mixed`, `RMG_receive`, `RMG_delivery`, a choice between `LL_receive` and `LL_delivery`, and a choice between `HO2_receive` and `HO2_delivery`. An optional `HO2_mixed` follows the last three branches. Every branch can be skipped, and all activities except the LL activities can be repeated through silent loops. The parallel operators express that the order of the stops is free. Two consequences follow directly from the structure. The net permits the visit `Gate In > Gate Out` without any handling, and it permits any number of stops, for example five consecutive `RMG_delivery` events. Neither occurs in the data. These are the additional behaviour behind the precision of 0.869, and they are examined in E2.

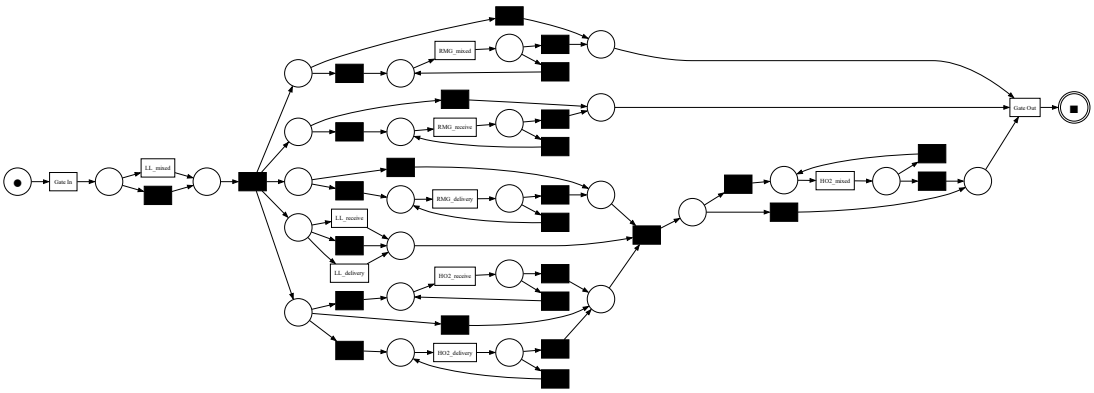


Figure 6.5: Selected net IM 0.0 (Inductive Miner without filtering, discovered in ProM on the full training partition): 28 places, 11 visible and 27 silent transitions (black). The silent AND-split after `Gate In` lets the branches occur in any order. Every branch can be skipped.

6.1.4 Conclusion of E1

Two sound nets preserve all observed behaviour, that is, every training sequence and every visit group including the multi-block visits: IM 0.0 and ILP 1.00. ILP 1.00 does so by permitting every sequence of stops, so its precision is that of a flower model for the stops. IM 0.0 restricts the order and repetition of the stops (2,009 instead of 6,561 sequences with four stops), is clearly

more precise (0.869 against 0.747) and rejects a single validation visit, whose sequence never occurs in training. IM 0.0 is retained for E2. All other candidates obtain their higher precision by rejecting observed behaviour. Noise filtering in IMf and ILP removes rare but regular visit types first, and the Heuristics nets restrict most combinations of stops to a single order. For a simulation model this matters more than the aggregate scores suggest: a sequence that the net rejects can never be generated.

E1 does not show that IM 0.0 is a good simulation model. Its precision of 0.869 reflects behaviour that the data do not contain, including visits without handling and unbounded repetitions. The results also concern the tested configurations only, not the algorithm families in general. Finally, the comparison depends on the population. On the restricted population without multi-block visits, the loss of this visit group by the filtered nets would not have been visible.

6.2 Domain-Rule Audit and Repair (E2)

To explain:

- OUTDATED (PM4Py era, single base model IM0, rule names R1–R12). Rewrite with the E2 run on the five E1 candidates on the split at 13 April (process_discovery/outputs/runs/e2_split13apr_20261106) and the rule numbers 1–7 of Table 3.3. Already current: Table 6.6 and the summary in Section 6.4.

Net	Rules violated	Fitting visits [%]	Precision (train / test) original	same language	repaired	Sequences with 4 stops original	repaired
IM 0.0	3, 5, 6 (bound) → 6 (bound)	100.00 / 99.99	0.838 / 0.838	0.838 / 0.838	0.888 / 0.889	2009	1583
HM 0.7	6 (bound) → 6 (bound)	99.44 / 99.14	0.998 / 0.995	0.998 / 0.995	0.998 / 0.995	27	27
HM 0.9	6 (bound) → 6 (bound)	98.02 / 97.48	1.000 / 0.997	1.000 / 0.997	1.000 / 0.997	13	13
ILP 1.00	3, 5, 6 (bound) → 6 (bound)	100.00 / 100.00	0.773 / 0.780	0.773 / 0.780	0.825 / 0.835	6561	3937
IMf 0.3	3, 5, 6 (bound) → none	96.50 / 94.86	0.906 / 0.902	0.906 / 0.902	0.993 / 0.990	11	1

Table 6.6: Rule audit and exact repair for rules 3 and 5 (E2). Rules violated: rules whose violation the language of the net permits, before → after the repair (6 (bound): more than four stops; enforced in the simulator by the stop guard, not in the net). Fitting visits (train / test) are the same before and after the repair. Same language: the unrepaired language written as a state machine, the control for the change of encoding. Sequences with 4 stops: distinct sequences the net permits; 5 occur in training.

6.2.1 Recorded Log

All 58,262 training visits and all 14,625 validation visits satisfy R1 to R5, R7, R11 and R12. The largest observed visit has four yard events. In the 5,090 training visits with at least two stops, no two stop intervals overlap, and start and completion times increase strictly. The gate events and the stop aggregation were constructed upstream. Conformance of the recorded log can therefore partly follow from that construction and is not independent evidence for the rules.

Repeated mixed events at the same unit occur in a few visits, for example `RMG_mixed` twice at block T19 in eight training visits and `HO2_mixed` twice in six. They are not counted as violations of R12, because it is not clear whether they are two separate approaches or an artefact of the stop reconstruction.

6.2.2 Model Language

Table 6.7 shows which rules the discovered nets violate. All sound nets satisfy the gate rules R1 to R4 by construction. R5 is violated by IM0, IMf 0.1 and ILP 1.0. In each case, the shortest counterexample is the visit `Gate In > Gate Out` without any handling. None of the counterexamples occurs in the training log.

Net (expanded)	R1–R4	R5	R7
IM0	satisfied	violated (`Gate In > Gate Out`)	unbounded
IMf 0.1	satisfied	violated (`Gate In > Gate Out`)	unbounded
IMf 0.2	satisfied	satisfied	unbounded
ILP 1.0	satisfied	violated (`Gate In > Gate Out`)	unbounded
Prefix reference	satisfied	satisfied	satisfied

Table 6.7: Rule audit of the model languages. “Unbounded” means that loops allow arbitrarily many yard events.

The empty visit is not the only additional behaviour. Table 6.8 counts the distinct sequences that each net permits, by the number of yard events. With one yard event, IM0 permits exactly the nine observed sequences. With two yard events, it permits 61 sequences, of which 38 are observed. With four, it permits 2,009 against 3 observed. Examples of permitted but unobserved two-stop sequences are `RMG_mixed > RMG_delivery` and `LL_mixed > LL_receive`. Both contradict R12 when the two events take place at the same unit.

Yard events per visit	1	2	3	4
Observed training sequences	9	38	30	3
IM0	9	61	365	2,009
IM0 + R5 + R12 (repaired)	9	57	313	1,583
ILP 1.0	9	81	729	6,561

Table 6.8: Number of distinct sequences permitted by each net, by number of yard events. IM0 additionally permits the empty visit.

The parallel operators of IM0 express that the order of stops is free. They do not mean that a truck is at two places at once. The R11 check in the recorded log supports this reading. Whether the simulator respects it is examined in Section 6.3.

6.2.3 Mechanism in the Simulator

The language audit shows what IM0 permits, not how often the simulator will generate it. Inspection of the ProSiT 1.0.3 source code identified two mechanisms that turn the structure of IM0 into empty visits and overlapping stops.

The first mechanism concerns parallel branches. ProSiT enables a transition at the latest timestamp among its input places. After an AND-split, all branches therefore receive the same enabling time. The second stop of a visit does not wait for the first to finish, and both are scheduled concurrently. This violates R11.

The second mechanism concerns empty visits. IM0 reaches the empty visit through a sequence of silent skip transitions. ProSiT learns a separate routing model for each transition and draws each skip decision independently. The product of several moderately likely skips gives the empty path a substantial probability, although it never occurs in the training data. Both effects shorten the simulated turnaround time.

6.2.4 Repair

Table 6.9 reports the repaired models. All variants are sound and accept all 80 observed training sequences. The share of fitting validation visits is identical for all of them.

The control isolates the effect of the representation. Encoding the same language as a state machine changes precision by only 0.0005. The gain of 0.043 from R5 therefore comes from removing the empty visit. After Gate In, Gate Out had always been a permitted continuation that the log never uses. R12 adds another 0.008 by excluding the four unobserved two-stop sequences that combine a mixed event with a separate delivery or receive at LL or HO2. The

	IM0	Control (same language)	+ R5	+ R5, R12	+ R5, R12, R7
Places / transitions (silent)	32 / 46 (35)	10 / 59 (0)	10 / 58 (0)	10 / 54 (0)	51 / 309 (0)
Fitting validation visits	99.993 %	99.993 %	99.993 %	99.993 %	99.993 %
Precision, validation	0.8131	0.8136	0.8559	0.8644	0.8644
Precision, training	0.8383	0.8386	0.8823	0.8886	–
R5 satisfied	no	no	yes	yes	yes
Concurrent stops possible	yes	no	no	no	no

Table 6.9: Repaired variants of IM0 on the expanded population. The control encodes the unrestricted IM0 language as a state machine.

upper bound of eight yard events (R7) changes no metric, because no observed visit exceeds four. It enlarges the net from 54 to 309 transitions and adds transitions for five to eight stops that have no training data. R7 is therefore not used in the simulation.

The slightly higher fitness of the state machines (0.99999 against 0.99976) is a cost artefact of the silent transitions in IM0. The set of accepted visits is the same.

The repaired net (Figure 6.6) still permits 19 unobserved two-stop sequences, for example `RMG_delivery > RMG_mixed`. They are excluded neither by the data nor by a confirmed rule. Under the generic labels they are admissible when the two events take place at different blocks.

6.2.5 Contribution of the Base Model

The restricted model is the language of the base model intersected with the rules. The rules can only remove behaviour. The base model therefore decides two things: which observed sequences are already lost, and which unobserved sequences remain. Table 6.10 applies the same restriction (R5 and R12) to every sound base model.

Three observations follow. First, the restriction cannot restore what the base model has already excluded. Base models that filter observed behaviour keep only 4 to 27 of the 80 training sequences. Second, among the base models that keep all observed sequences, the choice of base model is a choice of how far the model generalises. The prefix reference keeps only the observed sequences and rejects the six validation visits whose sequence never occurs in training. The restricted IM0 accepts five of them. The restricted ILP net accepts all of them, but permits

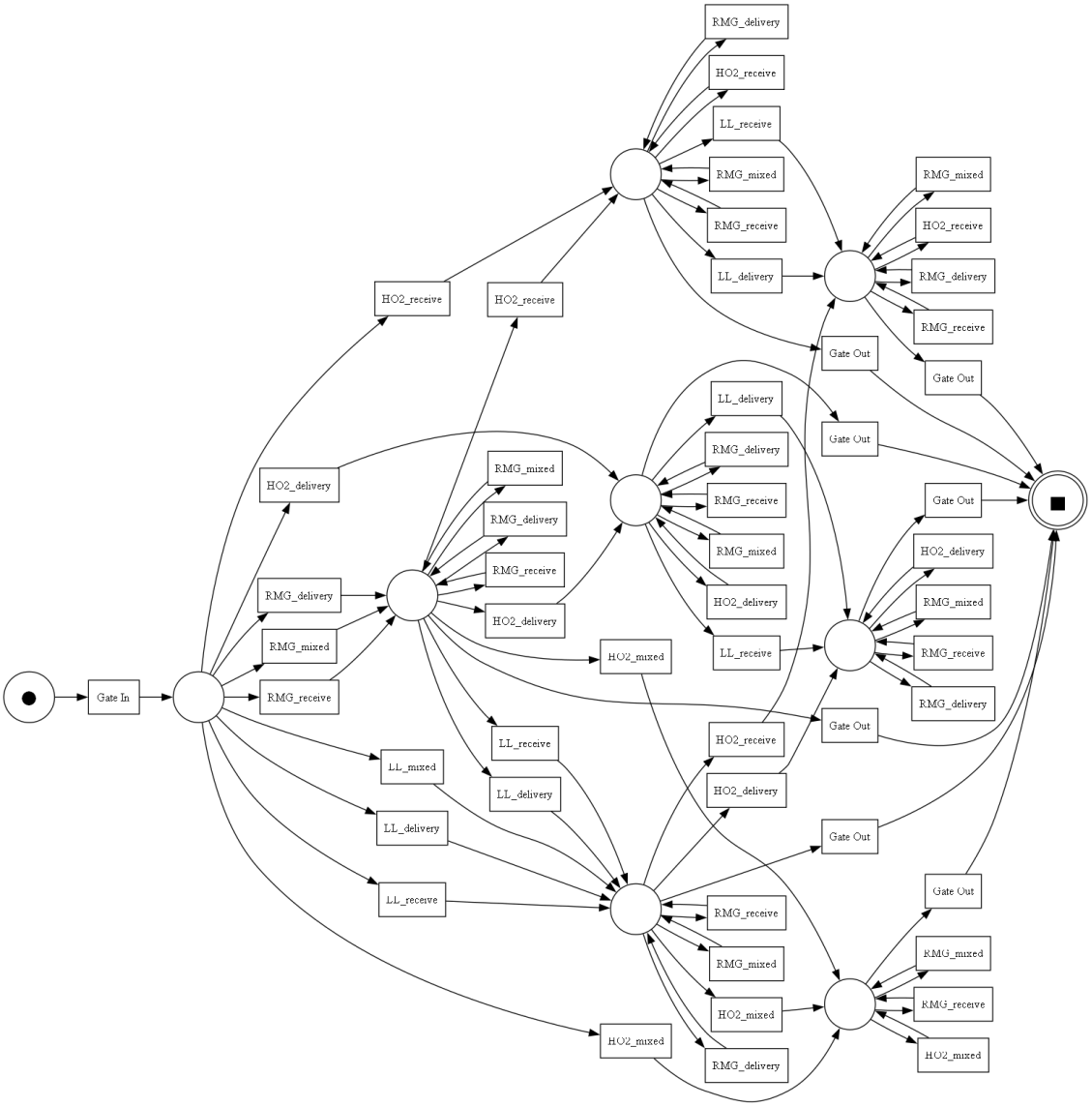


Figure 6.6: Repaired control-flow model `IM0_R5_R12` as a state machine (10 places, 54 transitions). Each place is a state of the visit. Repeated labels belong to different states.

Base model	Permitted sequences, 1 / 2 / 3 / 4 yard events	Training sequences kept (of 80)	Validation visits accepted	Precision (validation), base → restricted
Observed training log	9 / 38 / 30 / 3	80	–	–
Prefix reference	9 / 38 / 30 / 3	80	99.96 %	0.969 → 0.969
Heuristics, no AND	9 / 7 / 1 / 0	17	97.26 %	0.9999 → 0.9999
Split Miner, $\varepsilon = 0$	4 / 0 / 6 / 12	4	40.33 %	0.938 → 0.961
IMf 0.2	6 / 16 / 18 / 20	24	85.20 %	0.999 → 0.999
IMf 0.1	9 / 16 / 18 / 20	27	94.64 %	0.927 → 0.999
IMo	9 / 57 / 313 / 1,583	80	99.99 %	0.813 → 0.864
ILP, alpha 1.0	9 / 73 / 549 / 3,937	80	100 %	0.747 → 0.800

Table 6.10: The same rule restriction (R5, R12) applied to every sound base model. Permitted sequences are counted after the restriction. Validation visits accepted refers to the restricted model.

3,937 sequences with four yard events against three observed. Third, IMo lies between these extremes. It generalises to unobserved combinations of observed stop types, at a precision of 0.864, without becoming a flower model. Selecting IMo is thus a choice of a moderate level of generalisation, supported by its acceptance of new validation sequences.

6.2.6 Conclusion of E2

IMo permits the empty visit and, through its parallel branches, allows the simulator to schedule two stops of one truck at the same time. The repair removes both without losing an observed sequence and raises validation precision from 0.813 to 0.864. The remaining overgeneralisation consists of unobserved sequences that are plausible under the generic labels. E2 shows that the repaired model is a better description of the permitted truck behaviour. It does not yet show that it produces a better simulation. That is the question of E3.

6.3 Control-Flow Candidates in the Simulation (E3)

To explain:

- OUTDATED (ProSiT 1.0.3, candidates CONTRACT/IM0/IM0_R5_R12, validation partition). Rewrite with the E3 runs on the split at 13 April: ILP 1.00, IM 0.0, HM 0.7 and IMf 0.3 repaired, the 2×2 contrast and the same-language control (simulation_e3/runs/split13apr_20261106: main, noguard, guard_orig, samelang). Already current: Tables 6.11 and 6.12 and the summary in Section 6.4.

Net (repaired)	Conformance (training log) Precision	Seq. accepted	Multi-block accepted [%]	Simulation (test period) Stop-count TVD	Sequence TVD	2 stops [%]	RMG visits to 2 blocks [%]	TTT median / P90 [min]
Observed (test log)	–	–	–	0	0	10.66	2.86	33 / 75
ILP 1.00	0.825	82 of 82	100.00	0.021	0.098	8.61	1.11	59 / 141
IM 0.0	0.888	82 of 82	100.00	0.021	0.097	8.62	1.16	58 / 141
IMf 0.3	0.993	19 of 82	1.16	0.054	0.114	5.46	0.04	45 / 104
HM 0.7	0.998	38 of 82	99.22	0.025	0.096	8.26	1.17	58 / 136

Table 6.11: The four repaired nets taken to E3, ordered by precision. Left: conformance on the training log (E1, E2). Right: simulated visits in the test period with stop guard, means over five seeds. TVD: total-variation distance to the observed test log (0 = identical). RMG visits to 2 blocks: share of the visits with an RMG stop that use two different blocks. The turnaround time is far too long for every net; the cause lies outside the control flow (Section 6.5).

Table 6.13 compares the three candidates with the validation log. All values are means over five replications. The range across replications is small, with at most one minute for the medians and about one percentage point for the structural shares.

6.3.1 Visit Structure

The unrepaired IM0 generates both effects predicted in E2. 15.7 % of its simulated visits contain no yard activity. Among its multi-stop visits, 71.6 % have stops that overlap in time. Both values are zero in the recorded log. The stop distribution is also distorted. IM0 generates too few single-stop visits and far more visits with three or more stops than observed (5.5 % against 0.1 %).

The repaired model removes both violations completely, as the state-machine encoding guarantees. Its stop distribution is the closest to the observed one. The total variation distance to the observed distribution is 0.02, against 0.05 for `CONTRACT` and 0.27 for IM0. `CONTRACT` satisfies R5 and R11 as well, because its structure is sequential. It generates too many visits with

IM 0.0, simulated with ProSiT	Visits without stop [%]	Overlapping stops [%]	Visits with 5+ stops [%]	Unseen sequences [%]	Sequence TVD	Stop-co
Observed (test log)	0.00	0.00	0.000	0.00	0.000	
Naive: original net, no guard	15.07	22.71	0.031	19.02	0.360	
Guard only: original net + guard	15.08	21.30	0.022	17.62	0.323	
Repair only: repaired net, no guard	0.00	0.00	0.061	0.53	0.103	
Both: repaired net + guard	0.00	0.00	0.000	0.05	0.097	
Control: same language as state machine, no guard	0.00	0.00	0.061	0.50	0.100	

Table 6.12: Effect of the rule treatment on the simulated visits (E3), test period, means over five seeds. Original net: IM 0.0 as discovered (parallel branches that silent transitions can skip). Repaired net: language restricted to rules 3 and 5, written as a state machine. Guard: data-aware stop guard for rule 6. Control: the unrepaired language written as a state machine. Overlapping stops: share of the visits with more than one stop. Unseen sequences: generated visits whose sequence never occurs in the training log. TVD: total-variation distance to the distribution of the observed test log (0 = identical).

Measure	Observed	CONTRACT	IMo	IMo_R5_R12
Visit structure				
Visits without yard event	0 %	0 %	15.7 %	0 %
Visits with 1 / 2 / ≥3 stops	89.8 / 10.1 / 0.1 %	92.9 / 4.8 / 2.3 %	62.6 / 16.2 / 5.5 %	91.1 / 8.2 / 0.7 %
Multi-stop visits with overlapping stops	0 %	0 %	71.6 %	0 %
RMG visits with several blocks	2.9 %	2.9 %	1.2 %	2.1 %
Turnaround time [min]				
TTT mean / median / P90	42.2 / 34 / 78	33.1 / 28 / 57	28.0 / 25 / 49	31.2 / 27 / 53
Median TTT, delivery / receive / mixed	28 / 35 / 48	25 / 28 / 39	22 / 26 / 31	24 / 27 / 38
Mean approach / exit	13.2 / 13.6	7.0 / 13.2	6.4 / 11.2	6.0 / 12.8
Median stop, RMG / LL / HO2	9 / 7 / 12	8 / 7 / 9	8 / 7 / 10	8 / 7 / 9
Load and delay peaks				
Maximum arrivals per hour	255	132	132	132
Hours with median TTT ≥ 37 min	26.4 %	0 %	0 %	0 %

Table 6.13: Generated visits of the three control-flow candidates compared with the validation log. Means over five replications of 14,625 visits. Approach is Gate In to the start of the first stop, exit is the end of the last stop to Gate Out.

three or more stops (2.3 % against 0.1 %), since its loop does not restrict how often a yard activity is repeated.

The repaired model reproduces the stop distribution partly because its routing trees split on the sampled case attributes `n_stops` and `visit_complexity`. In 99 % of its simulated visits, the number of generated stops equals the sampled `n_stops`. For IMo the share is 65 %. These attributes are part of the planned visit manifest and are known at Gate In (Section 3.2.6). A close stop distribution therefore shows that control flow and case attributes are consistent. It is weaker evidence that the routing has been learned independently.

The share of RMG visits with more than one block is reproduced by `CONTRACT` and underestimated by the repaired model (2.1 % against 2.9 %). In the simulator the block is chosen per stop by a learned resource model. A visit with two RMG stops may therefore be assigned the same block twice or two different blocks. This share depends on the resource model and not on the control flow alone. It is examined in E4.

6.3.2 Turnaround Time

The control-flow model affects the turnaround time in the expected direction. Removing empty visits and overlapping stops raises the median TTT from 25 to 27 minutes and the P90 from 49 to 53 minutes. The effect is largest for mixed visits, which often have two stops. Their median rises from 31 minutes with IM0 to 38 minutes with the repaired model. For IM0 a large part of the second stop is hidden by the overlap.

The effect is small compared with the remaining error. All three candidates underestimate the observed turnaround time. The repaired model reaches a median of 27 minutes against 34 observed, and a P90 of 53 minutes against 78. `CONTRACT` is about one minute closer in median and P90. This is partly due to its additional long visits with three or more stops and is therefore not evidence of a better structure.

The decomposition shows where the gap arises. Exit time and stop durations are close to the observed values for all candidates. The approach from Gate In to the first stop is not. Its observed mean is 13.2 minutes, the simulated means are 6.0 to 7.0 minutes. In the repaired model, the approach accounts for about 7 of the 11 minutes difference in mean TTT. The approach time contains the drive to the block and the waiting before the stop starts. In ProSiT it is generated by the waiting-time model of the first yard resource.

6.3.3 Load and Delay Peaks

None of the candidates generates a single delay-peak hour. In the validation log, 26.4 % of the Gate-In hours with at least 20 visits have a median TTT of 37 minutes or more. The arrival process explains a large part of this difference. The simulated arrivals reach at most 132 trucks per hour, against 255 observed, and no simulated hour reaches the arrival-peak threshold of 165. The calibrated arrival model samples inter-arrival times from one empirical distribution and gates them only by an on/off calendar. It reproduces the average arrival rate but not the daily profile. Simulated arrivals are spread nearly evenly over the day, whereas the observed arrivals concentrate between 6:00 and 18:00. Without the load of the observed peak hours, the simulated resources do not congest.

6.3.4 Conclusion of E3

Under identical simulation conditions, the repaired model produces visits that respect all checked rules and reproduces the observed visit structure most closely. The unrepaired IM0 generates empty visits and overlapping stops at a scale that distorts the stop distribution and the

turnaround time of multi-stop visits. Discovery metrics did not anticipate this. IMo and the repaired model differ by 0.05 in precision and not at all in the share of accepted visits, but by 15.7 percentage points in simulated empty visits.

The repaired model is therefore selected as the control-flow model for the following experiments. The comparison does not show that the simulator reproduces the observed turnaround times. For all candidates, the remaining error is dominated by the arrival process and the approach time, which do not depend on the control-flow model. These two components are the subject of E4.

6.4 Answer to RQ1

To explain:

- Short answer, part 1: the unfiltered Inductive Miner net (IM 0.0), restricted exactly to rules 3 and 5 and written as a state machine, with the stop guard in ProSiT for rule 6. It is the only sound net that accepts every observed sequence with a precision clearly above that of a flower-like net, and the repair raises its precision from 0.838 to 0.888 without losing a visit. Justified by E1 (Section 6.1, Figures 6.1 and 6.3, Tables 6.1 and 6.2) and E2 (Section 6.2, Table 6.6). Measures and selection rule: Section 5.6.3; coverage measures: Section 2.4; repair: Section 5.7.3.
- Short answer, part 2: only partly. Precision did not anticipate the simulation (same precision, very different simulation; higher precision, same simulation). Coverage did: IMf 0.3, the net with the lowest coverage, is the only one with a clearly worse simulated structure. The state-machine encoding, which no language measure sees, decided whether impossible visits were generated. Justified by E3 (Section 6.3, Tables 6.12 and 6.11).

RQ1 has two parts. The first asks which discovery and rule treatment yields a sound, executable control-flow model that preserves the observed truck-visit behaviour. The second asks whether conformance metrics anticipate the behaviour of this model in simulation. This section answers the first part with E1 and E2. The second part needs the simulation results of E3.

6.4.1 Part 1: Discovery and Rule Treatment

The best basis is the unfiltered Inductive Miner net (IM 0.0). Its language is restricted exactly to the sequence rules 3 and 5 and written as a state machine. In the simulator, a data-aware guard makes every truck perform as many stops as its manifest plans (rule 6). Each of the three parts solves a different problem.

Discovery: keep all observed behaviour. Only two sound nets accept every training sequence: IM 0.0 and ILP 1.00 (Figure 6.3, Table B.1). ILP 1.00 allows every sequence of stops, for example all 6,561 sequences with four stops, and reaches a precision of 0.773. IM 0.0 restricts the order and the repetition of the stops (2,009 sequences with four stops) and reaches 0.838. Every discovered net with a higher precision gets it by rejecting observed sequences (Figure 6.1). The usual measures hide this loss. Because the six most frequent sequences cover 88.5 % of the visits, IMf 0.3 fits 96.5 % of the visits and reaches a log fitness of 0.993, although it accepts only 19 of the 82 sequences and 1.2 % of the multi-block visits (Table 6.1). The Heuristics nets keep more visits (HM 0.7: 99.4 %), but still reject 44 of the 82 sequences. Discovery from start and complete events does not help either: with corrected timestamps, the lifecycle Inductive Miner yields exactly the language of IM 0.0 (Section 3.2.3).

Is keeping every observed sequence overfitting? One could object that preferring the net that accepts every observed sequence is overfitting. In general, discovery balances fitness, precision, generalisation and simplicity, and perfect fitness is not a goal in itself [58]. A net overfits if it allows only the behaviour of the training log and fails on new behaviour [24]. The prefix tree is such a net: it accepts all 82 training sequences, but none of the 16 test sequences that do not occur in training. IM 0.0 is not: its precision is the same on the training and the test log (0.838), it fits 99.99 % of the test visits, and it accepts 13 of the 16 new test sequences. Its problem is the opposite: it allows too much. The rule treatment removes this surplus with domain knowledge, not with frequencies.

Noise filtering, as in IMf, assumes that rare behaviour is noise [29]. Here it is not. Every training sequence satisfies the sequence rules 1, 2, 3 and 5, since the repaired net still accepts all of them. Most sequences that the filtered nets reject occur again in the test log (IMf 0.3: 41 of 63; Table 6.2). Not all rare sequences matter for the simulation, though. The simulation has to reproduce the turnaround time and the load of the terminal, and for these the truly rare sequences count little: the 47 sequences with fewer than ten visits in six weeks make up 0.19 % of the visits and 0.42 % of the truck-minutes. But the filtered nets do not cut only this tail. IMf 0.3 also rejects the ninth most frequent sequence, a delivery and a receive at two different RMG blocks (814 visits). Its filter removes steps between two activities that are rare compared with the most frequent step from the same activity, no matter how often the whole sequence occurs. The visits that IMf 0.3 rejects are 3.5 % of all visits, but 5.5 % of the truck-minutes, 10 % of the visits above the P90 turnaround time of 64 minutes and 98.8 % of the multi-block visits. They are long visits, and the multi-block visits are directly affected by the block allocation studied in E5.

The two errors also have different costs in the simulation. A sequence that the net rejects can never be generated. A sequence that the net accepts is generated only as often as the learned routing probabilities allow, and these are small for rare branches. Keeping rare but valid sequences therefore costs little, while removing them takes away long visits that matter for the KPIs. The preference for IM 0.0 is thus not a preference for perfect fitness. It follows from the purpose of the model.

Rule treatment: remove what is physically impossible. IM 0.0 allows too much. Its language contains visits without a stop (rule 3), a mixed stop together with a separate delivery or receive at the same LL or HO2 area (rule 5), and more than four stops (bound of rule 6). Through its parallel branches, it would also let the simulator start two stops of the same truck at the same time (rule 4). The exact repair restricts the language to rules 3 and 5 and writes it as a state machine (Section 5.7.3). It keeps every observed sequence: 100 % of the training visits and 99.99 % of the test visits still fit. Precision rises from 0.838 to 0.888 on the training log and to 0.889 on the test log (Table 6.6). The unrepaired language written as a state machine has the same precision as the original net (0.838). The gain therefore comes from the removed behaviour, not from the new encoding. The state machine holds one token per truck, so the stops of a truck can no longer overlap (rule 4). The bound of rule 6 is not encoded in the net. The stop guard enforces the stronger equality with the planned stops in the simulator.

Why not another candidate. The repair does not change the ranking. HM 0.7 and HM 0.9 already satisfy rules 3 and 5. The repair leaves them unchanged, and they still reject 44 and 55 of the 82 training sequences. The repaired ILP 1.00 keeps all sequences, but it stays less precise than the repaired IM 0.0 (0.825 against 0.888) and permits 3,937 instead of 1,583 sequences with four stops. The repaired IMf 0.3 reaches a precision of 0.993, but only because it still rejects 63 of the 82 training sequences and 98.8 % of the multi-block visits.

Limits of the answer. The answer holds for the tested configurations of four miner families (Table 5.3) and for the domain rules of Table 3.3, which the terminal has not yet confirmed. The repaired IM 0.0 still permits sequences that never occur: 1,583 sequences with four stops, of which five occur in training. They are plausible under the generic activity labels, and the learned routing probabilities decide how often the simulator generates them. In the simulation with the stop guard, visits with an unobserved sequence make up 0.05 % of the generated visits (Table 6.12).

6.4.2 Part 2: Do Conformance Metrics Anticipate the Simulation?

Only partly. Precision did not anticipate the simulated behaviour. Coverage did. And the way a language is written as a net decides behaviour that no measure on the language can see.

Same language, same scores, different simulation. The original IM 0.0 and its same-language state machine accept exactly the same sequences. They therefore have the same fitness and the same precision (0.838). In the simulation they differ strongly (Table 6.12). Without the stop guard, the original net generates 15.1 % visits without a stop, overlapping stops in 22.7 % of the visits with more than one stop, and 19.0 % visits whose sequence never occurs. The state machine generates no visit of the first two kinds and 0.5 % of the third. The reason is where the decisions sit. In the original net, every type of stop is a parallel branch that a silent transition can skip. ProSiT learns each skip on its own, and in every branch skipping is the more frequent choice, because most visits have only one stop. Together, these separate choices produce empty visits and parallel stops, although no observed visit has them. In the state machine, each decision is a choice between the next visible steps, learned from the observed sequences. Precision cannot show this difference, because it is computed on the language, and the language is the same.

Higher precision, same simulation. The repair raises precision from 0.838 to 0.888, but it does not change the simulation. The repaired net and the same-language state machine generate practically the same visits (stop-count TVD 0.027 for both, sequence TVD 0.103 and 0.100). The behaviour that the repair removes was almost never generated anyway, because ProSiT learns the routing from the observed visits and gives unobserved branches little weight. The precision gain comes from the restricted language; the gain in the simulation comes from the state-machine encoding. The same holds across the candidates (Table 6.11). ILP 1.00, IM 0.0 and HM 0.7 range from 0.825 to 0.998 in precision, but they generate nearly the same sequences (sequence TVD 0.096 to 0.098). With the stop guard, the number of stops follows the plan of each truck whenever the net allows it. The stop-count distribution therefore mainly shows whether a net can produce the planned stops.

Coverage anticipated the one structural failure. IMf 0.3 has the second highest precision (0.993 after the repair) and the lowest coverage: it accepts 19 of the 82 training sequences and 1.2 % of the multi-block visits. It is also the only candidate whose simulated structure is clearly worse. Its stop-count TVD is about 2.5 times that of the other nets (0.054). It

generates 5.5 % instead of 10.7 % visits with two stops, and almost no RMG visits to two blocks (0.04 % against 2.86 % observed). The stop guard shows the mechanism directly. When a truck plans a number of stops that the net cannot produce, the guard finds no permitted step and falls back to the learned routing. This happens about 1,050 times per run with IMf 0.3, about 117 times with HM 0.7, about 4 times with ILP 1.00 and never with IM 0.0. This is the order of their coverage. It also answers the objection discussed in Part 1: the behaviour that the filtered net loses becomes visible in the simulation.

Repair and guard. The 2×2 contrast separates the two rule treatments (Table 6.12). The guard alone hardly changes the original net (15.1 % visits without a stop in both cases). The original net makes its decisions with silent transitions, which the guard does not see, so the guard falls back to the learned routing about 10,000 times per run. On the state machine, every decision is visible and the guard works as intended. It removes the visits with more than four stops (0.06 % $\rightarrow$ 0) and reduces the unseen sequences from 0.5 % to 0.05 %. The data-aware guard therefore needs the state-machine encoding.

Turnaround time. No candidate reproduces the turnaround time. ILP 1.00, IM 0.0 and HM 0.7 reach a median of 58 to 59 minutes and a P90 of 136 to 141 minutes, against 33 and 75 minutes observed. The stops themselves have about the right duration (medians within one to four minutes of the observed values). The difference is waiting. About 55 % of all simulated RMG stops go to a single block (observed: 6.3 %), and the time from Gate In to the start of the first stop is 43 minutes on average (observed: 13 minutes). IMf 0.3 comes closer (45 and 104 minutes), but not because of its control flow: it sends somewhat fewer stops to this block (50 %), and its trucks need 30 minutes to the first stop. The concentration on one block is caused by the simulator, not by the control-flow model. In a control run without ProSiT's workload features, the largest block receives 6.0 % of the RMG stops, as observed. This mechanism is examined in E4 (Section 6.5). At this stage, the turnaround time is therefore not a usable criterion for the control flow.

Answer to part 2. Conformance metrics anticipate the simulation only partly. Precision did not: higher precision did not lead to a better simulated structure, and two nets with the same precision behaved very differently. Coverage did: the only candidate with low coverage was the only one with a clearly worse structure, and its errors concern exactly the visits it does not accept. What the measures cannot see is how a language is written as a net, and this decided whether the simulator generated impossible visits. A control-flow model for simulation should therefore be judged by its coverage and its representation, and then checked in the simulation.

6.5 Mechanism Audit and Simulation Validity (E4)

To explain:

- Short answer: under replay of the observed trucks and blocks, the simulation reproduces the turnaround time to within 5 minutes at the median and 13 minutes at the P90 (28 against 33, 62 against 75 minutes) and reproduces the rise of the approach with the block load. It needs four changes to get there. The error of the E3 baseline (median 58, P90 141 minutes) came from the block choice (self-count in its queue feature, amplified by a too low concurrency count), and the remaining error under load came from the queue (capacity checked at enabling, single-server waiting target) and the service (no load in the execution time). The remaining error is mainly in the stop durations; part of it is the change between the periods.
- Tables: 6.15 (path of the truck), 6.17 (each change reversed), 6.16 (choice on the training period), 6.19 (load profiles), 6.20 (where the error is), 6.21 (acceptance).

E4 follows the three levels of the evaluation design (Section 5.9.1): verification, conceptual validity and operational validity. The operational results follow the path of a truck through the simulator, from the demand to the service (Table 6.15). The final configuration is then chosen on the training period, each of its changes is reversed alone, and it is checked against the acceptance criteria.

6.5.1 Verification

Four checks passed. With all changes switched off, the modified simulation loop generates exactly the same log as ProSiT. Every component that is retrained on unchanged training data is identical to the fitted one. The replayed arrivals are exact: every simulated visit starts at the Gate-In time of its observed visit and carries its manifest, and the case-arrival distance to the test log is zero. No configuration generates a visit without a stop, overlapping stops of one truck or more than four stops.

One check failed. The number of stops in progress at a block, recomputed from the generated log, is larger than the number the simulator used for 13 to 14 % of the RMG stops (five seeds), by one or two. ProSiT counts the activities in progress at a resource by walking backwards through its schedule and stops at the first activity that has already ended. The simulation loop stores the activities in the order in which it processes them, not by their end time, so the walk can stop too early. In training, the same quantities are computed exactly. The simulated trucks therefore see less load than the models were trained on, in the workload and queue features and in the capacity check. All configurations from b3 on were repeated with an exact

count (suffix x), and only these are reported below. The effect is large where a model reacts strongly to load: with the exact count alone, the E3 baseline sends 29 % instead of 55 % of the RMG stops to a single block (a0x in Table 6.15).

6.5.2 Conceptual Validity: What the Components Learned

The fitted models were inspected to see whether they use plausible features and targets for the terminal.

Block choice. Of the 26 block-choice models, 22 split on the queue length of a block and 16 on its workload. In training, ProSiT counts the queue of a block at the moment a stop is enabled, and it includes the stop itself whenever the stop starts later than it was enabled. This is the case for 99.98 % of the training stops. The chosen block therefore always has a queue of at least one, and the models learn that trucks go to the block with the longer queue. In the simulation, the new stop is not yet in any queue, so the rule becomes a positive feedback: the more trucks a block has, the more new trucks choose it. Retrained without this self-count, only 4 of the 26 models split on the queue length. The others split on the manifest, for example on reefer containers, the share of full containers and the visit complexity.

Waiting time. The waiting-time models split mainly on the queue length (25 of 26 models) and the workload (24). The features are plausible, but the target is not. ProSiT measures the waiting time from the moment the block was last completely free, that is, from the latest end of all stops in progress when the stop is enabled. A block serves up to three trucks at once, so a truck often starts while others are still served. The target is then zero or negative. Table 6.14 shows the consequence. The busier the block, the shorter the learned waiting time, although the real delay grows.

Other stops in progress at enabling	**0**	**1**	**2**	**3+**
RMG stops in training	23,080	12,136	7,267	4,499
ProSiT waiting target, median (min)	6	3	1	0
Share of targets ≤ 0	0 %	31 %	50 %	54 %
Observed delay from enabling to start, median (min)	6	7	9	16

Table 6.14: ProSiT's waiting-time target at the RMG blocks by the load of the block when the stop is enabled, training log. The target is measured from the moment the block was last completely free (single server).

Execution time. The 11 execution-time models split mainly on the hour (7 models) and the weekday (6). They have no load feature, because ProSiT does not offer one for execution times.

Retrained with the number of other stops in progress at the block, 8 of the 11 models split on it.

Capacity. ProSiT checks the capacity of a block when a stop is enabled, not when it starts. A stop can therefore be enabled at a free block and start much later at a full one. In the E3 baseline, up to 23 trucks are at one block at the same time (Table 6.15), although a block serves at most three.

6.5.3 Operational Validity Along the Path of the Truck

Configuration	Med.	P90	CTD	Appr.	Stops	Top bl.	Max	Spear.	G-appr.	G-stop	G-WIP
Observed (test log)	33	75	0.0	12.7	14.4	6.3	4	0.55	0.0	0.0	0.0
Real: week 2 vs. week 1	32	72	3.1	13.0	14.7	6.7	4	0.54	2.0	0.5	5.8
Real: training vs. test period	29	64	5.5	11.0	12.1	6.1	4	0.49	1.2	1.0	2.0
E4a Demand											
a0 E3 baseline	58	141	29.8	42.5	11.8	54.9	23	-0.35	24.5	5.5	29.9
a0x E3 baseline, exact count	34	110	9.8	23.4	11.9	28.6	16	-0.02	12.1	5.2	6.1
a1 hour-of-week arrivals	56	137	27.6	40.3	12.1	48.2	27	-0.49	23.2	5.3	24.8
a2 replayed arrivals	55	136	26.5	37.7	12.2	44.4	26	-0.59	22.2	5.3	27.4
E4b Block allocation											
b1 learned blocks, no self-count	25	53	11.5	4.6	12.0	5.8	7	-0.33	9.8	5.2	6.8
b2 training block shares	25	53	11.6	4.6	12.0	5.8	7	-0.28	9.8	5.2	7.0
b3x observed blocks	24	51	12.3	4.0	11.9	5.8	8	-0.39	8.7	5.2	7.0
E4c Queue at the block											
c1x capacity at start	24	52	11.8	4.5	11.9	6.0	3	-0.25	5.4	2.9	6.4
c2x + waiting queue without self-count	25	53	11.6	4.6	11.9	6.0	3	-0.18	5.4	2.9	5.8
c3x lane-free waiting	29	64	5.4	10.5	11.9	6.0	3	0.38	1.6	3.0	7.0
E4d Service under load											
d3x = c3x + load-dependent service	28	62	6.3	10.5	10.8	6.0	3	0.48	1.2	1.3	6.2
Sensitivity (more than three trucks at a block, not admissible)											
c4x lane-free waiting, 4 lanes	29	63	5.4	10.4	11.9	6.0	4	0.36	3.5	5.2	7.8
c5x whole delay, no capacity	29	63	5.6	10.2	11.9	6.0	7	0.34	3.4	5.2	7.0
d4x = c4x + load-dependent service	28	62	6.2	10.5	11.0	6.0	4	0.49	2.7	2.4	7.0
d5x = c5x + load-dependent service	28	62	6.4	10.5	10.9	6.0	9	0.45	3.5	2.5	7.3

Table 6.15: Error attribution along the path of the truck (E4a–E4d), against the test log, means over five seeds. From b3x on, the observed demand and the observed blocks are used, so that only the queue and the service vary. The rows in italics are differences between real periods. Med., P90: median and P90 of the TTT; CTD: cycle-time distance; Appr., Stops: mean approach and stop time per visit (all in minutes). Top bl.: share of the RMG stops at the busiest block in %. Max: largest number of trucks at one block at once. Spear.: Spearman correlation of hourly arrivals and hourly median TTT. G-appr., G-stop, G-WIP: profile gaps for the approach by trucks at the block, the RMG stop duration by other trucks at the block and the TTT by trucks in the terminal (minutes).

Demand (E4a). The E3 baseline overestimates the turnaround time strongly: median 58 and P90 141 minutes, against 33 and 75 observed. A realistic arrival profile (a1) and even the exact

observed arrivals and manifests (a2) hardly change this (median 55, P90 136 minutes). The demand is therefore not the cause. The time is lost before the first stop: the approach takes 43 minutes on average instead of 13, because 55 % of the RMG stops go to a single block. With an exact count, this share falls to 29 % (a0x). The median TTT is then close to the observed one (34 minutes), but for the wrong reason: the P90 is still 110 minutes, and the turnaround time does not rise with the arrivals (Spearman −0.02). A single summary measure would have hidden this.

Block allocation (E4b). Without the self-count, the learned block choice no longer concentrates the trucks (b1): the busiest block receives 5.8 % of the RMG stops, as observed (6.3 %). The self-count is therefore the cause of the concentration. But no block choice learned on the training period reproduces which blocks are busy in the test period. The correlation of the block shares with the observed ones is −0.03 for the retrained model and 0.02 for the training shares (b2), and it is also low between two real periods (−0.004). With the observed block of each stop (b3x), the correlation is 0.96; it is not 1 because the routing stays learned, and a simulated visit with more RMG stops than its observed visit draws the extra block from the training shares.

With the observed demand and blocks, the error changes its sign. The turnaround time is now too short (median 24, P90 51 minutes), the approach takes 4 instead of 13 minutes, and it falls when the block is busy (Table 6.19): from 6 minutes at an empty block to 0 minutes with three trucks, while the observed approach rises from 6 to 16 minutes. This is the single-server target of Table 6.14. At the same time, up to eight trucks are at one block, because the capacity is checked at enabling.

Queue at the block (E4c). Checking the capacity over the whole stop interval at its start (c1x) keeps at most three trucks at a block, but it does not create waiting: the approach stays at 5 minutes, and the turnaround time still falls with the arrivals (Spearman −0.25). Removing the self-count from the queue feature of the waiting time (c2x) changes little. Only the lane-free waiting target together with the capacity at start (c3x) lets the queue arise where it arises at the terminal, when all three lanes are busy. The approach now rises with the block load (7, 7, 9 and 11 minutes for 0 to 3 trucks at the block, observed 6, 6, 9 and 16), and the median and P90 rise to 29 and 64 minutes. The turnaround time now rises with the arrivals (Spearman 0.38). Four lanes (c4x) or a learned whole delay without capacity (c5x) give similar turnaround times, but allow four and up to seven trucks at a block; they are not admissible.

Service under load (E4d). In c3x, an RMG stop takes 8 minutes no matter how many trucks are at the block, while the observed median grows from 6 to 9 and 14 minutes. With the load feature in the execution time (d3x), the simulated stop takes 6, 8 and 11 minutes. The turnaround time follows the hourly arrivals more closely (Spearman 0.48, observed 0.55), and the hourly median TTT of the simulation correlates with the observed one at 0.56. The level, however, falls slightly (median 28, P90 62 minutes), because the simulated stops are shorter on average (10.8 instead of 14.4 minutes per visit).

6.5.4 Choice of the Final Configuration (E4f)

Configuration (training period)	Admissible	G-appr.	G-stop	G-WIP	Sum	CTD
c1x capacity at start	yes (max 3)	4.8	3.0	4.8	12.6	7.4
c2x + waiting queue without self-count	yes (max 3)	4.8	3.0	4.2	12.1	7.4
c3x lane-free waiting	yes (max 3)	1.1	3.0	3.0	7.1	2.1
d3x = c3x + load-dependent service ←	yes (max 3)	0.8	1.3	3.5	5.6	2.7

Table 6.16: Choice of the final configuration on the training period (E4f): replayed training visits with their observed blocks, compared with the training log, means over five seeds. Admissible: no rule violation and at most three trucks at a block at once. The configuration with the smallest sum of the profile gaps is chosen (←). Gaps and CTD in minutes.

The rule of Section 5.9.4 was applied on the training period (Table 6.16). The four admissible configurations were run with the replayed training visits and their observed blocks and compared with the training log. All four keep at most three trucks at a block and produce no rule violation. The configuration with lane-free waiting and load-dependent service (d3x) has the smallest sum of the profile gaps (5.6 minutes) and is the final configuration. Without the load-dependent service (c3x), the sum is 7.1 minutes, because the stop duration does not grow with the load (gap 3.0 minutes). Without the lane-free waiting (c1x, c2x), it is about 12 minutes, because the approach does not grow with the load. The ranking is the same on the test period (Table 6.15), so the choice does not depend on the period. The training period also shows the price of the load-dependent service: the cycle-time distance of c3x is smaller (2.1 against 2.7 minutes). The rule puts the load response first, because E5 depends on it.

6.5.5 Is Every Change Needed? (E4e)

Each change of the final configuration was reversed alone (Table 6.17). Every reversal makes the load response worse. With ProSiT's count instead of the exact count, the sum of the profile gaps rises from 8.7 to 12.1 minutes and the Spearman correlation falls from 0.48 to 0.32. With the

Configuration	Med.	P90	CTD	Appr.	Stops	Top bl.	Max	Spear.	G-appr.	G-stop	G-WIP
Observed (test log)	33	75	0.0	12.7	14.4	6.3	4	0.55	0.0	0.0	0.0
Final configuration											
d3x = c3x + load-dependent service	28	62	6.3	10.5	10.8	6.0	3	0.48	1.2	1.3	6.2
One change reversed											
ProSiT count	27	60	7.5	10.2	10.0	6.0	3	0.32	1.6	2.0	8.4
capacity at enabling	27	59	7.6	9.6	10.6	5.9	7	0.37	3.8	2.8	7.8
single-server waiting	25	53	11.4	5.0	11.7	6.0	3	0.08	5.0	1.2	5.9
service without load	29	64	5.4	10.5	11.9	6.0	3	0.38	1.6	3.0	7.0
Block allocation instead of the observed blocks											
blocks: training shares	28	59	7.4	9.6	10.7	6.0	3	0.38	1.7	1.3	6.5
blocks: learned, no self-count	28	60	7.1	9.9	10.7	6.1	3	0.41	1.6	1.3	7.3
Arrival intensity (extreme condition)											
arrival intensity 0.8	27	57	8.4	8.8	10.7	6.0	3	0.38	2.0	1.3	3.2
arrival intensity 1.2	33	80	1.8	15.5	12.0	6.1	3	0.75	1.8	1.3	5.2
Fully generative (no replay; training block shares)											
hour-of-week arrivals	26	54	9.9	8.3	10.2	6.1	3	0.43	1.9	1.3	4.2
ProSiT arrivals	25	51	11.4	7.4	9.7	6.0	3	0.15	2.5	1.3	2.6
First day of the test period											
Observed (13 April)	49	96	0.0	18.4	14.4	7.5	4	0.34	0.0	0.0	0.0
final configuration*	30	70	17.4	14.0	11.0	6.5	3	0.59	2.0	1.0	13.0
blocks: learned (ProSiT)*	3,199	7,727	3,417	3,100	15.2	100.0	3	-0.46	3,031	1.0	1,081

Table 6.17: Final configuration with each change reversed alone, with other block allocations, other arrival intensities and without replay (E4e), against the test log, means over five seeds. *Only the 2,393 visits of the first day, because the queue of the learned block choice grows without bound. Med., P90: median and P90 of the TTT; CTD: cycle-time distance; Appr., Stops: mean approach and stop time per visit (all in minutes). Top bl.: share of the RMG stops at the busiest block in %. Max: largest number of trucks at one block at once. Spear.: Spearman correlation of hourly arrivals and hourly median TTT. G-appr., G-stop, G-WIP: profile gaps for the approach by trucks at the block, the RMG stop duration by other trucks at the block and the TTT by trucks in the terminal (minutes).

capacity checked at enabling, up to seven trucks are at a block again, and the configuration is no longer admissible. With the single-server waiting target, the approach falls to 5 minutes and the turnaround time no longer follows the arrivals (Spearman 0.08); this is the largest single effect. Without the load in the service time, the stop-duration gap rises from 1.3 to 3.0 minutes and the Spearman correlation falls to 0.38. This last change is the only one that costs level: without it, the cycle-time distance is 5.4 instead of 6.3 minutes, because the load-dependent stops are shorter on average (Section 6.5.7).

Block allocation. If the blocks are not taken from the log, the level of the turnaround time stays the same (median 28 minutes) and the profiles change little, both with the training shares and with the learned choice without the self-count. What is lost is which block is busy: the correlation of the block shares with the observed ones falls to about zero, and the correlation

of the hourly median TTT with the observed one falls from 0.56 to 0.47 and 0.50. The learned choice of ProSiT, with the self-count, cannot be used together with the real capacity. Because the chosen block attracts further trucks while it can serve only three, its queue grows without bound. On the first day of the test period, this choice sends every RMG stop to a single block, the capacity check delays a stop by 71 hours on average, and the median turnaround time is 53 hours (Table 6.17). The self-count must therefore be removed before the learned block choice can be used with a realistic capacity. The same day also shows the limit of the final configuration: the observed median is 49 minutes on this very busy Monday, the simulated one 30 minutes.

Arrival intensity. The extreme-condition test passes. With the arrivals stretched to an intensity of 0.8, the median and mean TTT fall to 27.0 and 32.8 minutes; with the arrivals compressed to 1.2, they rise to 33.2 and 42.7 minutes, and the share of delay-peak hours rises from 1.4 % to 22.8 %. At an intensity of 1.2, the simulated turnaround time matches the observed one (median 33 against 33, P90 80 against 75 minutes, cycle-time distance 1.8 minutes). The simulated terminal therefore needs about 20 % more trucks to become as congested as the real terminal.

Fully generative. Without replay, the arrivals come from the training period. With the hour-of-week arrivals and the training block shares (eG), the median is 26 and the P90 54 minutes (cycle-time distance 9.9 minutes); with ProSiT's arrival model (eGn), 25 and 51 minutes. The turnaround time still follows the arrivals with the hour-of-week model (Spearman 0.43), but not with ProSiT's arrival model (0.15), which has no daily peaks (E4a). There are no delay-peak hours, and the terminal holds 58 instead of 92 trucks on average, because the training period had less demand than the test period. The profile gaps of these runs are not comparable with the others: the high-load bins are missing, and the gaps are computed only over the bins that both logs reach.

6.5.6 The Final Model at a Glance

Table 6.18 and Figures 6.7 to 6.10 show the final model (Chapter 4) against the observed logs. The simulated turnaround time has the right shape but a shorter tail (Figure 6.7). Over the hours of the test period, the simulated median follows the observed one (Spearman 0.57 for seed 42), but it does not reach the observed peaks of 80 minutes and more (Figure 6.8). Under load, the approach to the first stop rises as observed; the stop duration rises, but less than observed; and the turnaround time falls behind when more than 100 trucks are in the terminal (Figure 6.9). The time that is missing is mainly stop time (Figure 6.10).

	Median	P90	Mean	CTD	Approach	Stops	Spearman	Max at block
Test period (13–30 April)								
Observed	33	75	41.2	–	12.7	14.4	0.55	4
Simulated	28	62	34.9	6.3	10.5	10.8	0.48	3
Training period (in-sample)								
Observed	29	64	36.6	–	11.0	12.1	0.49	4
Simulated	28	59	33.8	2.7	10.5	10.6	0.46	3

Table 6.18: Key values of the final simulation model against the observed logs (replayed visits and observed blocks, means over five seeds). Times in minutes; CTD: cycle-time distance; Spearman: hourly arrivals against hourly median TTT; Max at block: largest number of trucks at one block at once (the log records 4 because of short recording overlaps, Section 5.9.3).

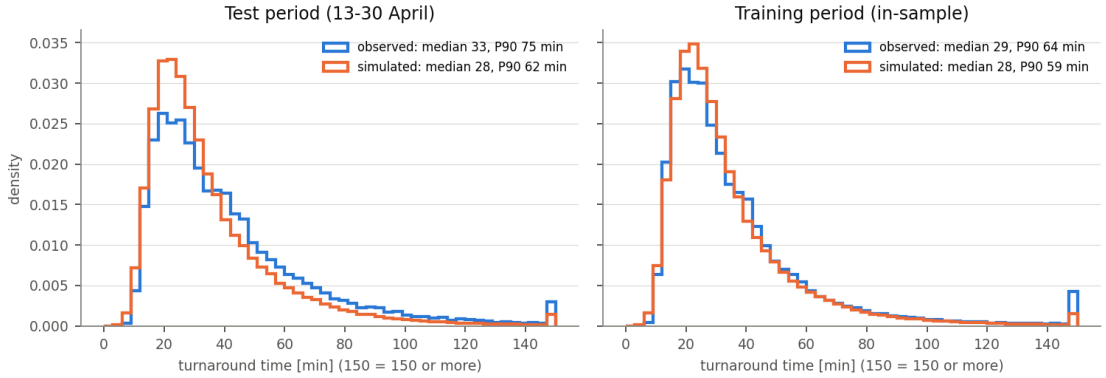


Figure 6.7: Turnaround time, observed and simulated (final model; replayed visits and observed blocks; five seeds pooled).

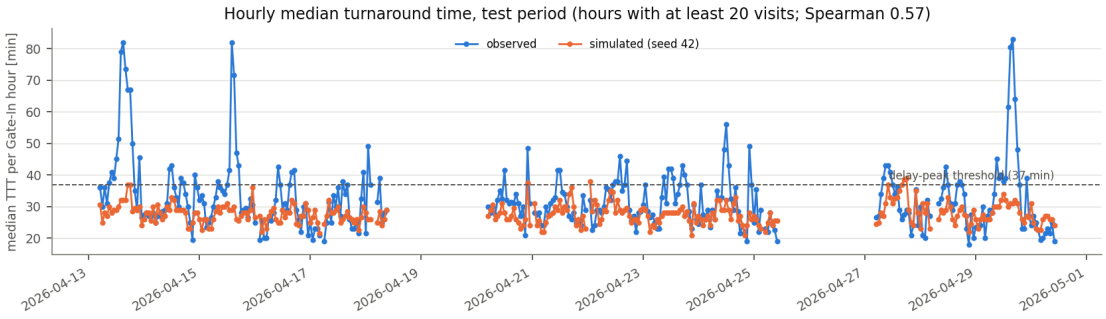


Figure 6.8: Median turnaround time per Gate-In hour over the test period, observed and simulated with the same trucks (seed 42).

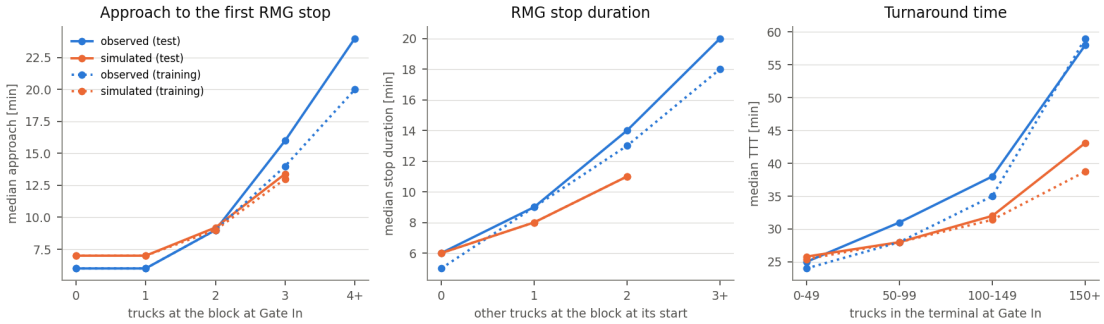


Figure 6.9: Load profiles, observed and simulated, in the test period (solid) and the training period (dotted); medians, means over five seeds. With three lanes, the states "4+ trucks at the block" and "3+ other trucks" cannot occur in the simulation.

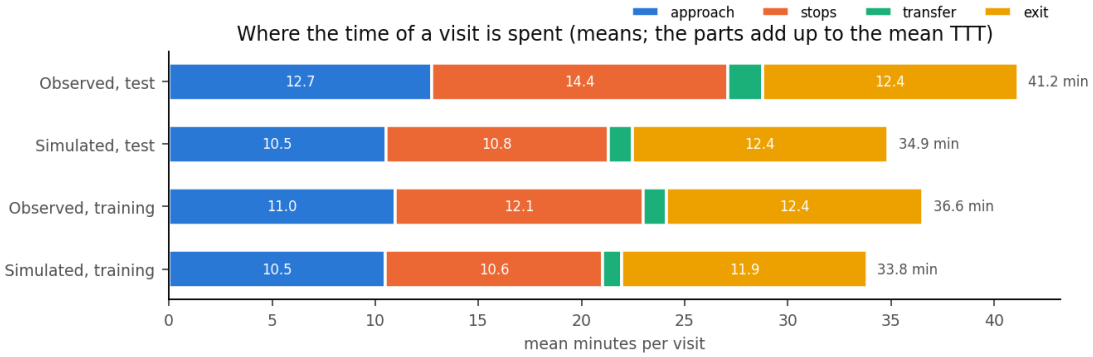


Figure 6.10: Where the time of a visit is spent, observed and simulated (means).

6.5.7 Where the Remaining Error Lies

In-sample and out-of-sample. On the training period, the final configuration is close to the log (fo): median 28 against 29 minutes, P90 59 against 64 minutes, and a cycle-time distance of 2.7 minutes, which is within the variation between two real weeks. The approach profile is reproduced (gap 0.8 minutes), the stop-duration profile almost (gap 1.3 minutes). On the test period, the cycle-time distance is 6.3 minutes. Table 6.20 shows where the difference comes from. The mean error grows from −2.7 to −6.3 minutes, mainly in the approach (−0.5 to −2.2) and in the stop durations (−1.5 to −3.4). These are the parts in which the real terminal changed between the periods: from training to test, the approach grew by 1.8 minutes, and the stops took 2.0 minutes longer per visit with nearly the same routing. The stops took longer at every load: an RMG stop with no other truck at the block took 8.2 instead of 7.5 minutes on average, one with two others 17.7 instead of 15.9 minutes. A model learned on the training

Profile	Bin	Observed	a0	b3x	c1x	c3x	d3x
Approach to an RMG first stop by trucks at the block at Gate In							
	0	6	8	6	6	7	7
	1	6	39	3	3	7	7
	2	9	40	2	2	9	9
	3	16	41	0	4	11	13
	4+	24	57	0	–	–	–
RMG stop duration by other trucks at the block at its start							
	0	6	7	8	8	8	6
	1	9	7	8	8	8	8
	2	14	7	8	8	8	11
	3+	20	8	8	–	–	–
TTT by trucks in the terminal at Gate In							
	0–49	25	75	25	25	27	26
	50–99	31	57	24	24	29	28
	100–149	38	58	24	26	32	32
	150+	58	70	–	–	40	43

Table 6.19: Load-response profiles against the test log: medians in minutes, means over five seeds. –: fewer than 50 cases in the bin. With three lanes, the bins "4+ trucks at the block" and "3+ other trucks" cannot occur; in the log they are short recording overlaps.

						Stops split	
Comparison	Mean TTT	Approach	Stops	Transfer	Exit	Count	Duration
Final, test period	-6.3	-2.2	-3.6	-0.5	0.0	-0.2	-3.4
Final, training period	-2.7	-0.5	-1.5	-0.2	-0.5	0.0	-1.5
Real: training vs. test period	-4.6	-1.8	-2.3	-0.6	0.0	-0.3	-2.0

Table 6.20: Error of the mean turnaround time (simulated minus observed, minutes) and its parts; the four parts add up to the TTT error up to rounding. The stop error is split into a count part (stops per visit by area, i.e. routing) and a duration part (mean stop duration by area). Last row: training log minus test log.

period cannot know this. The replayed demand captures the higher number of trucks, but not the slower service. Following Özdemir et al. [74], the error on the test period is therefore not only model error.

The model error. In both periods the routing is right and the stops are too short. The count of stops per area explains less than 0.2 minutes of the error per visit; the durations explain the rest. Part of this is how the load is distributed. With three lanes, the simulation never has a fourth truck at a block, while the training log records one at 5.6 % of the RMG stops, with a mean duration of 21 minutes; these recorded stops include the waiting for a lane (Section 5.9.3). The simulated stops are also slightly shorter in each load state (0.2 to 1.7 minutes on average). Shorter stops mean fewer trucks in the terminal (68 against 74 on average in training), which again means less load at the blocks. The TTT by trucks in the terminal shows the same weakness: up to 100 trucks the simulated median is within 3 minutes of the observed one, but with 150 or more trucks it is 39 to 43 minutes against 58 to 59. The simulation reproduces the congestion of normal hours, but not the worst hours. This is also why it has almost no delay-peak hours (1.4 % against 25.7 % in the test period).

6.5.8 Acceptance for the Intended Use

Run	1 Physical (max)	2a Spearman $\geq$ 0.49	2b Gaps $\leq$ 2.0 / 1.0 / 5.75	4 Level: CTD within
Conditional, test period	yes (3)	no (0.48)	no (1.2 / 1.3 / 6.2)	no (6.3)
Conditional, training period	yes (3)	no (0.46)	no (0.8 / 1.3 / 3.5)	weeks (2.7)
Generative, test period	yes (3)	no (0.43)	no (1.9 / 1.3 / 4.2)	no (9.9)
3 Extreme condition: TTT median / mean at arrival intensity 0.8, 1.0, 1.2				
Conditional, test period	27.0 / 32.8 → 28.4 / 34.9 → 33.2 / 42.7 min: rises: yes			

Table 6.21: Acceptance criteria of Section 5.9.4 for the final configuration. Thresholds from the differences between real periods: Spearman of the weakest test week; profile gaps (approach / stop duration / TTT by trucks in the terminal, minutes) of the largest difference between weeks; CTD within the differences between weeks (3.1 minutes) or between periods (5.5 minutes).

Table 6.21 applies the criteria that were fixed before the results were read. The final configuration is physically admissible: no rule violations and never more than three trucks at a block. It passes the extreme condition. It reproduces the rise of the approach with the block load within the variation between real weeks, in both periods. It fails the other parts of the load-response criterion narrowly: the Spearman correlation is 0.48 on the test period and 0.46 on the train-

ing period (threshold 0.49), the stop-duration gap is 1.3 minutes (threshold 1.0), and on the test period the gap of the TTT by trucks in the terminal is 6.2 minutes (threshold 5.75). The level criterion holds on the training period (2.7 minutes, within the variation between weeks), but not on the test period (6.3 minutes) or in the fully generative run (9.9 minutes).

By the rule of Section 5.9.4, E5 is admissible only if criteria 1 to 3 hold under replay. Criterion 2 does not hold. The model reproduces the mechanism that E5 acts on, the waiting for a lane at a busy block, but it underestimates how much the stops slow down at the busiest blocks and hours. An effect of moving trucks away from these blocks would therefore be underestimated rather than overestimated.

6.6 Answer to RQ2

To explain:

- Short answer: with the observed trucks and blocks replayed and four changes to ProSiT's queue and service, the simulation reproduces the turnaround time to within 5 minutes at the median (in-sample within 1 minute) and the rise of the approach with the block load. It underestimates the slowdown at the busiest blocks and hours. The error of the E3 baseline came from the block choice; the remaining errors came from the queue and the service mechanisms. Justified by E4 (Tables 6.15 to 6.21).

RQ2 asks to what extent the ProSiT simulation reproduces the turnaround time of the trucks and its response to load at the capacity-limited yard blocks, and which model components cause the remaining error.

To what extent. As fitted in E3, the simulation does not reproduce the turnaround time: the median is 58 instead of 33 minutes and the P90 141 instead of 75. With the observed trucks and blocks replayed and four changes to the simulation, it does to a large extent. The median is 28 and the P90 62 minutes on the test period, and 28 and 59 minutes against 29 and 64 on the training period, where the cycle-time distance (2.7 minutes) lies within the variation between two real weeks. The waiting before a stop rises with the number of trucks at the block, as at the terminal, and the turnaround time rises with the arrival intensity. Two things are not reproduced. The simulation underestimates the slowdown at the busiest blocks and hours, so it has almost no delay-peak hours. And no model learned on the training period reproduces which blocks are busy in the test period, nor the slower service of that period.

Which components. The components are listed in the order in which a truck meets them.

- **Demand.** Not a cause of the E3 error: replaying the exact arrivals changes the median TTT by 3 minutes. ProSiT's arrival model has no daily peaks, which matters when the simulation runs without replay.

- **Block choice.** The main cause of the E3 error. The queue feature counts the stop itself in training, so the learned rule sends trucks to the longest queue. ProSiT's count of the activities in progress, which is too low for 13 to 14 % of the RMG stops in the simulation, makes this concentration stronger. Even without the self-count, the block loads of a later period cannot be learned.

- **Queue at the block.** Two mechanisms prevent waiting under load: the capacity is checked at enabling, and the waiting time is measured as if a block had one server. With three lanes, capacity at start and the waiting measured from a free lane, the waiting arises where it arises at the terminal.

- **Service.** The execution time does not depend on the load. Adding the number of trucks at the block reproduces most of the observed slowdown, but not the slowest states.

- **Exit.** Not a cause: it does not depend on the load and is reproduced within half a minute.

The physical limit of the blocks connects these findings. A block serves at most three trucks. Every mechanism that ignores this limit, a capacity check at enabling or a single-server waiting target, produces either physically impossible states or no waiting at all. In the log, the limit is visible only indirectly, because the recorded stop interval includes the waiting for a lane.

Limits of the answer. The final configuration uses two oracles: the observed arrivals and the observed blocks. Without them, the error is larger (median 26 against 33 minutes, cycle-time distance 9.9 minutes). The load-response criterion is missed narrowly, so the model does not pass the pre-registered test for the intended use of E5. The answer holds for the turnaround time and its parts; other measures, such as the use of the cranes, were not examined.

6.7 Load-Aware Block Allocation (E5)

To explain:

- Only if E4b is passed. Otherwise: what the model would need to evaluate S1, and why the current evidence does not allow a quantitative statement.

6.7.1 Effect on the KPI

To explain:

- Paired differences (S1 minus baseline) for the three variants ±1, ±2, any block: delay-peak hours, TTT P90 and median, with 95 % intervals.

6.7.2 Mechanism and Distribution of Effects

To explain:

- Where the effect comes from: fewer stops at full blocks, shorter approach. Who benefits: deliveries, receives at relieved blocks, mixed visits.
- Costs: share of deliveries moved to another block; what is not modelled.

6.7.3 Sensitivity

To explain:

- Repeat with the occupancy effect halved (if a timing extension is used) and on hour-of-week arrivals. Does the conclusion hold?

6.7.4 Conclusion of E5

To explain:

- Conditional recommendation: under which eligibility assumption, with which effect size, and what the terminal must confirm. This is the answer to RQ3.

6.8 Chapter Summary

To explain:

- One paragraph per RQ with the key number. Hand over to the discussion.

7 Discussion

To explain:

- Bring the results together and judge them. No new results here.
- Separate clearly: what the evidence supports, what depends on assumptions, and what remains open.

7.1 Adequacy for the Intended Use

To explain:

- Which decisions can the final model support (and which not)? Validity is judged for turnaround time and delay peaks only.
- Where does a policy evaluation depend on uncertain assumptions (eligible blocks, occupancy effect, timing extension)?

7.2 Discovery Metrics and Simulation Behaviour

To explain:

- Do good discovery metrics predict a usable simulation? No: IMo had high fitness and precision but produced empty visits and parallel stops; the leaked block choice was invisible to every

discovery metric.

- What this means for model selection in simulation studies: validate per layer in the simulator, not only on the log.

7.3 Data-Aware Simulation in a Physical Logistics Process

To explain:

- What ProSiT represents well (routing, case attributes) and what it does not represent out of the box (load-dependent timing, block-bound resources, multi-server capacity).
- The version issue: model selection in 1.0.3 vs. 1.0.4; the queue-length feature; the capacity check. Formulate as findings about transferability, not as criticism of the tool.
- Historical context features vs. dynamic state: case-constant demand and utilisation values do not react to simulated load.

7.4 Threats to Validity

To explain:

- Construct validity: timestamp semantics (readiness to fulfilment includes waiting), possible double counting of waiting, the KPI definition by Gate-In hour.
- Internal validity: model choices made on validation data; criteria fixed in advance; retrospective hold-out.
- External validity: one terminal, eight weeks, the selected population (completed visits, known activities); transfer to other periods (validation was more congested).
- Conclusion validity: replications, intervals, associational vs. causal relations (occupancy and waiting).

7.5 Domain Knowledge and Stakeholder Review

To explain:

- Separate the author's domain knowledge from the documented external review (who, when, what was confirmed, what was not).

- What the review changed in the rules, the KPI target and the scenario design.

7.6 Implications for Practice

To explain:

- What the terminal can take from the results (bottleneck at the handover, conditional effect of S1) and what it would need before acting on them.

8 Conclusion

To explain:

- One page summary of what was done: data, model construction, layer-wise validation, scenario.

8.1 Answers to the Research Questions

To explain:

- One paragraph per RQ, in the pattern: the question in one sentence, the direct answer, the key evidence (one or two numbers), the qualification. Same RQ wording as in Chapter 1.
- RQ1: take the short answer of Section 6.4. Part 1: IM 0.0 restricted exactly to rules 3 and 5 as a state machine, plus the stop guard; the only sound net that accepts every observed sequence with precision clearly above a flower-like net (0.838, repaired 0.888); preserving rare behaviour is not overfitting (13 of 16 new test sequences accepted). Part 2: only partly; precision did not anticipate the simulation (same language and precision, but 15.1 % vs. 0 % visits without a stop), coverage did (IMf 0.3: lowest coverage, only clearly worse structure); the state-machine encoding decided.
- RQ2: take the short answer of Section 6.6. As fitted, no (median 58 vs. 33 min, P90 141 vs. 75); with replayed trucks and blocks and four changes (exact count, capacity at start with three lanes, lane-free waiting target, load-dependent service) to a large extent: median 28, P90 62 min on the test period, in-sample CTD 2.7 min (within the variation between weeks); approach rises with the block load as observed. Causes in the order of the truck: block choice (self-count in the queue

feature, amplified by the too low concurrency count) for the E3 error; capacity at enabling and single-server waiting for the missing waiting under load; no load in the service time. Qualification: underestimates the slowdown at the busiest blocks and hours; the load-response criterion is missed narrowly, so E5 is not admissible by the pre-registered rule; part of the test error is the slower service of the test period.

- RQ3: take the short answer of the E5 results. What the simulation can and cannot say about load-aware block allocation for deliveries: direction and mechanism only if robust in all four robustness runs; effect sizes as model values under the conditions of E4; no recommendation for the terminal. Frame it as a demonstration of what-if use of process mining based simulation under physical constraints, with its conditions of validity.

8.2 Contributions

To explain:

- Restate the contributions from Chapter 1, now with the evidence that supports each.

8.3 Limitations

To explain:

- The three or four most important limitations, short (details in Chapter 7).

8.4 Future Work

Future work should retain the reproducible discovery-and-validation protocol while extending the mechanisms and evidence that currently limit physical and counterfactual interpretation:

- **Investigate an object-centric process representation.** Future work could examine whether an object-centric event log is better suited to container-terminal operations than the current truck-visit case representation. In the present model, container characteristics are aggregated to case level, so information such as container type, loading status, reefer status, or hazardous-goods status is combined across all containers handled during one truck visit. An object-centric approach could keep each container as a separate object with its own attributes while linking it to the truck visit, handling activity, yard

block, and resource. This would reduce information loss and preserve container-specific information throughout the process and could also support multi-container and multi-block visits without forcing one aggregated context value onto the complete case.

- **Add an explicit spatial and movement representation.** Future work could separate physical movement from the combined pre-service delay by using route, position, or equipment observations. Rather than representing every movement as an additional process activity, a richer simulation formalism could include location directly in the model state, for example through timed or coloured Petri-net tokens and location-dependent transition conditions. This would allow truck or container movement between yard areas to consume time and respect spatial constraints without treating travel as a recorded handling activity. Movement times would then have to be estimated from spatial observations and the remaining access and waiting-time models refitted, so that travel time is not counted twice.

- **Represent physical assignment and queueing explicitly.** Future models could bind trucks and containers to physically feasible yard locations and resources instead of treating resource labels as fully interchangeable. Equipment availability and assignment should be represented as dynamic state, and trucks waiting for handling should enter explicit simulated queues rather than having these effects absorbed into a fitted pre-service-delay distribution. An object-centric representation could preserve the relations between trucks, containers, yard blocks, and equipment, while a corresponding simulation model would still need explicit movement, resource-allocation, and queueing rules [87, 88].

A

Observed Truck-Visit Behaviour

This appendix describes the recorded behaviour that the simulation must reproduce, in more detail than Section 3.3. All statements are descriptive associations. They do not establish causes. Thresholds are fitted on the training partition and applied unchanged to the validation partition. The test partition is not used.

A.1 Arrival Pattern

Trucks arrive in a stable weekly and daily pattern (Figure A.1). Arrivals take place from Monday to Friday, with few visits on Saturdays and none on Sundays. On weekdays, about 22 trucks per hour arrive between midnight and 5:00. The rate rises to about 118 per hour at 6:00 and stays on a plateau of 126 to 152 trucks per hour between 7:00 and 16:00. After 17:00 it falls again. The week around Easter (1 to 7 April) has a markedly lower volume.

An *arrival peak* is a Gate-In hour with at least 165 arrivals, the 90th percentile of all training hours with arrivals. Arrival peaks are short spikes on top of the daytime plateau. They make up 70 of 610 training hours with at least 20 visits (11.5 %) and 18 of 147 validation hours. The highest hourly count is 255 trucks, in the validation period.

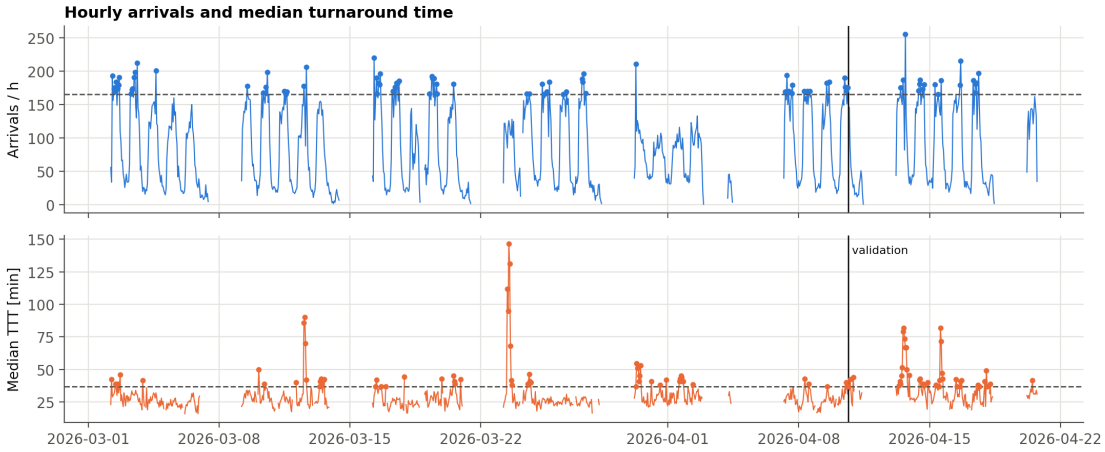


Figure A.1: Hourly arrivals (top) and median turnaround time by Gate-In hour (bottom) for the training and validation periods. Dots mark arrival peaks and delay peaks. Dashed lines show the thresholds of 165 arrivals and 37 minutes. The vertical line marks the start of the validation period.

A.2 Congestion over the Day

The turnaround time grows over the working day, although the arrival rate stays on its plateau. On training weekdays, the median TTT rises from 26 minutes for trucks arriving at 7:00 to 34 minutes at 14:00. The P90 rises from 57 to 81 minutes at 13:00. The mean number of trucks already inside the terminal at Gate In grows from 69 at 7:00 to 106 at 15:00. Figure A.2 shows that this growth comes mainly from the approach and the exit. The stop intervals change comparatively little over the day.

The turnaround time increases with the number of trucks in the terminal, and the increase is non-linear (Figure A.3). Up to about 100 trucks, the median TTT rises slowly from 23 to 34 minutes. Between 140 and 160 trucks, the median reaches 43 minutes and the P90 about 100 minutes. Above 160 trucks, the P90 exceeds two hours. Such occupancy levels are rare and occur on a few days only. The Spearman correlation between hourly arrivals and the median TTT of the same hour is 0.51. It is 0.53 for the arrivals of the preceding three hours and drops to 0.05 for a lag of six hours.

The same pattern appears at the handling units (Figure A.4). At an RMG block, the median stop interval is 5 minutes when the truck is alone at the block. It is 9, 13 and 18 minutes when two, three or four trucks have an open interval at the same block. The Spearman correlation between unit occupancy and stop duration is 0.49 for RMG, 0.45 for LL and 0.32 for HO2. Up to four concurrent intervals occur at an RMG block, although a block has three handover

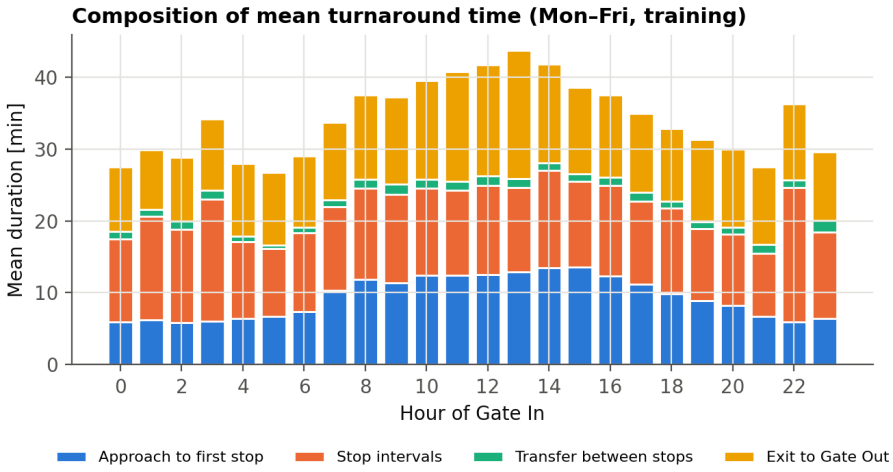


Figure A.2: Mean turnaround time by Gate-In hour, split into approach, stop intervals, transfer and exit. Training weekdays.

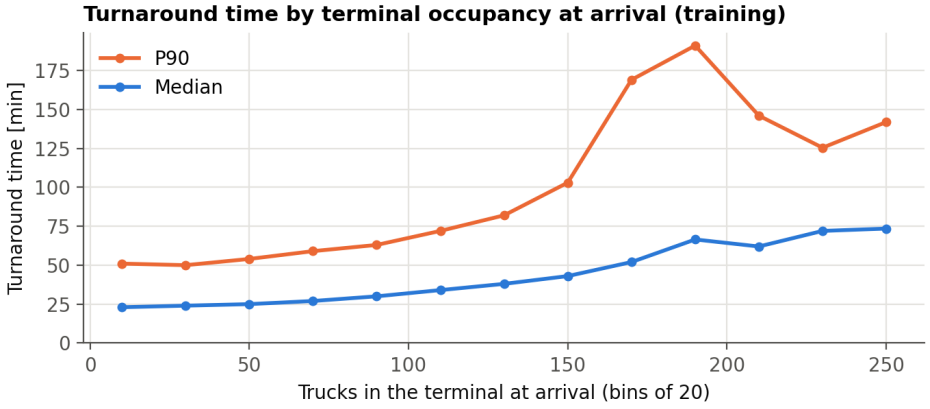


Figure A.3: Median and P90 of the turnaround time by the number of trucks in the terminal at Gate In, in classes of 20. Training, classes with at least 50 visits.

lanes and serves at most three trucks at once. A fourth interval starts at 5.6 % of the RMG stops. It is short: the first of the other three intervals ends after a median of 3 minutes. The fourth truck is therefore already registered at the block while it waits for a lane, and the recorded stop interval includes this waiting (source: simulation_e3/analyse_block_overlap.py). At LL and HO2, more than ten concurrent intervals occur. This confirms that the timestamps of these areas do not represent physical occupancy.

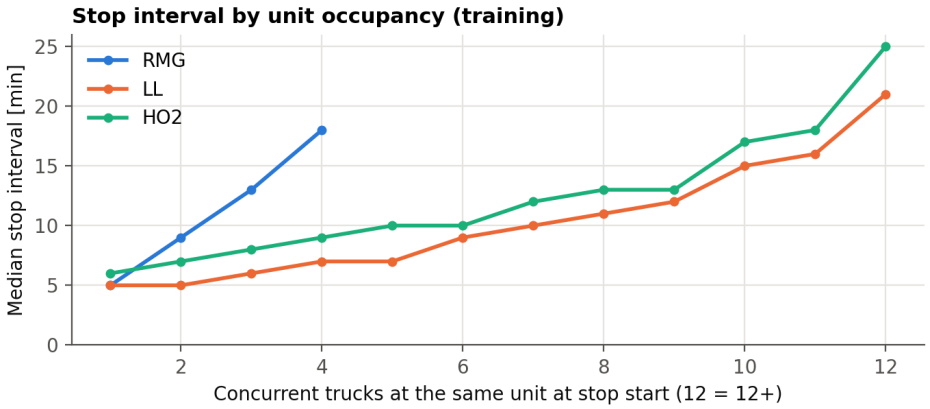


Figure A.4: Median stop interval by the number of trucks with an open interval at the same handling unit when the stop starts. Training.

A.3 Delay Peaks

A *delay peak* is a Gate-In hour with at least 20 visits whose median TTT is at least 37 minutes, the 90th percentile of the training hours. Delay peaks are a different phenomenon from arrival peaks. Training contains 62 delay-peak hours (10.2 %) and validation 39 (26.5 %). Only nine delay-peak hours in each partition are also arrival-peak hours.

Table A.1 decomposes the turnaround time in and outside delay peaks. In training, the mean TTT in delay-peak hours is 24.4 minutes longer. Approach and exit account for 21.4 minutes of this difference, or 88 %. The stop intervals grow by only 2.5 minutes. Arrival peaks, in contrast, are associated with an increase of only 2.4 minutes in mean TTT.

The exit share is driven by two disruption days (Section A.4). Without 12 and 23 March, the mean TTT in delay-peak hours is 17.0 minutes longer. The approach accounts for 10.1 minutes (59 %), the stop intervals for 3.3 minutes (19 %) and the exit for 3.1 minutes (18 %). Under normal

operation, the additional delay therefore arises mainly before the truck reports ready at its first stop.

Training, mean [min]	Approach	Stops	Transfer	Exit	TTT
Delay-peak hours (7,273 visits)	19.3	14.2	1.6	22.9	57.9
Other hours (50,989 visits)	9.8	11.7	1.1	11.0	33.5
Difference	+9.5	+2.5	+0.5	+11.9	+24.4
Arrival-peak hours (12,732 visits)	12.0	12.4	1.2	12.9	38.4
Other hours (45,530 visits)	10.7	11.9	1.1	12.3	36.0

Table A.1: Turnaround-time components in and outside peak hours. Visits are assigned to the hour of their Gate In.

Not all visits are affected equally. In arrival peaks, the median TTT of receive visits rises from 28 to 34 minutes, that of delivery visits only from 23 to 25 minutes. Multiblock visits rise from 45 to 52 minutes. Visits without RMG are almost unaffected (27 to 28 minutes). Among the RMG blocks, T18 shows the largest increase of the median stop interval, from 10 to 17 minutes. Most other blocks increase by at most three minutes, and HO2 stops are not longer in arrival peaks.

A.4 Disruption Episodes

A few delay peaks follow a different pattern. On Monday, 23 March 2026, the hourly median TTT rose to between 112 and 146 minutes from about 9:30, while arrivals stayed at a normal 85 to 126 trucks per hour. Up to 306 trucks were inside the terminal. The additional time lies almost entirely in the exit, with approach and stop intervals at normal values. Similar but shorter episodes occur on 12 March and on 13 April (validation). Possible explanations include a disruption at the exit gate or in exit processing, or an artefact of the Gate Out recording. Such episodes are not driven by demand. A simulation learned from normal operations cannot be expected to reproduce them. They are therefore identified by a rule fixed in advance and reported separately, but they are never removed from the evaluation.

A.5 Differences between the Partitions

The validation period is more congested than the training period (Table A.2). Its daily median TTT is 32 minutes against 29, and the share of delay-peak hours is 26.5 % against 10.2 %. Visits

without RMG show the largest shift, with a median TTT of 39 against 27 minutes. The mix of visits shifts as well: the validation period has more deliveries, more visits without RMG and more visits with two or more stops.

Measure	Training	Validation
Visits	58,262	14,625
TTT mean / median / P90 [min]	36.5 / 28.5 / 64	42.2 / 34 / 78
Daily median TTT [min]	29	32
Mean approach / exit [min]	10.9 / 12.5	13.2 / 13.6
Delay-peak hours (of operating hours)	10.2 %	26.5 %
Visits without RMG: share, median TTT	22.5 %, 27 min	27.1 %, 39 min
Delivery / receive / mixed visits	38.6 / 48.2 / 13.2 %	42.2 / 43.4 / 14.5 %
Visits with two or more stops	8.7 %	10.2 %

Table A.2: Observed visits per partition. Delay-peak threshold (37 minutes) fixed on the training period.

The shift affects the frequencies of activity sequences rather than their set. Only six validation visits (0.04 %) follow a sequence that does not occur in training, each a different one. The control-flow log distance between equal-size samples of training visits and the validation log is 0.024 to 0.027, and 0.036 for the last 14,625 training visits. The distance between their turnaround-time distributions is 6.3 to 6.6 minutes (Section 6.3 uses the same measures for the generated logs).

A model fitted on the training period is therefore expected to underestimate the validation TTT, independently of its control flow. Errors measured on the validation period mix model error and drift. Two references separate them. A replay of the training period with the fitted model measures the model error without drift. The distance between two real periods shows how close any model fitted on one period can be to another. In addition, model variants are compared with each other on the same period.

A.6 Bottlenecks at the Handover

To explain:

- Evidence used by scenario S1: approach to the first RMG stop by trucks already at the block at Gate In (median 6, 6, 9, 14, 20 min for 0–4 trucks); stop duration growth by block (T18 6 to 19 min); in 11 % of arrivals the target block is full while a median of 14 other blocks have at most one

truck; exit gate not a bottleneck in normal operation (source: peak_analysis/bottleneck_20261009, scenario_evidence.py).

B

Conformance of All Discovered Nets

Table B.1 lists the values of every net discovered in E1 on the training log and the test log of the split at 13 April 2026 (Section 5.2). The measures are defined in Section 5.6.3. The selection used the training values only (Section 6.1).

Net	Sound	P / T (silent)	Fitting visits [%] train	Fitting visits [%] test	Log fitness train	Log fitness test	Precision train	Precision test	Gen. train	Seq. accepted	Unseen test seq.
IM 0.0	yes	28 / 38 (27)	100.00	99.99	1.000	1.000	0.838	0.838	0.946	82	13
IMf 0.1	yes	10 / 17 (6)	95.02	92.69	0.990	0.985	0.914	0.913	0.980	14	1
IMf 0.2	yes	7 / 15 (4)	95.56	93.36	0.991	0.987	0.897	0.898	0.978	15	2
IMf 0.3	yes	11 / 18 (7)	96.50	94.86	0.993	0.990	0.906	0.902	0.981	19	2
IMf 0.4	yes	11 / 17 (6)	68.46	62.85	0.943	0.934	0.982	0.975	0.967	7	1
HM 0.7	yes	22 / 45 (34)	99.44	99.14	0.999	0.999	0.998	0.995	0.921	38	2
HM 0.8	yes	22 / 42 (31)	98.33	97.75	0.997	0.996	0.999	0.997	0.932	32	0
HM 0.9	yes	22 / 38 (27)	98.02	97.48	0.997	0.996	1.000	0.997	0.949	27	0
HM 0.9 / fo.1	yes	10 / 11 (6)	71.53	67.88	0.920	0.912	1.000	1.000	0.992	3	0
HM 1.0	yes	10 / 14 (9)	68.98	63.98	0.911	0.898	1.000	0.999	0.961	8	0
ILP 0.00	yes	6 / 5 (2)	40.82	33.39	0.814	0.789	1.000	1.000	0.996	1	0
ILP 0.25	yes	6 / 5 (2)	40.82	33.39	0.814	0.789	1.000	1.000	0.996	1	0
ILP 0.55	yes	6 / 6 (2)	63.85	57.39	0.893	0.874	1.000	1.000	0.995	2	0
ILP 0.75	yes	6 / 7 (2)	78.02	76.13	0.956	0.953	0.855	0.840	0.994	21	2
ILP 0.95	yes	11 / 11 (2)	99.03	98.42	0.998	0.997	0.838	0.828	0.989	75	10
ILP 1.00	yes	15 / 13 (2)	100.00	100.00	1.000	1.000	0.773	0.780	0.976	82	16
Alpha	no	4 / 11 (0)	15.75	18.98	0.862	0.867	0.633	0.643	0.973	41	11
Alpha+	no	5 / 11 (0)	15.75	18.98	0.862	0.867	0.718	0.729	0.973	41	11
Alpha++	no	22 / 11 (0)	final marking not reachable, no alignments								
Prefix tree	yes	169 / 249 (82)	100.00	99.95	1.000	1.000	1.000	0.988	0.509	82	0

Table B.1: Conformance of all discovered nets on the training log (2 March to 11 April 2026) and the test log (13 to 30 April 2026). Fitting visits: share of visits whose complete sequence the net accepts. Precision: escaping-edges precision on alignments with exact silent closure. Seq. accepted: training sequences accepted, of 82. Unseen test seq.: test sequences that do not occur in training and are accepted, of 16. Gen.: token-based generalisation on the training log; it is reported for completeness but not used for the selection, because it rewards nets that use few transitions often and barely separates the nets. Values of unsound nets are diagnostic only. The prefix tree is a reference that accepts exactly the training sequences.

C

ProSiT Model Inventory

To explain:

- The assessment explicitly asks for the concrete decision trees and all parameters needed to set up the ProSiT simulation. This appendix delivers them for the final model.

C.1 Discovery Settings and Versions

To explain:

- All hyperparameters, package versions, input hashes and run folders of the final model.

C.2 Learned Decision Trees

To explain:

- For each component (arrivals, routing per transition, resource choice per resource, waiting time per resource, execution time per activity): tree or single distribution, split features, thresholds, leaf distributions and sample sizes.
- Generate the tables automatically from prosit_params.pkl so that they are exact.

C.3 Calendars and Capacities

To explain:

- Arrival calendar, resource calendars, capacity per resource (discovered and after the cap of three).

C.4 Rule Register and Unobserved Sequences

To explain:

- The full rule register with status and sources, and the 19 two-stop sequences that the repaired model allows but the data never shows, with the terminal's assessment.

C.5 Discovered Models

To explain:

- Figures of all candidate nets of E1 on the expanded population (IMf 0.1/0.2, Heuristics, ILP, Split Miner) with their sizes.

D

Model Inspection and Reproduction

To explain:

- Most of this appendix describes the first version of the study (T22 closure, demand scenario, Train-80 bundles, old figure). Rewrite it as "Reproduction: repositories, run folders, commands, environments (ProSiT 1.0.3 and 1.0.4), input hashes, and which run produced which table.

This appendix separates visual process inspection from executable model state. The complete directly-follows graph exposes every observed training-log relation. The graph is descriptive. The discovered Petri net and fitted ProSiT parameters define the executable simulation model.

D.1 Unfiltered Training-Log Control Flow

Figure D.1 shows the unfiltered 59-arc directly-follows graph of the order-preserving training log. It is included for audit because the dominant-frequency view in Chapter 5 deliberately suppresses rare relations. The complete view confirms the fixed gate boundary and makes low-frequency repetitions and cross-activity relations inspectable. Its density also demonstrates why it is unsuitable as the primary explanatory figure. It is a descriptive view and should not be interpreted as a Petri net. In particular, a directly-follows arc records observed succession but does not encode choice, concurrency, markings, or firing rules.

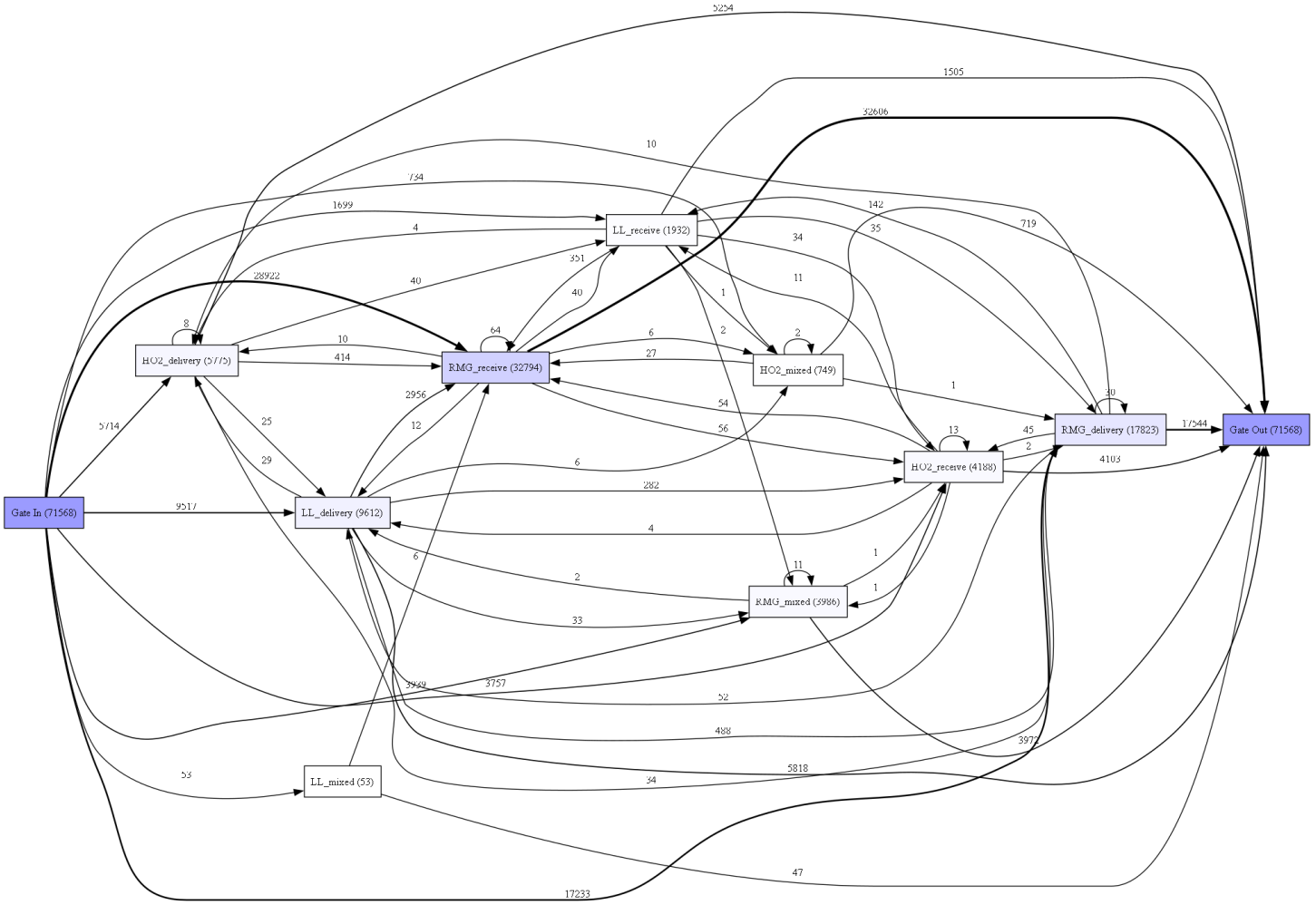


Figure D.1: Complete frequency directly-follows graph of the order-preserving CTB training log. Node labels report activity occurrences and arc labels report directly-follows counts. All 59 observed arcs are retained. The figure is a descriptive audit view and is not the executable simulation model.

D.2 Executable Model State

The baseline, T22-closure and increased-demand models are retained as separate serialized ProSiT parameter objects. The baseline contains the common Petri net, markings, fitted distributions, resources, capacities and calendars. The T22 scenario removes the block from the eligible RMG resource mappings and its availability calendar. The demand scenario changes only the empirical working-time inter-arrival sample. These state differences are described in Section 5.10. No source-code listing is required to interpret the interventions.

The public ProSiT save-and-load workflow serializes discovered parameters so that simulation can be repeated without rerunning discovery [76]. The project follows that workflow and additionally retains the exact Python objects used for the CTB experiments. A machine-readable round-trip audit covers the baseline, T22-closure and increased-demand bundles of the Train-80 *native-only* demand-mechanism configuration. Fitted rules, capacities, calendars, workload settings and empirical case-attribute distributions survive reconstruction. Each pickle additionally contains 40 timing or rule nodes with stored `sampled` arrays. This includes 71,530 calibrated inter-arrival values in the arrival node. None of these arrays is present immediately after `from_json()`, and the CTB-specific calibration metadata attached to the Python objects is not reconstructed either.

The JSON files consequently remain valid portable descriptions of the discovered parameters. Exact numerical reproduction instead uses the trusted pickles because they preserve the complete post-calibration in-memory state that produced the reported runs. This distinction is specific to the audited CTB bundles and software version and is not a general criticism of ProSiT's JSON interface. Pickles must only be loaded from this trusted project state and with the recorded environment.

The serialized objects and provenance records are retained in the research project and can be supplied separately if executable reproduction is requested. They are not required to follow the methodological argument or the result tables in the thesis.

References

[1] W. Y. Yun and Y. S. Choi, "A simulation model for container-terminal operation analysis using an object-oriented approach," *International Journal of Production Economics*, vol. 59, no. 1, pp. 221–230, 1999. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0925527398002138

[2] D. Steenken, S. Voss, and R. Stahlbock, "Container terminal operation and operations research - a classification and literature review," *OR Spectrum*, vol. 26, pp. 3–49, 01 2004.

[3] B. Weerasinghe, N. Perera, and X. Bai, "Optimizing container terminal operations: a systematic review of operations research applications," *Maritime Economics and Logistics*, vol. 26, pp. 1–35, 02 2023.

[4] D. Bett, I. Ali, M. Gheith, and A. Eltawil, "Simulation-based optimization of truck appointment systems in container terminals: A dual transactions approach with improved congestion factor representation," *Logistics*, vol. 8, p. 80, 08 2024.

[5] A. Rozinat, R. Mans, M. Song, and W. Aalst, "Discovering simulation models," *Information Systems*, vol. 34, pp. 305–327, 05 2009.

[6] V. Leno, M. Dumas, F. Maggi, M. La Rosa, and A. Polyvyanyy, "Automated discovery of declarative process models with correlated data conditions," *Information Systems*, vol. 89, p. 101482, 12 2019.

[7] F. Mannhardt, S. J. J. Leemans, C. T. Schwanen, and M. de Leoni, "Modelling data-aware stochastic processes - discovery and conformance checking," in *Application and Theory of Petri Nets and Concurrency: 44th International Conference, PETRI NETS 2023, Lisbon, Portugal, June 25–30, 2023, Proceedings*. Berlin, Heidelberg: Springer-Verlag, 2023, p. 77–98. [Online]. Available: https://doi.org/10.1007/978-3-031-33620-1_5

[8] M. {de Leoni}, F. Vinci, S. Leemans, and F. Mannhardt, "Investigating the influence of data-aware process states on activity probabilities in simulation models: Does accuracy

improve?" in *Business Process Management*, ser. Lecture Notes in Computer Science (LNCS), C. {Di Francescomarino}, A. Burattin, C. Janiesch, and S. Sadiq, Eds. Germany: Springer, Sep. 2023, pp. 129–145, 21st International Conference on Business Process Management, BPM 2023, BPM 2023 ; Conference date: 11-09-2023 Through 15-09-2023.

[9] O. López-Pintado, S. Murashko, and M. Dumas, "Discovery and simulation of data-aware business processes," 2024. [Online]. Available: https://arxiv.org/abs/2408.13666

[10] F. Meneghello, C. Di Francescomarino, C. Ghidini, and M. Ronzani, "Runtime integration of machine learning and simulation for business processes: Time and decision mining predictions," *Information Systems*, vol. 128, p. 102472, 10 2025.

[11] F. Vinci, G. Park, W. M. P. van der Aalst, and M. de Leoni, "Reliable and configurable process simulations via probabilistic white-box models," in *Service-Oriented Computing: 23rd International Conference, ICSOC 2025, Shenzhen, China, December 1–4, 2025, Proceedings, Part II*, ser. Lecture Notes in Computer Science, vol. 16321. Springer Singapore, 2026, pp. 337–352.

[12] F. Vinci, D. Aloini, E. Benevento, A. Stefanini, F. Zen, and M. de Leoni, "Improving organizational processes in healthcare through simulation-driven resource allocation: Methodology and real-world case study," *Artificial Intelligence in Medicine*, vol. 176, p. 103391, 2026. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0933365726000436

[13] M. Camargo, M. Dumas, and O. González-Rojas, "Automated discovery of business process simulation models from event logs," *Decision Support Systems*, vol. 134, p. 113284, 03 2020.

[14] M. Gracia, J. Mar-Ortiz, C. Caballini, and S. Sacone, "A combined data mining – optimization approach to manage trucks operations in container terminals with the use of a tas: Application to an italian and a mexican port," *Transportation Research Part E Logistics and Transportation Review*, vol. 142, 08 2020.

[15] A. Bruzzone, F. Longo, L. Nicoletti, E. Bottani, and R. Montanari, "Simulation, analysis and optimization of container terminals processes," *International Journal of Modeling, Simulation, and Scientific Computing*, vol. 03, p. 1240006, 11 2012.

[16] D. Muravev, A. Rakhmangulov, H. Hu, and H. Zhou, "The introduction to system dynamics approach to operational efficiency and sustainability of dry port's main parameters," *Sustainability*, vol. 11, p. 2413, 04 2019.

[17] A. Carboni, F. Deflorio, C. Caballini, and S. Cangelosi, "Advances in terminal management: Simulation of vehicle traffic in container terminals," *Maritime Economics & Logistics*, vol. 27, no. 3, pp. 500–524, 2025. [Online]. Available: https://doi.org/10.1057/s41278-024-00300-5

[18] M. Kastner, N. Nellen, A. Schwientek, and C. Jahn, "Integrated simulation-based optimization of operational decisions at container terminals," *Algorithms*, vol. 14, 01 2021.

[19] S. Hartmann, *Generating scenarios for simulation and optimization of container terminal logistics*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2005, pp. 101–122. [Online]. Available: https://doi.org/10.1007/3-540-26686-0_4

[20] J. Alnahas, "Application of process mining in logistic processes of manufacturing organizations: A systematic review," *Sustainability*, vol. 15, no. 15, p. 11783, 2023.

[21] T. Becker and W. Intoyoad, "Context aware process mining in logistics," *Procedia CIRP*, vol. 63, pp. 557–562, 2017.

[22] M. Grobis and S. Ihlenfeldt, "Process mining for supporting data processing and process design in production and logistics," *Procedia CIRP*, vol. 136, pp. 468–473, 2025.

[23] ——, "Application of process mining to support simulation model development: A use case in production planning," *Procedia CIRP*, vol. 142, pp. 992–997, 2026.

[24] W. Van der Aalst, *Process Mining: Data Science in Action*, 01 2016.

[25] B. F. van Dongen, A. K. A. de Medeiros, H. M. W. Verbeek, A. J. M. M. Weijters, and W. M. P. van der Aalst, "The ProM framework: A new era in process mining tool support," in *Applications and Theory of Petri Nets 2005*, ser. Lecture Notes in Computer Science, vol. 3536. Springer, 2005, pp. 444–454.

[26] A. Berti, S. van Zelst, and D. Schuster, "PM4Py: A process mining library for Python," *Software Impacts*, vol. 17, p. 100556, 2023.

[27] W. M. P. van der Aalst, "Foundations of process discovery," in *Process Mining Handbook*, ser. Lecture Notes in Business Information Processing, W. M. P. van der Aalst and J. Carmona, Eds. Cham: Springer, 2022, vol. 448, pp. 37–75.

[28] T. Murata, "Petri nets: Properties, analysis and applications," *Proceedings of the IEEE*, vol. 77, no. 4, pp. 541–580, 1989.

[29] S. J. J. Leemans, D. Fahland, and W. M. P. van der Aalst, "Discovering block-structured process models from event logs containing infrequent behaviour," in *Business Process Management Workshops*, ser. Lecture Notes in Business Information Processing, vol. 171. Springer, 2014, pp. 66–78.

[30] W. M. P. van der Aalst, T. Weijters, and L. Maruster, "Workflow mining: Discovering process models from event logs," *IEEE Transactions on Knowledge and Data Engineering*, vol. 16, no. 9, pp. 1128–1142, 2004.

[31] A. K. A. de Medeiros, B. F. van Dongen, W. M. P. van der Aalst, and A. J. M. M. Weijters, "Process mining for ubiquitous mobile systems: An overview and a concrete algorithm," in *Ubiquitous Mobile Information and Collaboration Systems (UMICS 2004)*, ser. Lecture Notes in Computer Science, vol. 3272. Springer, 2004, pp. 151–165.

[32] L. Wen, W. M. P. van der Aalst, J. Wang, and J. Sun, "Mining process models with non-free-choice constructs," *Data Mining and Knowledge Discovery*, vol. 15, no. 2, pp. 145–180, 2007.

[33] A. Weijters, W. Aalst, and A. Medeiros, *Process Mining with the Heuristics Miner-algorithm*, 01 2006, vol. 166.

[34] A. J. M. M. Weijters and J. T. S. Ribeiro, "Flexible heuristics miner (FHM)," in *2011 IEEE Symposium on Computational Intelligence and Data Mining (CIDM)*. IEEE, 2011, pp. 310–317.

[35] F. Mannhardt, M. de Leoni, H. A. Reijers, and W. M. P. van der Aalst, "Data-driven process discovery – revealing conditional infrequent behavior from event logs," in *Advanced Information Systems Engineering (CAiSE 2017)*, ser. Lecture Notes in Computer Science, vol. 10253. Springer, 2017, pp. 545–560.

[36] F. Mannhardt, M. de Leoni, and H. A. Reijers, "Heuristic mining revamped: An interactive, data-aware, and conformance-aware miner," in *Proceedings of the BPM Demo Track and BPM Dissertation Award 2017*, ser. CEUR Workshop Proceedings, vol. 1920, 2017.

[37] S. J. van Zelst, B. F. van Dongen, W. M. P. van der Aalst, and H. M. W. Verbeek, "Discovering workflow nets using integer linear programming," *Computing*, vol. 100, pp. 529–556, 2018.

[38] A. Augusto, R. Conforti, M. Dumas, M. La Rosa, F. M. Maggi, A. Marrella, M. Mecella, and A. Soo, "Automated discovery of process models from event logs: Review and benchmark," *IEEE Transactions on Knowledge and Data Engineering*, vol. 31, no. 4, pp. 686–705, 2019.

[39] J.-R. Rehse, S. J. J. Leemans, P. Fettke, and J. M. E. M. van der Werf, "On process discovery experimentation: Addressing the need for research methodology in process discovery," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 1, 2024, publisher PDF dated December 2024; institutional issue record dated 2025.

[40] S. J. J. Leemans, D. Fahland, and W. M. P. van der Aalst, "Using life cycle information in process discovery," in *Business Process Management Workshops: BPM 2015, Revised Papers*, ser. Lecture Notes in Business Information Processing, vol. 256. Springer, 2016, pp. 204–217.

[41] C. Di Ciccio and M. Montali, "Declarative process specifications: Reasoning, discovery, monitoring," in *Process Mining Handbook*, ser. Lecture Notes in Business Information Processing, W. M. P. van der Aalst and J. Carmona, Eds. Cham: Springer, 2022, vol. 448, pp. 108–152.

[42] G. De Giacomo and M. Y. Vardi, "Linear temporal logic and linear dynamic logic on finite traces," in *Proceedings of the 23rd International Joint Conference on Artificial Intelligence (IJCAI 2013)*, F. Rossi, Ed. IJCAI/AAAI, 2013, pp. 854–860.

[43] F. M. Maggi, M. Montali, M. Westergaard, and W. M. P. van der Aalst, "Monitoring business constraints with linear temporal logic: An approach based on colored automata," in *Business Process Management (BPM 2011)*, ser. Lecture Notes in Computer Science, vol. 6896. Springer, 2011, pp. 132–147.

[44] J. Carmona, B. van Dongen, and M. Weidlich, "Conformance checking: Foundations, milestones and challenges," in *Process Mining Handbook*, ser. Lecture Notes in Business Information Processing, W. M. P. van der Aalst and J. Carmona, Eds. Cham: Springer, 2022, vol. 448, pp. 155–190.

[45] T. Slaats, D. M. M. Schunselaar, F. M. Maggi, and H. A. Reijers, "The semantics of hybrid process models," in *On the Move to Meaningful Internet Systems: OTM 2016 Conferences*, ser. Lecture Notes in Computer Science, vol. 10033. Cham: Springer, 2016, pp. 531–551.

[46] F. Mannhardt, M. de Leoni, H. A. Reijers, and W. M. P. van der Aalst, "Measuring the precision of multi-perspective process models," in *Business Process Management Workshops (BPM 2015)*, ser. Lecture Notes in Business Information Processing, vol. 256. Springer, 2016.

[47] D. Schuster, S. J. van Zelst, and W. M. P. van der Aalst, "Utilizing domain knowledge in data-driven process discovery: A literature review," *Computers in Industry*, vol. 137, p. 103612, 2022.

[48] G. Greco, A. Guzzo, F. Lupia, and L. Pontieri, "Process discovery under precedence constraints," *ACM Transactions on Knowledge Discovery from Data*, vol. 9, no. 4, pp. 32:1–32:39, 2015.

[49] A. Norouzifar, M. Dees, and W. M. P. van der Aalst, "Rule-guided process discovery," *Data & Knowledge Engineering*, vol. 161, p. 102508, 2026.

[50] P. M. Dixit, J. C. A. M. Buijs, W. M. P. van der Aalst, B. F. A. Hompes, and H. Buurman, "Enhancing process mining results using domain knowledge," in *Proceedings of the 5th International Symposium on Data-Driven Process Discovery and Analysis (SIMPDA 2015)*, ser. CEUR Workshop Proceedings, P. Ceravolo and S. Rinderle-Ma, Eds., vol. 1527. CEUR-WS.org, 2015, pp. 79–94. [Online]. Available: https://ceur-ws.org/Vol-1527/paper6.pdf

[51] K. Revoredo, "On the use of domain knowledge for process model repair," *Software and Systems Modeling*, 2023.

[52] D. Fahland and W. M. P. van der Aalst, "Model repair – aligning process models to reality," *Information Systems*, vol. 47, pp. 220–243, 2015.

[53] M. de Leoni, "Foundations of process enhancement," in *Process Mining Handbook*, ser. Lecture Notes in Business Information Processing, W. M. P. van der Aalst and J. Carmona, Eds. Cham: Springer, 2022, vol. 448, pp. 243–273.

[54] A. Kalenkova, J. Carmona, A. Polyvyanyy, and M. La Rosa, "Automated repair of process models using non-local constraints," in *Application and Theory of Petri Nets and Concurrency (PETRI NETS 2020)*, R. Janicki, N. Sidorova, and T. Chatain, Eds. Cham: Springer, 2020, pp. 280–300.

[55] C. Rennert and W. M. P. van der Aalst, "Improving precision in process trees using subprocess tree logs," in *Process Mining Workshops*, J. De Smedt and P. Soffer, Eds. Cham: Springer Nature Switzerland, 2024, pp. 110–122.

[56] F. Vinci and M. de Leoni, "Balancing fitness and precision in process model repair: Framework formalization, assessment, and benefits for simulation," *Process Science*, vol. 2, no. 1, p. 3, 2025. [Online]. Available: https://doi.org/10.1007/s44311-025-00007-7

[57] W. Aalst, A. Adriansyah, and B. Dongen, "Replaying history on process models for conformance checking and performance analysis," *WIREs Data Mining and Knowledge Discovery*, vol. 2, pp. 182–192, 03 2012.

[58] J. C. A. M. Buijs, B. F. van Dongen, and W. M. P. van der Aalst, "Quality dimensions in process discovery: The importance of fitness, precision, generalization and simplicity," *International Journal of Cooperative Information Systems*, vol. 23, no. 1, p. 1440001, 2014.

[59] A. Adriansyah, B. F. van Dongen, and W. M. P. van der Aalst, "Conformance checking using cost-based fitness analysis," in *2011 IEEE 15th International Enterprise Distributed Object Computing Conference*. IEEE, 2011, pp. 55–64.

[60] A. Adriansyah, J. Munoz-Gama, J. Carmona, B. F. van Dongen, and W. M. P. van der Aalst, "Alignment based precision checking," in *Business Process Management Workshops (BPM 2012)*, ser. Lecture Notes in Business Information Processing, vol. 132. Springer, 2012.

[61] ——, "Measuring precision of modeled behavior," *Information Systems and e-Business Management*, vol. 13, no. 1, pp. 37–67, 2015.

[62] A. Polyvyanyy, A. Solti, M. Weidlich, C. Di Ciccio, and J. Mendling, "Monotone precision and recall measures for comparing executions and specifications of dynamic systems," *ACM Transactions on Software Engineering and Methodology*, vol. 29, no. 3, 2020.

[63] W. M. P. van der Aalst, "Business process simulation survival guide," in *Handbook on Business Process Management 1: Introduction, Methods, and Information Systems*, 2nd ed., J. vom Brocke and M. Rosemann, Eds. Berlin, Heidelberg: Springer, 2015, pp. 337–370.

[64] A. M. Law, *Simulation Modeling and Analysis*, 5th ed. New York: McGraw-Hill Education, 2015.

[65] D. Chapela-Campa, O. López-Pintado, I. Suvorau, and M. Dumas, "SIMOD: Automated discovery of business process simulation models," *SoftwareX*, vol. 30, p. 102157, 2025.

[66] O. López-Pintado and M. Dumas, "Business process simulation with differentiated resources: Does it make a difference?" in *Business Process Management: 20th International Conference, BPM 2022, Münster, Germany, September 11–16, 2022, Proceedings*, ser. Lecture Notes in Computer Science, vol. 13420. Cham: Springer, 2022, pp. 361–378.

[67] O. López-Pintado, I. Halenok, and M. Dumas, "Prosimos: Discovering and simulating business processes with differentiated resources," in *Enterprise Design, Operations, and Computing. EDOC 2022 Workshops*, ser. Lecture Notes in Business Information Processing, vol. 466. Springer, 2023, pp. 346–352.

[68] A. Rogge-Solti, W. M. P. van der Aalst, and M. Weske, "Discovering stochastic petri nets with arbitrary delay distributions from event logs," in *Business Process Management Workshops: BPM 2013 International Workshops, Beijing, China, August 26, 2013, Revised Papers*, ser. Lecture Notes in Business Information Processing, vol. 171. Springer, 2014, pp. 15–27.

[69] A. Burke, S. Leemans, and M. Wynn, "Stochastic process discovery by weight estimation," in *Business Process Management Workshops*, ser. Lecture Notes in Business Information Processing, vol. 406, 2021, pp. 260–272.

[70] M. Camargo, M. Dumas, and O. González-Rojas, "Discovering generative models from event logs: data-driven simulation vs deep learning," *PeerJ Computer Science*, vol. 7, p. e577, 07 2021.

[71] R. G. Sargent, "Verification and validation of simulation models," *Journal of Simulation*, vol. 7, no. 1, pp. 12–24, 2013.

[72] D. Chapela-Campa, I. Benchekroun, O. Baron, M. Dumas, D. Krass, and A. Senderovich, "Can i trust my simulation model? measuring the quality of business process simulation models," in *Business Process Management*, ser. Lecture Notes in Computer Science, vol. 14159. Cham: Springer, 2023, pp. 20–37.

[73] ——, "A framework for measuring the quality of business process simulation models," *Information Systems*, vol. 127, p. 102447, 01 2025.

[74] K. Özdemir, L. Kirchdorfer, K. Amiri Elyasi, H. van der Aa, and H. Stuckenschmidt, "Rethinking business process simulation: A utility-based evaluation framework," 2025, arXiv preprint.

[75] E. Sulis, L. Genga, and N. Martin, "Simulation and process mining: Review and outlook," *International Journal of Data Science and Analytics*, vol. 22, p. 198, 2026.

[76] ProSiT Project, "Prosit: Process simulation tool—save and load parameters," GitHub repository documentation, 2026, accessed 30 August 2026. [Online]. Available: https://github.com/franvinci/prosit#save-and-load-parameters

[77] T. Gneiting and A. E. Raftery, "Strictly proper scoring rules, prediction, and estimation," *Journal of the American Statistical Association*, vol. 102, no. 477, pp. 359–378, 2007.

[78] Hamburger Hafen und Logistik AG, "Geschäftsbericht 2025," Hamburger Hafen und Logistik AG, Hamburg, Tech. Rep., 2026. [Online]. Available: https://bericht.hhla.de/geschaeftsbericht-2025/_assets/downloads/HHLA_2025_Geschaeftsbericht.pdf

[79] P. Felli, M. de Leoni, and M. Montali, "Soundness verification of data-aware process models with variable-to-variable conditions," *Fundamenta Informaticae*, vol. 182, no. 1, 2021.

[80] R. de la Briandais, "File searching using variable length keys," in *Proceedings of the Western Joint Computer Conference*. ACM, 1959, pp. 295–298.

[81] H. M. W. Verbeek, T. Basten, and W. M. P. van der Aalst, "Diagnosing workflow processes using Woflan," *The Computer Journal*, vol. 44, no. 4, pp. 246–279, 2001.

[82] F. Taymouri and J. Carmona, "Computing alignments of well-formed process models using local search," *ACM Transactions on Software Engineering and Methodology*, vol. 29, no. 3, pp. 15:1–15:41, 2020.

[83] N. Russell, A. H. M. ter Hofstede, W. M. P. van der Aalst, and N. Mulyar, "Workflow control-flow patterns: A revised view," BPMcenter.org, BPM Center Report BPM-06-22, 2006.

[84] D. Chapela-Campa and M. Dumas, "Enhancing business process simulation models with extraneous activity delays," *Information Systems*, vol. 122, p. 102346, 2024.

[85] B. F. van Dongen, "Efficiently computing alignments: Using the extended marking equation," in *Business Process Management (BPM 2018)*, ser. Lecture Notes in Computer Science, vol. 11080. Springer, 2018.

[86] ——, "Efficiently computing alignments: Algorithm and datastructures," 2018.

[87] W. M. P. van der Aalst, "Object-centric process mining: Unraveling the fabric of real processes," *Mathematics*, vol. 11, no. 12, p. 2691, 2023.

[88] B. Knopp, M. Pourbafrani, and W. M. P. van der Aalst, "Discovering object-centric process simulation models," in *2023 5th International Conference on Process Mining (ICPM)*. IEEE, 2023, pp. 81–88.